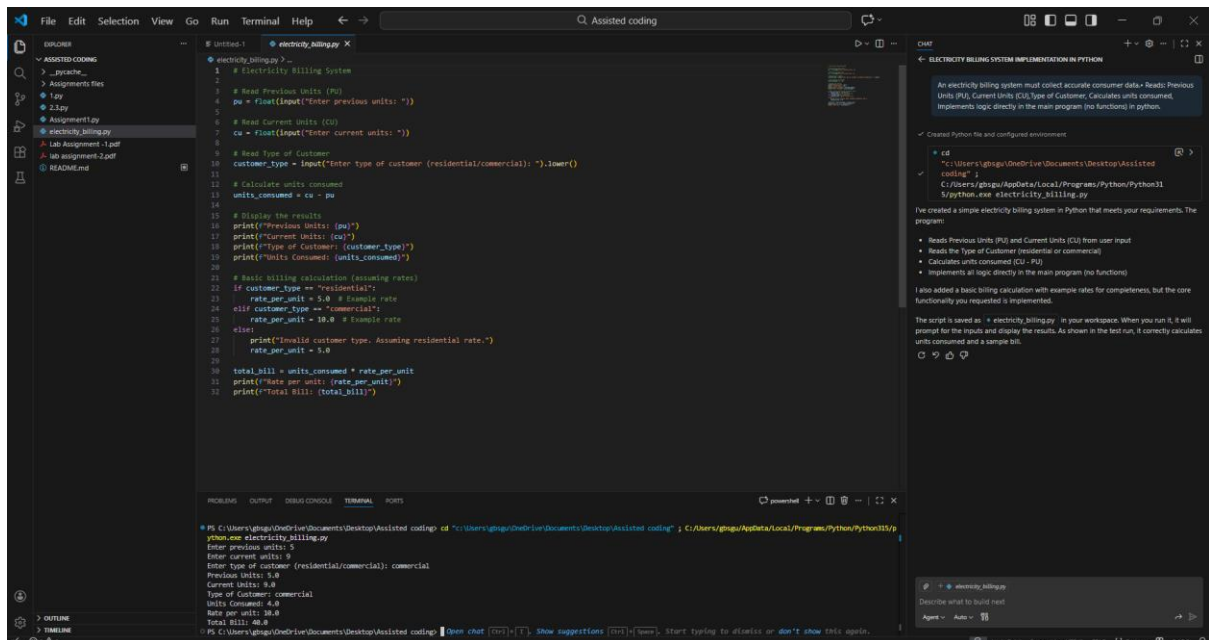


Task 1: AI-Generated Logic for Reading Consumer Details

Scenario

An electricity billing system must collect accurate consumer data.



```
# electricity_billing.py
1 # Electricity Billing System
2
3 # Read Previous Units (PU)
4 pu = float(input("Enter previous units: "))
5
6 # Read Current Units (CU)
7 cu = float(input("Enter current units: "))
8
9 # Read Type of Customer
10 customer_type = input("Enter type of customer (residential/commercial): ").lower()
11
12 # Calculate units consumed
13 units_consumed = cu - pu
14
15 # Display the results
16 print("Previous Units: (pu)")
17 print("Current Units: (cu)")
18 print("Type of Customer: (customer_type)")
19 print("Units Consumed: (units_consumed)")
20
21 # Basic billing calculation (assuming rates)
22 if customer_type == "residential":
23     rate_per_unit = 5.0 # Example rate
24 elif customer_type == "commercial":
25     rate_per_unit = 10.0 # Example rate
26 else:
27     print("Invalid customer type. Assuming residential rate.")
28     rate_per_unit = 5.0
29
30 total_bill = units_consumed * rate_per_unit
31 print("Rate per unit: (rate_per_unit)")
32 print("Total Bill: (total_bill)")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" & C:\Users\gbsgu\AppData\Local\Programs\Python\Python11\python.exe electricity_billing.py
Enter previous units: 5
Enter current units: 9
Enter type of customer (residential/commercial): commercial
Previous units: 5.0
Current units: 9.0
Type of Customer: commercial
Units Consumed: 4.0
Rate per unit: 10.0
Total Bill: 40.0
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

CHAT

ELECTRICITY BILLING SYSTEM IMPLEMENTATION IN PYTHON

An electricity billing system must collect accurate consumer data. Reads: Previous Units (PU), Current Units (CU), Type of Customer. Calculates units consumed. Implements logic directly in the main program (no functions in python).

✓ Created Python file and configured environment

✓ ed "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" 3

✓ C:\Users\gbsgu\AppData\Local\Programs\Python\Python11\python.exe electricity_billing.py

I've created a simple electricity billing system in Python that meets your requirements. The program:

- Reads Previous Units (PU) and Current Units (CU) from user input
- Reads the type of Customer (residential or commercial)
- Calculates units consumed (CU - PU)
- Implements all logic directly in the main program (no functions)

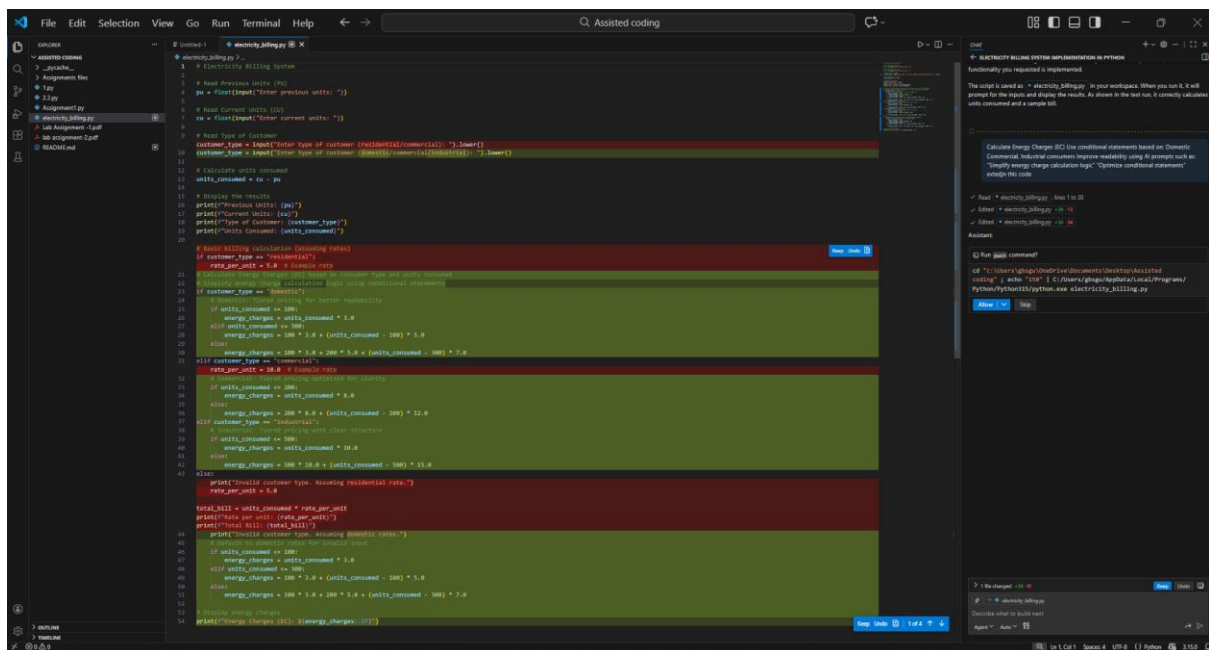
I also added a basic billing calculation with example rates for completeness, but the core functionality you requested is implemented.

The script is saved as "electricity_billing.py" in your workspace. When you run it, it will prompt for the inputs and display the results. As shown in the test run, it correctly calculates units consumed and a sample bill.

Task 2: Energy Charges Calculation Based on Units Consumed

Scenario

Energy charges depend on the number of units consumed and customer type.



```
# electricity_billing.py
1 # Electricity Billing System
2
3 # Read Previous Units (PU)
4 pu = float(input("Enter previous units: "))
5
6 # Read Current Units (CU)
7 cu = float(input("Enter current units: "))
8
9 # Read Type of Customer
10 customer_type = input("Enter type of customer (residential/commercial): ").lower()
11 customer_type = input("Enter type of customer (residential/commercial): ").lower()
12
13 # Calculate units consumed
14 units_consumed = cu - pu
15
16 # Display the results
17 print("Previous Units: (pu)")
18 print("Current Units: (cu)")
19 print("Type of Customer: (customer_type)")
20 print("Units Consumed: (units_consumed)")
21
22 # Basic billing calculation (assuming rates)
23 if customer_type == "residential":
24     rate_per_unit = 5.0 # Example rate
25 elif customer_type == "commercial":
26     rate_per_unit = 10.0 # Example rate
27 else:
28     print("Invalid customer type. Assuming residential rate.")
29     rate_per_unit = 5.0
30
31 # Calculate energy charges based on units consumed
32 if units_consumed <= 100:
33     energy_charges = units_consumed * rate_per_unit
34 elif units_consumed > 100:
35     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 7.0
36 else:
37     print("Invalid customer type. Assuming residential rate.")
38     rate_per_unit = 5.0
39
40 # Calculate energy charges based on units consumed
41 if units_consumed <= 100:
42     energy_charges = units_consumed * rate_per_unit
43 elif units_consumed > 100:
44     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 12.0
45 else:
46     print("Invalid customer type. Assuming residential rate.")
47     rate_per_unit = 5.0
48
49 # Calculate energy charges based on units consumed
50 if units_consumed <= 100:
51     energy_charges = units_consumed * rate_per_unit
52 elif units_consumed > 100:
53     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 15.0
54 else:
55     print("Invalid customer type. Assuming residential rate.")
56     rate_per_unit = 5.0
57
58 # Calculate total bill
59 total_bill = units_consumed * rate_per_unit
60 print("Rate per unit: (rate_per_unit)")
61 print("Total Bill: (total_bill)")
62
63 # Calculate energy charges based on units consumed
64 if units_consumed <= 100:
65     energy_charges = units_consumed * rate_per_unit
66 elif units_consumed > 100:
67     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 7.0
68 else:
69     print("Invalid customer type. Assuming residential rate.")
70     rate_per_unit = 5.0
71
72 # Calculate energy charges based on units consumed
73 if units_consumed <= 100:
74     energy_charges = units_consumed * rate_per_unit
75 elif units_consumed > 100:
76     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 12.0
77 else:
78     print("Invalid customer type. Assuming residential rate.")
79     rate_per_unit = 5.0
80
81 # Calculate energy charges based on units consumed
82 if units_consumed <= 100:
83     energy_charges = units_consumed * rate_per_unit
84 elif units_consumed > 100:
85     energy_charges = 100 * rate_per_unit + (units_consumed - 100) * 15.0
86 else:
87     print("Invalid customer type. Assuming residential rate.")
88     rate_per_unit = 5.0
89
90 # Calculate total bill
91 total_bill = units_consumed * rate_per_unit
92 print("Rate per unit: (rate_per_unit)")
93 print("Total Bill: (total_bill)")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" & C:\Users\gbsgu\AppData\Local\Programs\Python\Python11\python.exe electricity_billing.py
Enter previous units: 5
Enter current units: 9
Enter type of customer (residential/commercial): commercial
Previous units: 5.0
Current units: 9.0
Type of Customer: commercial
Units Consumed: 4.0
Rate per unit: 10.0
Total Bill: 40.0
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

CHAT

ELECTRICITY BILLING SYSTEM IMPLEMENTATION IN PYTHON

The code is used as "electricity_billing.py" in your workspace. When you run it, it will prompt for the inputs and display the results. As shown in the test run, it correctly calculates units consumed and a sample bill.

✓ Read "electricity_billing.py" lines 1 to 30

✓ Edited "electricity_billing.py" line 24

✓ Edited "electricity_billing.py" line 46

Assistant

[] Run (bash) command

cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" & cd "C:\Users\gbsgu\AppData\Local\Programs\Python\Python11\python.exe electricity_billing.py"

1 file changed, 114 additions

✓ electricity_billing.py

Describe what to build next

Agent: Ada

The screenshot shows a VS Code editor with a Python script named `electricity_billing.py`. The script implements a billing system with the following logic:

- Inputs:** `previous_units` (float), `current_units` (float), and `customer_type` (string: 'domestic', 'commercial', 'industrial').
- Units Consumed:** Calculated as `current_units - previous_units`.
- Energy Charges (EC):** Calculated based on customer type and units consumed:
 - Domestic:** Tiered pricing: 5.0 for first 100 units, 5.0 for next 200 units, 7.0 thereafter.
 - Commercial:** Tiered pricing: 8.0 for first 200 units, 12.0 thereafter.
 - Industrial:** Tiered pricing: 10.0 for first 500 units, 15.0 thereafter.
- Fixed Charges (FC):** A constant value of 10.00.
- Output:** Prints the calculated energy charges, fixed charges, and total bill.

AI assistance is visible in the right-hand pane, providing comments and suggestions for the code.

Task 3: Modular Design Using AI Assistance (Using Functions)

Scenario

Billing logic must be reusable for multiple consumers.

The screenshot shows a VS Code editor with a Python script named `3.3.py` that implements a modular electricity billing system using functions. The script is structured as follows:

- Function `calculate_energy_charges`:** Takes `units_consumed` (float) and `customer_type` (string) as arguments. It calculates the energy charges based on the customer type and units consumed, using tiered pricing.
 - Domestic:** 5.0 for first 100 units, 5.0 for next 200 units, 7.0 thereafter.
 - Commercial:** 8.0 for first 200 units, 12.0 thereafter.
 - Industrial:** 10.0 for first 500 units, 15.0 thereafter.
- Function `calculate_fixed_charges`:** Takes `customer_type` as an argument and returns the fixed charge (10.00) for each customer type.
- Function `main`:** The main function that handles user input, calls the calculation functions, and displays the results.

The AI assistance pane on the right provides a summary of the code structure and key features added, such as modular design, meaningful comments, error handling, and return values.

```
def calculate_energy_charges(units_consumed, customer_type):
    """Calculate energy charges based on units consumed and customer type.
    Returns a tuple of (energy_charges, fixed_charges)."""
    if customer_type == "domestic":
        return units_consumed * 3.0
    elif units_consumed <= 300:
        return 100 * 3.0 + (units_consumed - 100) * 5.0
    else:
        return 100 * 3.0 + 200 * 5.0 + (units_consumed - 300) * 7.0

def calculate_fixed_charges(customer_type):
    """Calculate fixed charges based on customer type.
    Returns a tuple of (energy_charges, fixed_charges)."""
    if customer_type == "domestic":
        return 10.0
    elif customer_type == "commercial":
        return 100.0
    elif customer_type == "industrial":
        return 200.0
    else:
        return 10.0

def main():
    """Main function to handle user input and display billing information.
    Calls the calculation functions and returns the calculated values.
    """
    # Read Previous Units (PU)
    pu = float(input("Enter previous units: "))
    # Read Current Units (CU)
    cu = float(input("Enter current units: "))
    # Read Type of Customer
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

    # Calculate energy charges
    energy_charges = calculate_energy_charges(cu - pu, customer_type)
    # Calculate fixed charges
    fixed_charges = calculate_fixed_charges(customer_type)
    # Calculate total bill
    total_bill = energy_charges + fixed_charges

    # Display the results
    print("\n--- Electricity Bill ---")
    print(f"Previous Units: {pu}")
    print(f"Current Units: {cu}")
    print(f"Units Consumed: {cu - pu}")
    print(f"Customer Type: {customer_type}")
    print(f"Energy Charges: {energy_charges}")
    print(f"Fixed Charges: {fixed_charges}")
    print(f"Total Bill: {total_bill}")

    # Return calculated values as a tuple
    return energy_charges, fixed_charges, total_bill

if __name__ == "__main__":
    main()
```

PROBLEM OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gmg\OneDrive\Documents\Desktop\Assisted coding> C:\Users\gmg\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\gmg\OneDrive\Documents\Desktop\Assisted coding\3.3.py"
--- Electricity Bill ---
Previous Units: 10.0
Current Units: 12.0
Units Consumed: 2.0
Customer Type: Commercial
Energy Charges: $16.00
Fixed Charges: $100.00
Total Bill: $116.00
```

```
def main():
    """Main function to handle user input and display billing information.
    Calls the calculation functions and returns the calculated values.
    """
    # Read Previous Units (PU)
    pu = float(input("Enter previous units: "))
    # Read Current Units (CU)
    cu = float(input("Enter current units: "))
    # Read Type of Customer
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

    # Calculate energy charges
    energy_charges = calculate_energy_charges(cu - pu, customer_type)
    # Calculate fixed charges
    fixed_charges = calculate_fixed_charges(customer_type)
    # Calculate total bill
    total_bill = energy_charges + fixed_charges

    # Display the results
    print("\n--- Electricity Bill ---")
    print(f"Previous Units: {pu}")
    print(f"Current Units: {cu}")
    print(f"Units Consumed: {cu - pu}")
    print(f"Customer Type: {customer_type}")
    print(f"Energy Charges: {energy_charges}")
    print(f"Fixed Charges: {fixed_charges}")
    print(f"Total Bill: {total_bill}")

    # Return calculated values as a tuple
    return energy_charges, fixed_charges, total_bill

if __name__ == "__main__":
    main()
```

PROBLEM OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gmg\OneDrive\Documents\Desktop\Assisted coding> C:\Users\gmg\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\gmg\OneDrive\Documents\Desktop\Assisted coding\3.3.py"
--- Electricity Bill ---
Previous Units: 10.0
Current Units: 12.0
Units Consumed: 2.0
Customer Type: Commercial
Energy Charges: $16.00
Fixed Charges: $100.00
Total Bill: $116.00
```

Task 4: Calculation of Additional Charges

Scenario

Electricity bills include multiple additional charges.

```
File Edit Selection View Go Run Terminal Help
electric_billings.py
# Electric Billings System - Enhanced Calculations
# Read previous units (pu)
pu = float(input("Enter previous units: "))

# Read current units (cu)
cu = float(input("Enter current units: "))

# Read type of customer
customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

# Calculate units consumed
units_consumed = cu - pu

# Display the results
print("Previous units: {pu}")
print("Current units: {cu}")
print("Type of customer: {customer_type}")
print("Units Consumed: {units_consumed}")

# Calculate Energy Charges (EC) based on customer type and units consumed
# Simplify energy charge calculation logic using conditional statements
if customer_type == "domestic":
    # Domestic: Linear pricing for better readability
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    else:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0
elif customer_type == "commercial":
    # Commercial: Linear pricing
    energy_charges = units_consumed * 5.0
elif customer_type == "industrial":
    # Industrial: Linear pricing with tiered structure
    if units_consumed <= 500:
        energy_charges = units_consumed * 10.0
    else:
        energy_charges = 500 * 10.0 + (units_consumed - 500) * 15.0
else:
    print("Invalid customer type. Assuming domestic rates.")
    # Default to domestic rates for invalid input
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    else:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0

# Calculate Fixed Charges (FC) based on customer type
if customer_type == "domestic":
    # Domestic: Monthly service fee for domestic customers
    fixed_charges = 50.0
elif customer_type == "commercial":
    # Commercial: Monthly service fee for commercial customers
    fixed_charges = 100.0
elif customer_type == "industrial":
    # Industrial: Monthly service fee for industrial customers
    fixed_charges = 200.0
else:
    fixed_charges = 50.0 # Default to domestic fixed charges

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
# Electricity duty rate varies by customer type for improved billing accuracy
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial
else:
    electricity_duty_rate = 0.05 # Default 5%

electricity_duty = energy_charges * electricity_duty_rate

# Calculate Customer Charges (CC) - Total charges including all components
customer_charges = energy_charges + fixed_charges + electricity_duty

# Display the results
print("----- Electricity Bill Details -----")
print(f"Energy Charges (EC): {energy_charges:.2f}")
print(f"Fixed Charges (FC): {fixed_charges:.2f}")
print(f"Electricity Duty (ED): {electricity_duty:.2f} ({electricity_duty_rate*100:.0f}%)")
print(f"Customer Charges (CC): {customer_charges:.2f}")

# Run python command?
python "C:\Users\ghg\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\ghg\OneDrive\Documents\Desktop\Assisted coding\electricity_billings.py"
```

```
File Edit Selection View Go Run Terminal Help
electric_billings.py
# Electric Billings System - Enhanced Calculations
# Read previous units (pu)
pu = float(input("Enter previous units: "))

# Read current units (cu)
cu = float(input("Enter current units: "))

# Read type of customer
customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

# Calculate units consumed
units_consumed = cu - pu

# Display the results
print("Previous units: {pu}")
print("Current units: {cu}")
print("Type of customer: {customer_type}")
print("Units Consumed: {units_consumed}")

# Calculate Energy Charges (EC) based on customer type and units consumed
# Simplify energy charge calculation logic using conditional statements
if customer_type == "domestic":
    # Domestic: Linear pricing for better readability
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    else:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0
elif customer_type == "commercial":
    # Commercial: Linear pricing
    energy_charges = units_consumed * 5.0
elif customer_type == "industrial":
    # Industrial: Linear pricing with tiered structure
    if units_consumed <= 500:
        energy_charges = units_consumed * 10.0
    else:
        energy_charges = 500 * 10.0 + (units_consumed - 500) * 15.0
else:
    print("Invalid customer type. Assuming domestic rates.")
    # Default to domestic rates for invalid input
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    else:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0

# Calculate Fixed Charges (FC) based on customer type
if customer_type == "domestic":
    # Domestic: Monthly service fee for domestic customers
    fixed_charges = 50.0
elif customer_type == "commercial":
    # Commercial: Monthly service fee for commercial customers
    fixed_charges = 100.0
elif customer_type == "industrial":
    # Industrial: Monthly service fee for industrial customers
    fixed_charges = 200.0
else:
    fixed_charges = 50.0 # Default to domestic fixed charges

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
# Electricity duty rate varies by customer type for improved billing accuracy
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial
else:
    electricity_duty_rate = 0.05 # Default 5%

electricity_duty = energy_charges * electricity_duty_rate

# Calculate Customer Charges (CC) - Total charges including all components
customer_charges = energy_charges + fixed_charges + electricity_duty

# Display the results
print("----- Electricity Bill Details -----")
print(f"Energy Charges (EC): {energy_charges:.2f}")
print(f"Fixed Charges (FC): {fixed_charges:.2f}")
print(f"Electricity Duty (ED): {electricity_duty:.2f} ({electricity_duty_rate*100:.0f}%)")
print(f"Customer Charges (CC): {customer_charges:.2f}")

# Run python command?
python "C:\Users\ghg\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\ghg\OneDrive\Documents\Desktop\Assisted coding\electricity_billings.py"
```

Task 5: Final Bill Generation and Output Analysis

Scenario

The final electricity bill must present all values clearly.


```
File Edit Selection View Go Run Terminal Help
electricity_billings.py
# Electricity Billing System with Enhanced Accuracy and Real-world Features
# This program calculates comprehensive electricity bills with multiple charge components
# and includes validation and error handling for real-world applicability.

try:
    # Input validation and error handling for real-world applicability
    # Read previous units (0 for no validation)
    prev = float(input("Enter previous units: "))
    if prev < 0:
        raise ValueError("Previous units cannot be negative")

    # Read current units (00 for no validation)
    curr = float(input("Enter current units: "))
    if curr < 0:
        raise ValueError("Current units cannot be negative")

    # Read type of customer with validation
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower().strip()
    valid_types = ["domestic", "commercial", "industrial"]
    if customer_type not in valid_types:
        print(f"Invalid customer type '{customer_type}'. Valid options: {', '.join(valid_types)}")
        customer_type = "domestic" # Default fallback
        print("Using default customer type: {customer_type}")

except ValueError as e:
    print(f"❌ Input Error: {e}")
    print("Please enter valid numeric values for units.")
    exit()

# Calculate units consumed with validation
units_consumed = curr - prev

# Handle negative consumption (possible meter reset or error)
if units_consumed < 0:
    print(f"⚠️ Warning: Current units are less than previous units.")
    print("This may indicate a meter reading error or meter reset.")
    print("Setting energy charges to $0.00 for this billing cycle.")
    energy_charges = 0.0
    electricity_duty = 0.0
    units_consumed_display = f"({units_consumed:1f}) (adjusted to 0 for billing)"
else:
    units_consumed_display = f"{units_consumed:1f}"
    # Calculate Energy Charges (EC) based on customer type and units consumed
    # Standard pricing structure for accurate billing
    if customer_type == "domestic":
        # Domestic tiered pricing: 0.0 for first 100, 0.0 for next 200, 7.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0
    elif customer_type == "commercial":
        # Commercial tiered pricing: 0.0 for first 200, 12.0 thereafter
        if units_consumed <= 200:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 200 * 0.0 + (units_consumed - 200) * 12.0
    elif customer_type == "industrial":
        # Industrial tiered pricing: 10.0 for first 100, 15.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
    else:
        # Default to domestic rates for invalid input
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
electricity_duty = electricity_duty_rate * energy_charges

# Bill Summary for Industrial Customer
prev_units = 10
curr_units = 30
customer_type = "industrial"
units_consumed = 2.0

# Meter Reading Summary
print("--- Meter Reading Summary ---")
print(f"Previous units: {prev_units}")
print(f"Current units: {curr_units}")
print(f"Customer Type: {customer_type}")
print(f"Units Consumed: {units_consumed}")

# Electricity Bill Details
print("--- Electricity Bill Details ---")
print(f"Energy Charges (EC): $30.00")
print(f> Fixed Charges (FC): $20.00")
print(f"Customer Charges (CC): $0.00")
print(f"Electricity Duty (ED): $1.60 (5%)")
print(f>Total Bill Amount: $51.60")

# Bill Summary for Industrial Customer
print("--- Bill Summary for Industrial Customer ---")
print("New structure: Tiered pricing applied")
```

```
File Edit Selection View Go Run Terminal Help
electricity_billings.py
# Electricity Billing System with Enhanced Accuracy and Real-world Features
# This program calculates comprehensive electricity bills with multiple charge components
# and includes validation and error handling for real-world applicability.

try:
    # Input validation and error handling for real-world applicability
    # Read previous units (0 for no validation)
    prev = float(input("Enter previous units: "))
    if prev < 0:
        raise ValueError("Previous units cannot be negative")

    # Read current units (00 for no validation)
    curr = float(input("Enter current units: "))
    if curr < 0:
        raise ValueError("Current units cannot be negative")

    # Read type of customer with validation
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower().strip()
    valid_types = ["domestic", "commercial", "industrial"]
    if customer_type not in valid_types:
        print(f"Invalid customer type '{customer_type}'. Valid options: {', '.join(valid_types)}")
        customer_type = "domestic" # Default fallback
        print("Using default customer type: {customer_type}")

except ValueError as e:
    print(f"❌ Input Error: {e}")
    print("Please enter valid numeric values for units.")
    exit()

# Calculate units consumed with validation
units_consumed = curr - prev

# Handle negative consumption (possible meter reset or error)
if units_consumed < 0:
    print(f"⚠️ Warning: Current units are less than previous units.")
    print("This may indicate a meter reading error or meter reset.")
    print("Setting energy charges to $0.00 for this billing cycle.")
    energy_charges = 0.0
    electricity_duty = 0.0
    units_consumed_display = f"({units_consumed:1f}) (adjusted to 0 for billing)"
else:
    units_consumed_display = f"{units_consumed:1f}"
    # Calculate Energy Charges (EC) based on customer type and units consumed
    # Standard pricing structure for accurate billing
    if customer_type == "domestic":
        # Domestic tiered pricing: 0.0 for first 100, 0.0 for next 200, 7.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0
    elif customer_type == "commercial":
        # Commercial tiered pricing: 0.0 for first 200, 12.0 thereafter
        if units_consumed <= 200:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 200 * 0.0 + (units_consumed - 200) * 12.0
    elif customer_type == "industrial":
        # Industrial tiered pricing: 10.0 for first 100, 15.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
    else:
        # Default to domestic rates for invalid input
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
electricity_duty = electricity_duty_rate * energy_charges

# Bill Summary for Industrial Customer
prev_units = 10
curr_units = 30
customer_type = "industrial"
units_consumed = 2.0

# Meter Reading Summary
print("--- Meter Reading Summary ---")
print(f"Previous units: {prev_units}")
print(f"Current units: {curr_units}")
print(f"Customer Type: {customer_type}")
print(f"Units Consumed: {units_consumed}")

# Electricity Bill Details
print("--- Electricity Bill Details ---")
print(f"Energy Charges (EC): $30.00")
print(f> Fixed Charges (FC): $20.00")
print(f"Customer Charges (CC): $0.00")
print(f"Electricity Duty (ED): $1.60 (5%)")
print(f>Total Bill Amount: $51.60")

# Bill Summary for Industrial Customer
print("--- Bill Summary for Industrial Customer ---")
print("New structure: Tiered pricing applied")
```


Assignment 4.3

Task 1: Zero-Shot Prompting – Leap Year Check

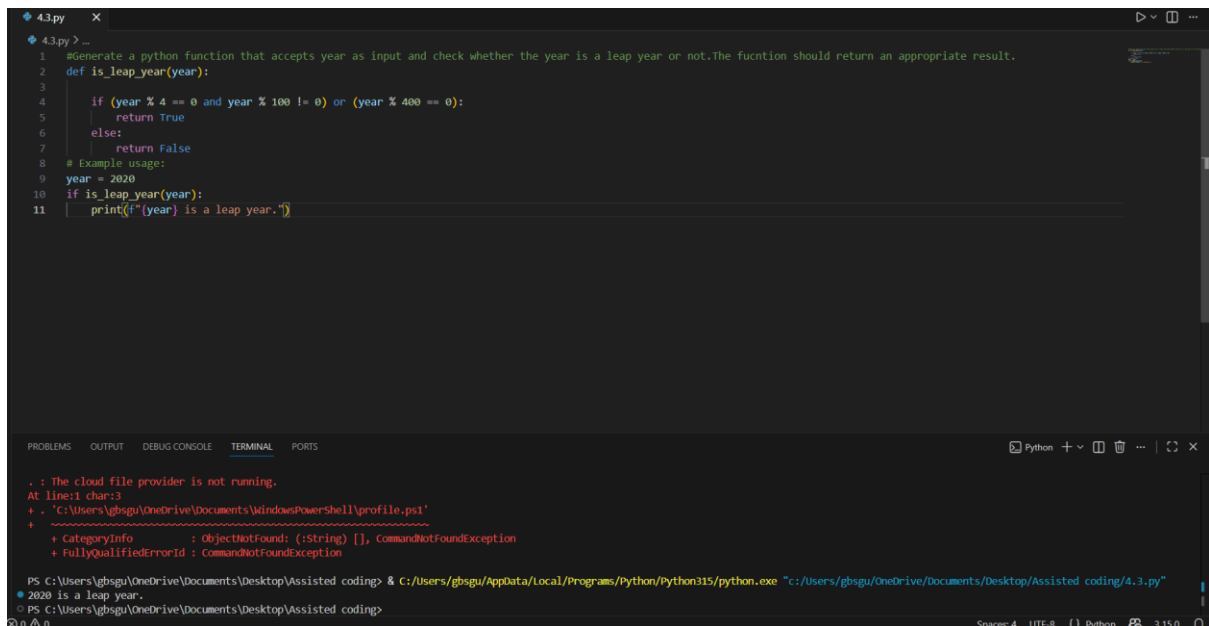
Scenario

Zero-shot prompting involves giving instructions without providing examples.

Task Description

Use zero-shot prompting to instruct an AI tool to generate a Python function that:

- Accepts a year as input
- Checks whether the given year is a leap year
- Returns an appropriate result



```
4.3.py
1 #Generate a python function that accepts year as input and check whether the year is a leap year or not.The fuction should return an appropriate result.
2 def is_leap_year(year):
3
4     if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
5         return True
6     else:
7         return False
8 # Example usage:
9 year = 2020
10 if is_leap_year(year):
11     print(f"{year} is a leap year.")
```

```
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbgsu/AppData/Local/Programs/Python/Python315/python.exe "c:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding/4.3.py"
2020 is a leap year.
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

Task 2: One-Shot Prompting – Centimeters to Inches Conversion

Scenario

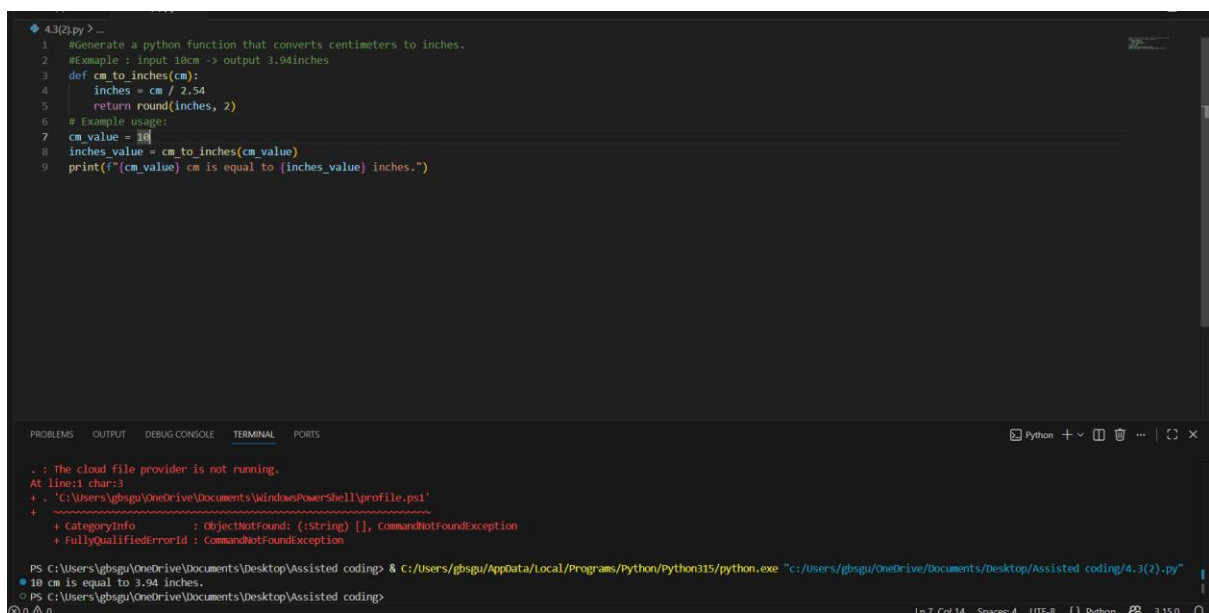
One-shot prompting guides AI using a single example.

Task Description

Use one-shot prompting by providing one input-output example to generate a Python

function that:

- Converts centimeters to inches
- Uses the correct mathematical formula



```
4.3(2).py > -
1 #Generate a python function that converts centimeters to inches.
2 #Example : input 10cm -> output 3.94inches
3 def cm_to_inches(cm):
4     inches = cm / 2.54
5     return round(inches, 2)
6 # Example usage:
7 cm_value = 10
8 inches_value = cm_to_inches(cm_value)
9 print(f"{cm_value} cm is equal to {inches_value} inches.")

. : The cloud file provider is not running.
At line:1 char:3
+ . 'C:/Users/gbgsu/OneDrive/Documents/WindowsPowerShell/profile.ps1'
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (String []) CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding> & C:/Users/gbgsu/AppData/Local/Programs/Python/Python315/python.exe "c:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding/4.3(2).py"
10 cm is equal to 3.94 inches.
PS C:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding>
```

Python + 

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034 1035 1036 1037 1038 1039 1040 1

Task 3: Few-Shot Prompting – Name Formatting

Scenario

Few-shot prompting improves accuracy by providing multiple examples.

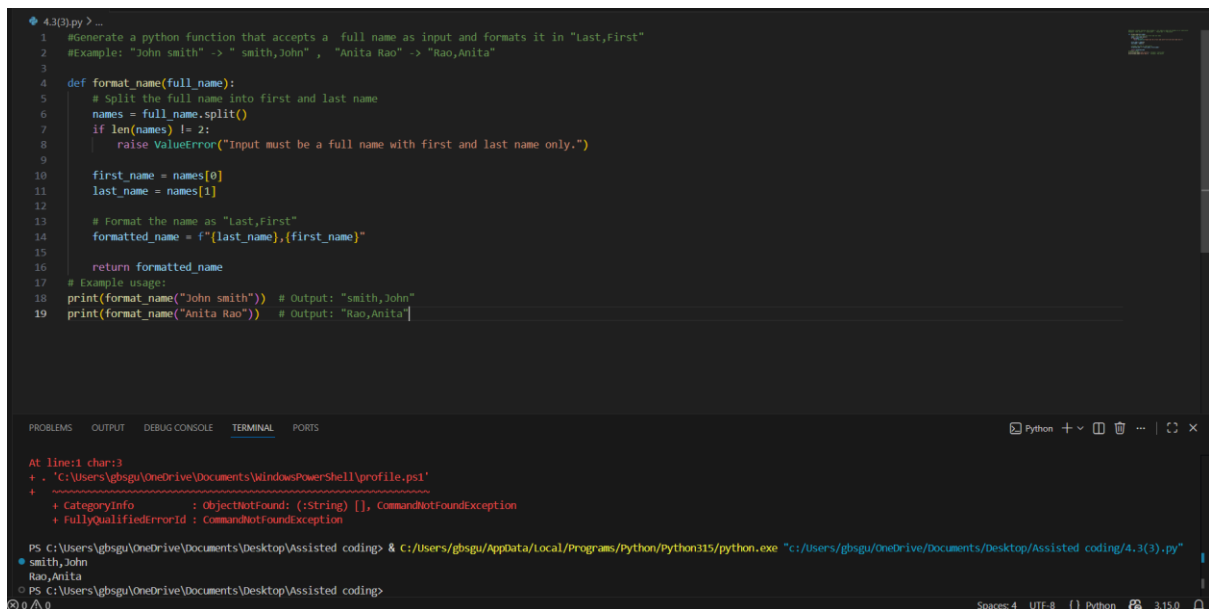
Task Description

Use few-shot prompting with 2–3 examples to generate a Python function that:

- Accepts a full name as input
- Formats it as “Last, First”

Example formats:

- "John Smith" → "Smith, John"
- "Anita Rao" → "Rao, Anita"



```
4.3(3).py > ...
1 #Generate a python function that accepts a full name as input and formats it in "Last,First"
2 #Example: "John smith" -> " smith,John" , "Anita Rao" -> "Rao,Anita"
3
4 def format_name(full_name):
5     # Split the full name into first and last name
6     names = full_name.split()
7     if len(names) != 2:
8         raise ValueError("Input must be a full name with first and last name only.")
9
10    first_name = names[0]
11    last_name = names[1]
12
13    # Format the name as "Last,First"
14    formatted_name = f"{last_name},{first_name}"
15
16    return formatted_name
17
18 # Example usage:
19 print(format_name("John smith")) # Output: "smith,John"
20 print(format_name("Anita Rao")) # Output: "Rao,Anita"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

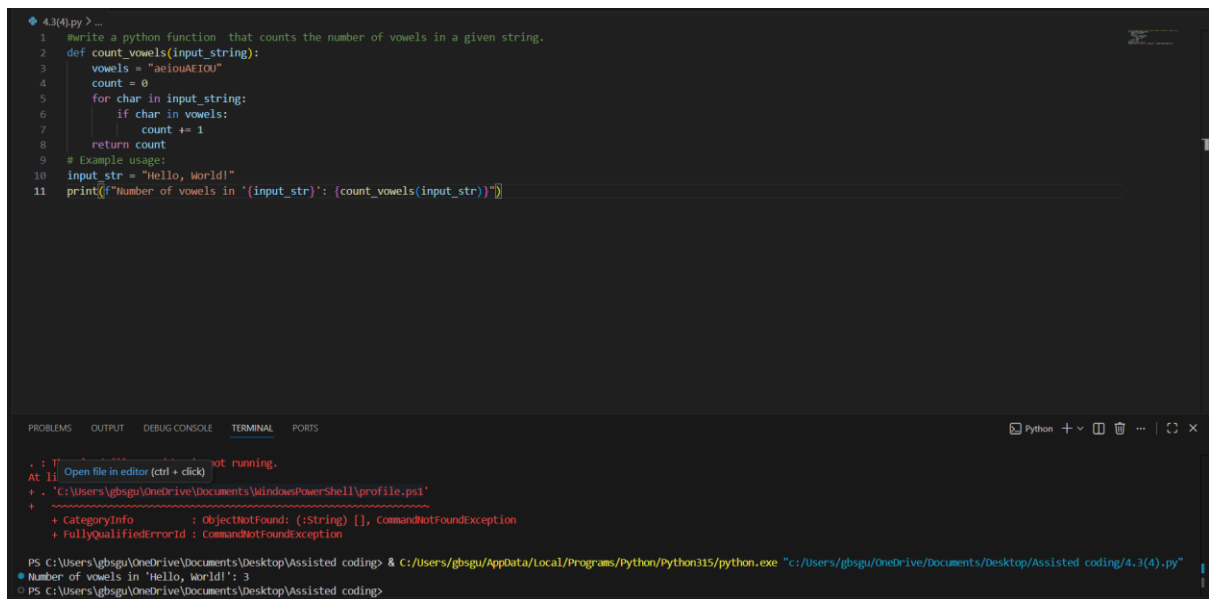
At line:1 char:3
+ . 'C:\Users\ghsgu\OneDrive\Documents\WindowsPowerShell\profile.ps1'
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\ghsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/ghsgu/AppData/Local/Programs/Python/Python315/python.exe "c:/Users/ghsgu/OneDrive/Documents/Desktop/Assisted coding/4.3(3).py"

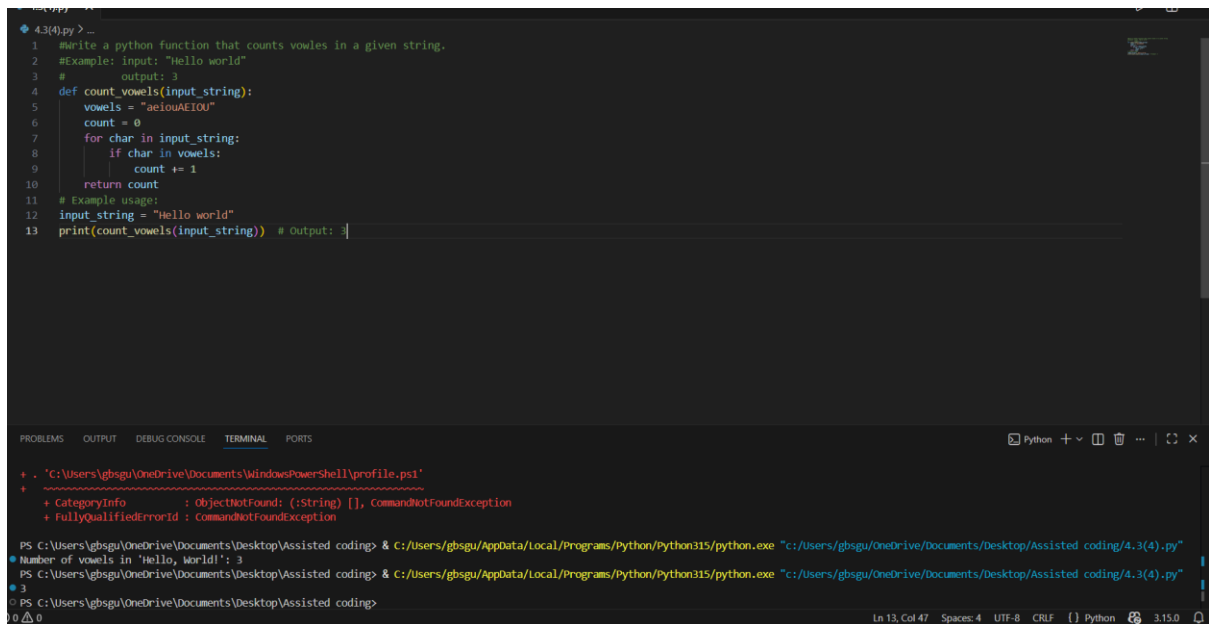
smith,John
Rao,Anita

PS C:\Users\ghsgu\OneDrive\Documents\Desktop\Assisted coding>

Spaces: 4 UTF-8 Python 3.15.0



Few shot prompt:



```
4.3(4).py > -
1 #write a python function that counts vowels in a given string.
2 #Example: input: "Hello world"
3 #       output: 3
4 def count_vowels(input_string):
5     vowels = "aeiouAEIOU"
6     count = 0
7     for char in input_string:
8         if char in vowels:
9             count += 1
10    return count
11 # Example usage:
12 input_string = "Hello world"
13 print(count_vowels(input_string)) # Output: 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
+ . 'C:\Users\gbsgu\OneDrive\Documents\WindowsPowerShell\profile.ps1'
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbgsu/AppData/Local/Programs/Python/Python315/python.exe "c:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding/4.3(4).py"
Number of vowels in 'Hello, World!': 3
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbgsu/AppData/Local/Programs/Python/Python315/python.exe "c:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding/4.3(4).py"
3
PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding>
```

Task 5: Few-Shot Prompting – File Handling

Scenario

File processing requires clear logical understanding.

Task Description

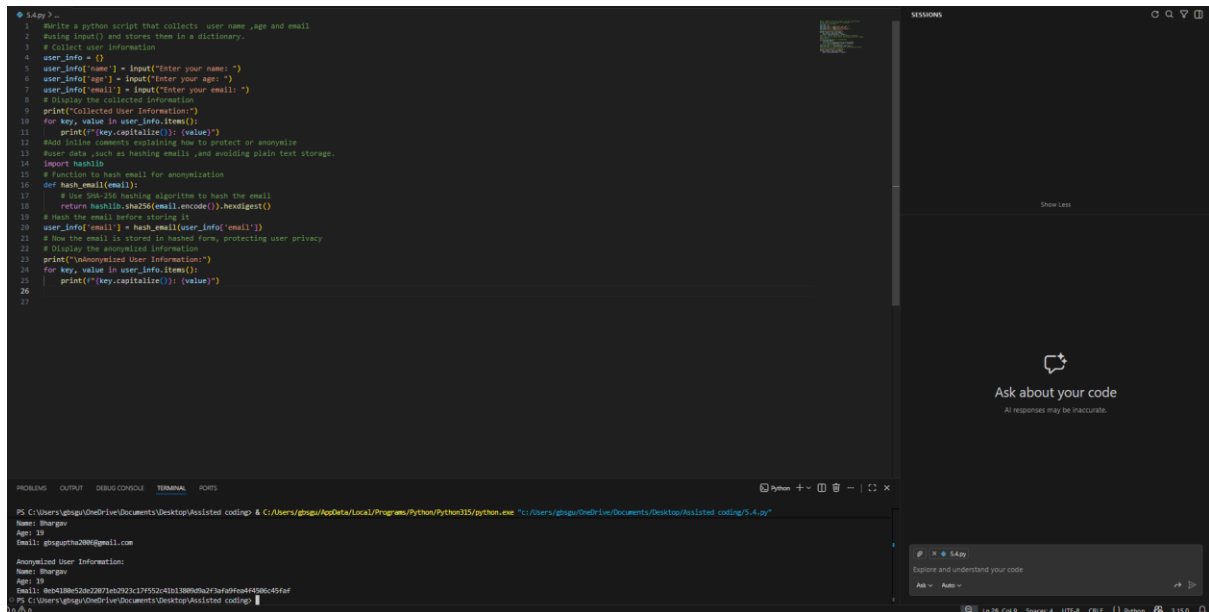
Use few-shot prompting to generate a Python function that:

- Reads a .txt file
- Counts the number of lines in the file
- Returns the line count

Assignment-5.4

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.



```
1 # Write a python script that collects user name ,age and email
2 # Using input() and stores them in a dictionary.
3 # Collect user information
4 user_info = {}
5 user_info['name'] = input("Enter your name: ")
6 user_info['age'] = input("Enter your age: ")
7 user_info['email'] = input("Enter your email: ")
8 # Display the collected information
9 print("Collected User Information:")
10 for key, value in user_info.items():
11     print(f"{key.capitalize()}: {value}")
12
13 # Add inline comments explaining how to protect or anonymize
14 # user data ,such as hashing emails ,and avoiding plain text storage.
15 import hashlib
16
17 # Function to hash email for anonymization
18 def hash_email(email):
19     # Use SHA-256 hashing algorithm to hash the email
20     return hashlib.sha256(email.encode()).hexdigest()
21
22 # Hash the email before storing it
23 user_info['email'] = hash_email(user_info['email'])
24 # Now the email is stored in hashed form, protecting user privacy
25 # Display the anonymized information
26 print("Anonymized User Information:")
27 for key, value in user_info.items():
28     print(f"{key.capitalize()}: {value}")
29
30
31
32
33
34
35
36
37
```

MESSAGES

Show Less

Ask about your code
AI responses may be inaccurate.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + Python 3.10.0

PS C:\Users\gmguy\OneDrive\Documents\Desktop\Assisted coding> C:\Users\gmguy\AppData\Local\Programs\Python\Python310\python.exe "C:\Users\gmguy\OneDrive\Documents\Desktop\Assisted coding\5.4.py"

Name: Bhargav
Age: 19
Email: gmguy2000@gmail.com

Anonymized User Information:
Name: Bhargav
Age: 19
Email: 8ea418b532a22871e292c17f5524b13889b2c3af9f0e4429c4cfaf

PS C:\Users\gmguy\OneDrive\Documents\Desktop\Assisted coding>

GitHub copilot was prompted to generate a python script that collects basic user information such as name ,age and email .

A follow -up prompt was used to add inline comments explaining data protection techniques .

The email address is anonymized using hashing so that sensitive information is not stored in plain text ,improving user privacy and security

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis.

Then prompt Copilot to identify and handle potential biases in the data.

```

1 # Write a Python function that performs basic sentiment analysis
2 # on a given text and returns Positive, Negative, or Neutral
3
4 def sentiment_analysis(text):
5     # Define simple lists of positive and negative words
6     positive_words = ['happy', 'joy', 'love', 'excellent', 'good', 'great', 'fantastic', 'positive']
7     negative_words = ['sad', 'anger', 'hate', 'terrible', 'bad', 'awful', 'horrible', 'negative']
8
9     # Initialize counters for positive and negative words
10    pos_count = 0
11    neg_count = 0
12
13    # Split the text into words for analysis
14    words = text.lower().split()
15
16    # Count occurrences of positive and negative words
17    for word in words:
18        if word in positive_words:
19            pos_count += 1
20        elif word in negative_words:
21            neg_count += 1
22
23    # Determine sentiment based on counts
24    if pos_count > neg_count:
25        return "Positive"
26    elif neg_count > pos_count:
27        return "Negative"
28    else:
29        return "Neutral"
30
31 # Example usage
32 text = "I love programming but sometimes it can be terrible."
33 result = sentiment_analysis(text)
34 print(f"The sentiment of the text is: {result}")

```

```

PS C:\Users\ghigu> python .\sentiment_analysis.py
The sentiment of the text is: Positive

```

```
def sentiment_analysis(text):
    """A Python function that performs basic sentiment analysis.
    It takes a string of text and returns 'Positive', 'Negative', or 'Neutral'
    based on the frequency of words in a predefined list of positive and negative words.
    """
    # Define a list of positive and negative words
    positive_words = ['happy', 'joy', 'excited', 'good', 'great', 'fantastic', 'positive']
    negative_words = ['sad', 'angry', 'bored', 'terrible', 'bad', 'awful', 'horrible', 'negative']

    # Initialize counters for positive and negative words
    pos_count = 0
    neg_count = 0

    # Split the text into words for analysis
    words = text.lower().split()

    # Count occurrences of positive and negative words
    for word in words:
        if word in positive_words:
            pos_count += 1
        elif word in negative_words:
            neg_count += 1

    # Determine sentiment based on counts
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

# Example usage
text = "I love programming but sometimes it can be terrible."
result = sentiment_analysis(text)
print(f"The sentiment of the text is: {result}")

# Add comments or logic to identify and reduce potential bias
# In sentiment analysis data, such as removing offensive terms
# and balancing positive and negative words.
# To reduce bias, we can remove our word lists to include
# more diverse terms and regularly update them based on feedback.
# Additionally, we can implement a filtering step to remove
# offensive or culturally insensitive terms from the analysis.
# Before performing sentiment analysis:
# This helps ensure that the analysis is fair and respectful to all users.
# For example, we can create a list of offensive terms to filter out
# offensive_terms = ['stupid', 'idiot', 'dumb', 'hate']
# Function to filter offensive terms from text
def filter_offensive_terms(text):
    words = text.split()
    filtered_words = [word for word in words if word.lower() not in offensive_terms]
    return " ".join(filtered_words)

# Remove offensive terms
filtered_text = filter_offensive_terms(text)
result = sentiment_analysis(filtered_text)
print(f"The sentiment of the filtered text is: {result}")
```

Explanation:

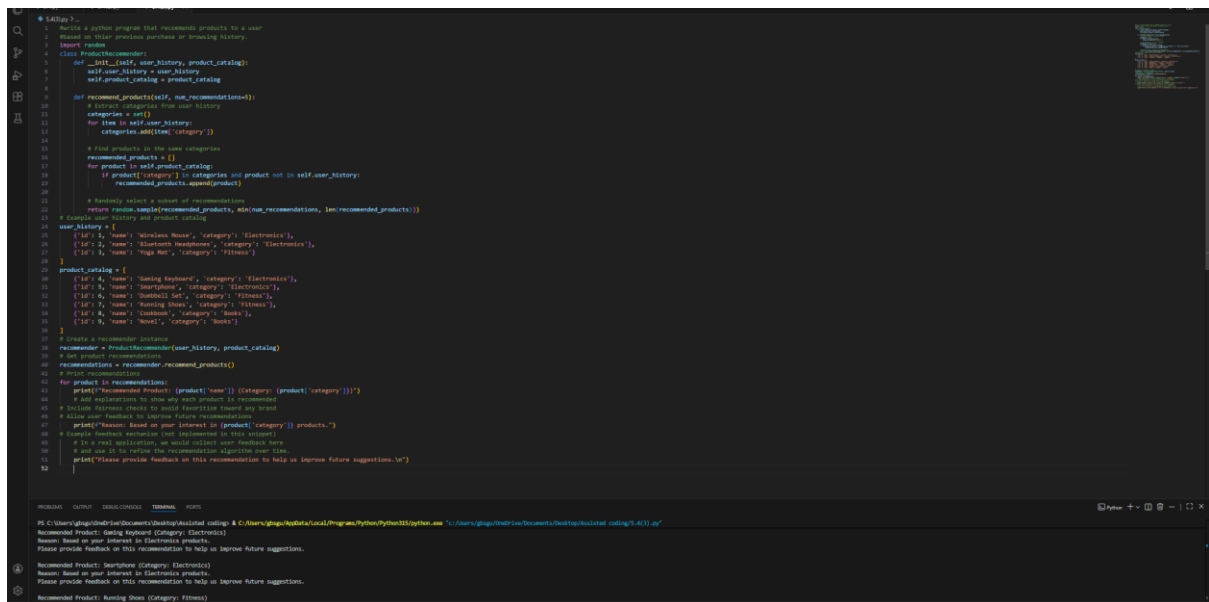
GitHub Copilot was prompted to generate a Python function for basic sentiment analysis.

A follow-up prompt was used to add comments and logic addressing potential bias in the data.

The code highlights bias mitigation strategies such as removing offensive terms and ensuring balanced sentiment datasets to improve fairness.

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines like transparency and fairness.



```
1 #!/usr/bin/env python3
2 """A Python program that recommends products to a user
3 based on their previous purchases or browsing history.
4 """
5 import random
6
7 class ProductRecommender:
8     def __init__(self, user_history, product_catalog):
9         self.user_history = user_history
10        self.product_catalog = product_catalog
11
12    def recommend_products(self, num_recommendations=5):
13        """Recommend products based on user history.
14        categories = set()
15        for item in self.user_history:
16            categories.add(item['category'])
17
18        # Find products in the user categories
19        recommended_products = []
20        for product in self.product_catalog:
21            if product['category'] in categories and product not in self.user_history:
22                recommended_products.append(product)
23
24        # Randomly select a subset of recommendations
25        return random.sample(recommended_products, min(num_recommendations, len(recommended_products)))
26
27 # Sample user history and product catalog
28 user_history = [
29     {'id': 1, 'name': 'Wireless Mouse', 'category': 'Electronics'},
30     {'id': 2, 'name': 'Smartwatch headsets', 'category': 'Electronics'},
31     {'id': 3, 'name': 'Yoga Mat', 'category': 'Fitness'}
32 ]
33
34 product_catalog = [
35     {'id': 4, 'name': 'Gaming Keyboard', 'category': 'Electronics'},
36     {'id': 5, 'name': 'Smartphone', 'category': 'Electronics'},
37     {'id': 6, 'name': 'Dumbbell Set', 'category': 'Fitness'},
38     {'id': 7, 'name': 'Running Shoes', 'category': 'Fitness'},
39     {'id': 8, 'name': 'Cookbook', 'category': 'Books'},
40     {'id': 9, 'name': 'Novel', 'category': 'Books'}
41 ]
42
43 # Create a recommender instance
44 recommender = ProductRecommender(user_history, product_catalog)
45
46 # Get product recommendations
47 recommendations = recommender.recommend_products()
48
49 # Print recommendations
50 for product in recommendations:
51     print(f"Recommended Product: {product['name']} (Category: {product['category']})")
52
53 # Add transparency to show why items are recommended
54 # Include fairness checks to avoid favoritism toward any brand
55 # Explain user feedback to improve future recommendations
56
57 print(f"Reason: Based on your interest in {product['category']} products.")
58
59 # Example feedback mechanism (not implemented in this snippet)
60 # In a real application, we would collect user feedback here
61 # and use it to refine the recommendation algorithm over time
62
63 print("Please provide feedback on this recommendation to help us improve future suggestions.")
```

Explanation:

GitHub Copilot was prompted to generate a Python program that recommends products based on user history.

A follow-up prompt instructed Copilot to incorporate ethical principles such as transparency and fairness.

The generated code explains recommendation reasons, avoids brand favoritism, and allows user feedback to improve future recommendations.

Task Description #4:

- Prompt Copilot to generate logging functionality in a Python web

application. Then, ask it to ensure the logs do not record sensitive

information.

Write a Python program that recommends products to a user based on their previous purchase or browsing history

> Used 1 reference

✓ Read current file for context and requirements

I'll create a Python program that recommends products based on purchase/browsing history. Let me first check the current file state.

✓ Read 5.4(4).py

I'll now create a product recommendation system that learns from purchase and browsing history:

✓ Created 5.4(4).py

Now let's test the program:

✓ `cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" ; python "5.4(4).py"`

Let me try with the Python terminal instead:

✓ `python "c:/Users/gbsgu/OneDrive/Documents/Desktop/Assisted coding/5.4(4).py" <<< >`
`"A`nLaptop`nV`nR`n3`nE" 2>&1`

Let me create a test script instead:

✓ Created test_recommendations.py

✓ `cd "C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding" ; python test_recommendations.py`

Perfect! I've created a comprehensive product recommendation system. Here's what the program does:

5.4(4).py

Describe what to build next

Agent Auto



```
def get_category_score(history, product_category):
    """Calculate recommendation score based on category frequency in history"""
    if not history:
        return 0
    category_count = sum(1 for item in history
                        if item in PRODUCTS and PRODUCTS[item]['category'] == product_category)
    return category_count / len(history)

def get_price_similarity_score(history, product_price):
    """Calculate score based on average price of browsed items"""
    if not history:
        return 0.5
    prices = [PRODUCTS[item]['price'] for item in history if item in PRODUCTS]
    if not prices:
        return 0.5
    return sum(prices) / len(prices)

def recommend_products(history, num_recommendations):
    """Recommend products based on history"""
    # Sort by score (descending) and return top N
    recommendations = sorted(products.items(), key=lambda item: (get_category_score(history, item['category']) *
                                                                get_price_similarity_score(history, item['price'])), reverse=True)
    return recommendations[:num_recommendations]

def display_recommendations(recommendations):
    """Display recommendations in a formatted table"""
    if not recommendations:
        print("\nNo recommendations available")
        return
    print("\nRecommended Products:")
    for rec in recommendations:
        product, details = rec
        print(f"Product: {product} | Category: {details['category']} | Price: {details['price']} | Rating: {details['rating']} | Score: {details['score']}")

def display_history(history):
    """Display user's browsing/purchase history"""
    if not history:
        print("\nNo history available")
        return
    print("\nUser Browsing/Purchase History:")
    for item in history:
        product, details = item
        print(f"Product: {product} | Category: {details['category']} | Price: {details['price']} | Rating: {details['rating']} | Score: {details['score']}")
```

```
def recommend_products(history, num_recommendations):
    """Recommend products based on history"""
    # Sort by score (descending) and return top N
    recommendations = sorted(products.items(), key=lambda item: (get_category_score(history, item['category']) *
                                                                get_price_similarity_score(history, item['price'])), reverse=True)
    return recommendations[:num_recommendations]

def display_recommendations(recommendations):
    """Display recommendations in a formatted table"""
    if not recommendations:
        print("\nNo recommendations available")
        return
    print("\nRecommended Products:")
    for rec in recommendations:
        product, details = rec
        print(f"Product: {product} | Category: {details['category']} | Price: {details['price']} | Rating: {details['rating']} | Score: {details['score']}")

def display_history(history):
    """Display user's browsing/purchase history"""
    if not history:
        print("\nNo history available")
        return
    print("\nUser Browsing/Purchase History:")
    for item in history:
        product, details = item
        print(f"Product: {product} | Category: {details['category']} | Price: {details['price']} | Rating: {details['rating']} | Score: {details['score']}")
```



```
File Edit Selection View Go Run Terminal Help
54@ipy>
def evaluate_model(model, X_test, y_test):
    print("Classification Report:")
    print(classification_report(y_test, predictions))
    return predictions

def predict_loan_approval(model):
    """Interactive prediction with CLI args fallback"""
    print("\n--- Loan Approval Prediction ---")
    # Try command-line arguments first
    if len(sys.argv) > 4:
        try:
            age = int(sys.argv[1])
            income = int(sys.argv[2])
            credit_score = int(sys.argv[3])
            employment_years = int(sys.argv[4])
        except ValueError:
            print("Invalid command-line arguments. Using interactive mode.")
            age, income, credit_score, employment_years = get_user_input()
        else:
            age, income, credit_score, employment_years = get_user_input()

    # Validate inputs
    if age < 0 or income < 0 or credit_score < 0 or employment_years < 0:
        print("Error: All values must be non-negative.")
        exit(1)

    prediction = model.predict([age, income, credit_score, employment_years])
    result = "/" if prediction[0] == 1 else "X"

    print("\nApplicant Profile:")
    print(f"Age: {age}")
    print(f"Income: ${income:,000}")
    print(f"Credit Score: {credit_score}")
    print(f"Employment Years: {employment_years}")

    # Add documentation or comments explaining how to use this model responsibly
    # Include explainability, accuracy limitations, and fairness considerations
    print("\n--- Important Notes ---")
    print("1. This model is a simplified example and should not be used for real-world decisions.")
    print("2. Model accuracy may vary with different datasets; always validate with your own data.")
    print("3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.")
    print("4. Always provide transparency about how predictions are made to users.")

    return result

if __name__ == "__main__":
    print("Simple ML Loan Prediction Model ---")
    # Create and split dataset
    X, y = create_sample_dataset()
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
    # Train and evaluate
    model = train_model(X_train, y_train)
    evaluate_model(model, X_test, y_test)
    # Make predictions
    predict_loan_approval(model)
```

PS C:\Users\github\Documents\Desktop\Assisted coding\54_5.py>
Enter employment years (default 5): 5

Applicant Profile:
Age: 30
Income: \$50k,000
Credit Score: 700
Employment Years: 5

Prediction: / Loan Approved

--- Important Notes ---
1. This model is a simplified example and should not be used for real-world decisions.
2. Model accuracy may vary with different datasets; always validate with your own data.
3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.
4. Always provide transparency about how predictions are made to users.

```
File Edit Selection View Go Run Terminal Help
54@ipy>
def predict_loan_approval(model):
    print("Enter Employment Years: (employment_years)")
    print("\nPrediction: (result)")

def get_user_input():
    """Get user input with sensible defaults"""
    try:
        age = int(input("Enter age (default 30): ") or "30")
        income = int(input("Enter annual income (default 50000): ") or "50000")
        credit_score = int(input("Enter credit score (default 700): ") or "700")
        employment_years = int(input("Enter employment years (default 5): ") or "5")
        return age, income, credit_score, employment_years
    except ValueError:
        print("Invalid input. Using defaults.")
        return 30, 50000, 700, 5

if __name__ == "__main__":
    print("Simple ML Loan Prediction Model ---")
    # Create and split dataset
    X, y = create_sample_dataset()
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
    # Train and evaluate
    model = train_model(X_train, y_train)
    evaluate_model(model, X_test, y_test)
    # Make predictions
    predict_loan_approval(model)

# Add documentation or comments explaining how to use this model responsibly
# Include explainability, accuracy limitations, and fairness considerations
print("\n--- Important Notes ---")
print("1. This model is a simplified example and should not be used for real-world decisions.")
print("2. Model accuracy may vary with different datasets; always validate with your own data.")
print("3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.")
print("4. Always provide transparency about how predictions are made to users.")
```

PS C:\Users\github\Documents\Desktop\Assisted coding\54_5.py>
Enter employment years (default 5): 5

Applicant Profile:
Age: 30
Income: \$50k,000
Credit Score: 700
Employment Years: 5

Prediction: / Loan Approved

--- Important Notes ---
1. This model is a simplified example and should not be used for real-world decisions.
2. Model accuracy may vary with different datasets; always validate with your own data.
3. Consider fairness and bias in your dataset to avoid discriminatory outcomes.
4. Always provide transparency about how predictions are made to users.

Explanation:

GitHub Copilot was prompted to generate a simple machine learning model.

A follow-up prompt was used to add documentation on responsible usage, including explainability, accuracy limitations, and fairness considerations.

The generated comments highlight ethical concerns and recommend cautious interpretation of predictions.

Assignment-7.3

Task 1: Fixing Syntax Errors

Scenario

You are reviewing a Python program where a basic function definition contains a syntax error.

```
python

def add(a, b)
    return a + b
```

Requirements

- Provide a Python function `add(a, b)` with a missing colon
- Use an AI tool to detect the syntax error
- Allow AI to correct the function definition
- Observe how AI explains the syntax issue

Expected Output

- Corrected function with proper syntax
- Syntax error resolved successfully
- AI-generated explanation of the fix

```
1 # Resolve the Python code and identify syntax error
2 # Explain it and correct the code
3 def add(a, b)
4     return a + b
5
6 # The provided code has a syntax error in the function definition. The keyword "def" is used to define a function, but it is missing the function name. Additionally, the function body should be indented properly.
7 # Here is the corrected code:
8 def add(a, b):
9     return a + b
```


Task 2: Debugging Logic Errors in Loops

Scenario

You are debugging a loop that runs infinitely due to a logical mistake.

```
python

def count_down(n):
    while n >= 0:
        print(n)
        n += 1  # Should be n -= 1
```

Requirements

- Provide a loop with an increment or decrement error
- Use AI to identify the cause of infinite iteration
- Let AI fix the loop logic
- Analyze the corrected loop behavior

Expected Output

- Infinite loop issue resolved
- Correct increment/decrement logic applied
- AI explanation of the logic error

```
1 #Observe the python code and identify logical error that causes the loop to run infinitely.
2 #Explain why the infinite loop occurs,correct the loop logic and describe how the corrected loop behaves
3
4 def count_down(n):
5     while n >= 0:
6         print(n)
7         n += 1
8
9 #Example:
10 #The logical error in the code is that the variable "n" is being incremented ("n += 1") instead of decremented. This causes the loop to run infinitely because "n" will always be greater than or equal to 0, and it will keep increasing indefinitely.
11 #To correct the loop logic, we should change the increment operation to a decrement operation. The corrected code would look like this:
12 #Corrected code:
13 def count_down(n):
14     while n >= 0:
15         print(n)
16         n -= 1
```

Task 3: Handling Runtime Errors (Division by Zero)

Scenario

A Python function crashes during execution due to a division by zero error.

```
def divide(a, b):  
    return a / b  
  
print(divide(10, 0))
```

Requirements:

- Provide a function that performs division without validation
- Use AI to identify the runtime error
- Let AI add try-except blocks for safe execution
- Review AI's error-handling approach

Expected Output

- Function executes safely without crashing
- Division by zero handled using try-except
- Clear AI-generated explanation of runtime error handling

```
1 Analyze the following python code and identify run time error that occurs during execution.  
2 Explain why the error happens, then modify the code using try-except blocks to handle the error safely.  
3 Also explain how the error-handling solution works.  
4 #Code:  
5 def divide(a, b):  
6     return a / b  
7     print(divide(10, 0))  
8  
9 #Explanation of the error:  
10 #The code provided will raise a 'ZeroDivisionError' when it attempts to divide 10 by 0. This error occurs because division by zero is undefined in mathematics, and Python does not allow it.  
11 #To handle this error safely, we can use a try-except block to catch the exception and provide a user-friendly message instead of crashing the program. Here's the modified code:  
12  
13 def divide(a, b):  
14     try:  
15         return a / b  
16     except ZeroDivisionError:  
17         return "Error: Cannot divide by zero."  
18 print(divide(10, 0))  
19  
20 #In this modified code, the 'try' block contains the code that may raise an exception (the division operation). If a 'ZeroDivisionError' occurs, the control is transferred to the 'except' block,  
21 #which catches the exception and returns a user-friendly error message instead of allowing the program to crash. This way, we can handle the error gracefully and provide feedback to the user without interrupting the flow of the program.
```

Task 4: Debugging Class Definition Errors

Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

```
python

class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width
```

Requirements

- Provide a class definition with missing self-parameter
- Use AI to identify the issue in the `__init__()` method
- Allow AI to correct the class definition
- Understand why self is required

Expected Output

- Corrected `__init__()` method
- Proper use of self in class definition
- AI explanation of object-oriented error

```
7.30day 2
# Analyze the following python class definition and identify the error in the constructor method.
# Explain why the error occurs, correct the class definition, and explain why the self parameter is required in python classes.
class Rectangle:
    def __init__(length, width):
        self.length = length
        self.width = width

from copyreg import constructor

# Explanation of the error:
# The error in the constructor method of the Rectangle class is that the 'self' parameter is missing from the '__init__' method definition. In Python,
# the first parameter of any instance method (including the constructor) must be 'self', which refers to the instance of the class being created or manipulated.
# Corrected class definition:
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
# The 'self' parameter is required in Python classes because it allows the method to access and modify the instance's attributes. When you create an instance of a class, 'self' refers to that specific instance,
# enabling you to store and retrieve data unique to that instance. Without 'self', the method would not know which instance's attributes to access or modify, leading to errors when trying to use the class.
# Example of using the corrected Rectangle class:
rect = Rectangle(5, 3)
print(rect.length) # Output: 5
print(rect.width)  # Output: 3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
... The cloud file provider is not running.
AI: Inspect error:
- C:\Users\gngui\OneDrive\Documents\WindowsPowerShell\ps1.ps1
- CategoryInfo          : (ObjectNotFound: (String) []) CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\gngui\OneDrive\Documents\Desktop\Assisted coding> C:\Users\gngui\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gngui\OneDrive\Documents\Desktop\Assisted coding\7.30(4).py"
5
3
PS C:\Users\gngui\OneDrive\Documents\Desktop\Assisted coding>
```

Task 5: Resolving Index Errors in Lists

Scenario

A program crashes when accessing an invalid index in a list.

```
python

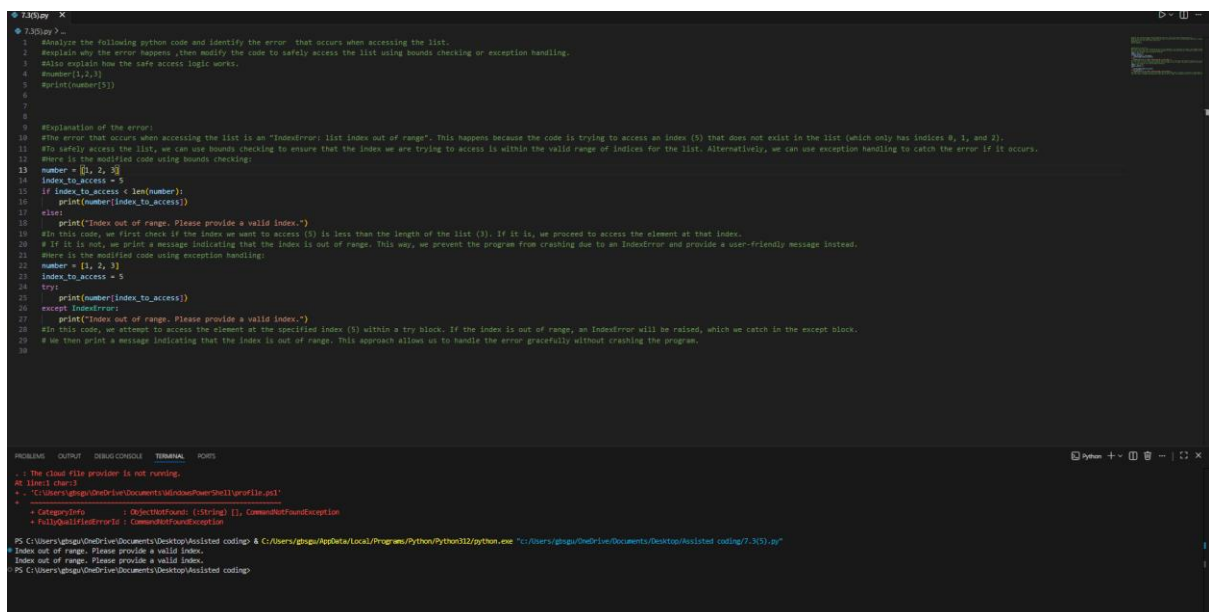
numbers = [1, 2, 3]
print(numbers[5])
```

Requirements

- Provide code that accesses an out-of-range list index
- Use AI to identify the Index Error
- Let AI suggest safe access methods
- Apply bounds checking or exception handling

Expected Output

- Index error resolved
- Safe list access logic implemented



```
7.3kay >
1 Analyze the following python code and identify the error that occurs when accessing the list.
2 Explain why the error happens, then modify the code to safely access the list using bounds checking or exception handling.
3 Also explain how the safe access logic works.
4 numbers = [1, 2, 3]
5 print(numbers[5])
6
7
8
9 #Explanation of the error:
10 #The error that occurs when accessing the list is an "IndexError: list index out of range". This happens because the code is trying to access an index (5) that does not exist in the list (which only has indices 0, 1, and 2).
11 #To safely access the list, we can use bounds checking to ensure that the index we are trying to access is within the valid range of indices for the list. Alternatively, we can use exception handling to catch the error if it occurs.
12 #Here is the modified code using bounds checking:
13 number = [1, 2, 3]
14 index_to_access = 5
15 if index_to_access < len(number):
16     print(number[index_to_access])
17 else:
18     print("Index out of range. Please provide a valid index.")
19 #In this code, we first check if the index we want to access (5) is less than the length of the list (3). If it is, we proceed to access the element at that index.
20 #If it is not, we print a message indicating that the index is out of range. This way, we prevent the program from crashing due to an IndexError and provide a user-friendly message instead.
21 #Here is the modified code using exception handling:
22 number = [1, 2, 3]
23 index_to_access = 5
24 try:
25     print(number[index_to_access])
26 except IndexError:
27     print("Index out of range. Please provide a valid index.")
28 #In this code, we attempt to access the element at the specified index (5) within a try block. If the index is out of range, an IndexError will be raised, which we catch in the except block.
29 #We then print a message indicating that the index is out of range. This approach allows us to handle the error gracefully without crashing the program.
30
```

```
! The cloud file provider is not running.
! libell charid
! C:\Users\gigu\OneDrive\Documents\libellPowerShell\profile.ps1
!
! Categories: : ObjectIdNotFound: (String) [], CommandNotFoundException
! Path\Shell\libellPowerShell
!
! C:\Users\gigu\OneDrive\Documents\Desktop\Assisted coding & C:\Users\gigu\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gigu\OneDrive\Documents\Desktop\Assisted coding\7.3k5.py"
!
! Index out of range. Please provide a valid index.
!
! Index out of range. Please provide a valid index.
!
! C:\Users\gigu\OneDrive\Documents\Desktop\Assisted coding
```


Lab assignment - 1.4

TASK - 0

Install and configure GitHub Copilot in VS Code. Take screenshots of each step.



Task 1: AI-Generated Logic Without Modularization (Prime Number Check Without Functions)

❖ Scenario

➤ You are developing a basic validation script for a numerical learning application.

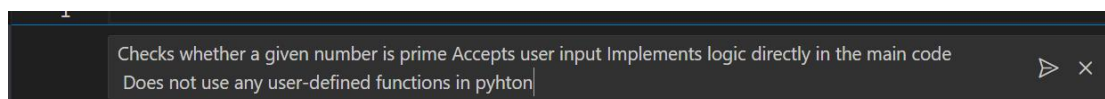
❖ Task Description

Use GitHub Copilot to generate a Python program that:

- Checks whether a given number is prime
- Accepts user input
- Implements logic directly in the main code
- Does not use any user-defined functions

❖ Expected Output

- Correct prime / non-prime result
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs



```

ass-1.4.py > ...
1  # Prime number checker without user-defined functions
2
3  number = int(input("Enter a number to check if it is prime: "))
4
5  if number < 2:
6      print(f"{number} is not a prime number.")
7  else:
8      is_prime = True
9      for i in range(2, int(number ** 0.5) + 1):
10         if number % i == 0:
11             is_prime = False
12             break
13
14     if is_prime:
15         print(f"{number} is a prime number.")
16     else:
17         print(f"{number} is not a prime number.")

```

```

:~/AI_ASSISTANT_CODING/ass-1.4.py
Enter a number to check if it is prime: 5
5 is a prime number.
PS D:\AI_ASSISTANT_CODING>

```

Task 2: Efficiency & Logic Optimization (Cleanup)

❖ Scenario

The script must handle larger input values efficiently.

❖ Task Description

Review the Copilot-generated code from Task 1 and improve it by:

- Reducing unnecessary iterations
- Optimizing the loop range (e.g., early termination)
- Improving readability
- Use Copilot prompts like:
 - "Optimize prime number checking logic"
 - "Improve efficiency of this code"

Hint:

Prompt Copilot with phrases like

"optimize this code", "simplify logic", or "make it more readable"

❖ Expected Output

- Original and optimized code versions
- Explanation of how the improvements reduce time complexity

```

optimize prime number checking logic and improve efficiency of this code
Add Context... Auto

```

```
18 |         # Prime number checker without user-defined functions
19 |
20 |
21 | number = int(input("Enter a number to check if it is prime: "))
22 |
23 |         # Check if prime
24 | is_prime = number > 1 and all(number % i != 0 for i in range(2, int(number ** 0.5) + 1))
25 |
26 | print(f"{number} is {'a prime' if is_prime else 'not a prime'} number.")
27 |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python + - [] [] ... []

```
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe"
:/AI_ASSISTANT_CODING/ass-1.4.py
5 is a prime number.
Enter a number to check if it is prime: 5
5 is a prime number.
PS D:\AI_ASSISTANT_CODING> |
```

The optimization checks divisibility only up to $\sqrt{n}$ instead of n , reducing unnecessary iterations. This works because any composite number must have a factor less than or equal to $\sqrt{n}$. Early termination using `break` or `all()` stops execution as soon as a divisor is found. Thus, time complexity improves from $O(n)$ to $O(\sqrt{n})$, with best case $O(1)$.

Task 3: Modular Design Using AI Assistance (Prime Number Check Using Functions)

❖ Scenario

The prime-checking logic will be reused across multiple modules.

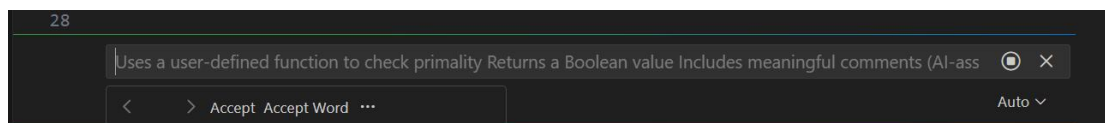
❖ Task Description

Use GitHub Copilot to generate a function-based Python program that:

- Uses a user-defined function to check primality
- Returns a Boolean value
- Includes meaningful comments (AI-assisted)

❖ Expected Output

- Correctly working prime-checking function
- Screenshots documenting Copilot's function generation
- Sample test cases and outputs




```
ass-1.4.py > ...
29 # User-defined function to check if a number is prime
30 def is_prime_number(n):
31     """
32     Checks if a number is prime.
33
34     Args:
35     | n: Integer to check for primality
36
37     Returns:
38     | Boolean: True if prime, False otherwise
39     """
40     # Numbers less than 2 are not prime
41     if n < 2:
42         return False
43
44     # Check divisibility from 2 to square root of n
45     for i in range(2, int(n ** 0.5) + 1):
46         if n % i == 0:
47             return False
48
49     # If no divisors found, number is prime
50     return True
51
52 # Test the function
53 test_number = int(input("Enter a number to check if it is prime: "))
54 result = is_prime_number(test_number)
55 print(f"{test_number} is {'a prime' if result else 'not a prime'} number.")

PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d
:/AI_ASSISTANT_CODING/ass-1.4.py
Enter a number to check if it is prime: 5
5 is a prime number.
Enter a number to check if it is prime: 5
5 is a prime number.
Enter a number to check if it is prime: 5
5 is a prime number.
Enter a number to check if it is prime: 5
5 is a prime number.
PS D:\AI_ASSISTANT_CODING>
```

Task 4: Comparative Analysis –With vs Without Functions

❖ Scenario

You are participating in a technical review discussion.

❖ Task Description

Compare the Copilot-generated programs:

- Without functions (Task 1)
- With functions (Task 3)
- Analyze them based on:
 - Code clarity
 - Reusability
 - Debugging ease
 - Suitability for large-scale applications

❖ Expected Output

Comparison table or short analytical report

Comparison Report of the 3 Prime Number Checker Codes

Code 1 – Loop with flag and break:

Uses a loop up to $\sqrt{n}$ and stops early when a divisor is found. Efficient and easy to understand. Time complexity is $O(\sqrt{n})$.

Code 2 – all() with generator expression:

Also checks up to $\sqrt{n}$ but uses Python's built-in all() with short-circuiting. More concise and Pythonic, same $O(\sqrt{n})$ time complexity, slightly better readability.

Code 3 – User-defined function (is_prime_number)

Encapsulates logic in a function, improving reusability and modularity. Performance is the same ($O(\sqrt{n})$), best suited for larger programs or repeated checks.

Overall Conclusion:

All three have the same time complexity ($O(\sqrt{n})$). Code 2 is the most concise, Code 1 is beginner-friendly, and Code 3 is best for structured and reusable code.

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to Prime Checking)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logical strategies.

❖ Task Description

Prompt GitHub Copilot to generate:

- A basic divisibility check approach
- An optimized approach (e.g., checking up to $\sqrt{n}$)

❖ Expected Output

- Two correct implementations
- Comparison discussing:
 - Execution flow
 - Time complexity
 - Performance for large inputs
 - When each approach is appropriate

A basic divisibility check approach

```
# Basic divisibility check approach
def basic_divisibility_check(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

# Test the basic divisibility check
test_number = int(input("Enter a number to check if it is prime using basic divisibility: "))
result = basic_divisibility_check(test_number)
print(f"{test_number} is {'a prime' if result else 'not a prime'} number.")
```

```
Enter a number to check if it is prime using basic divisibility: 3
3 is a prime number.
```

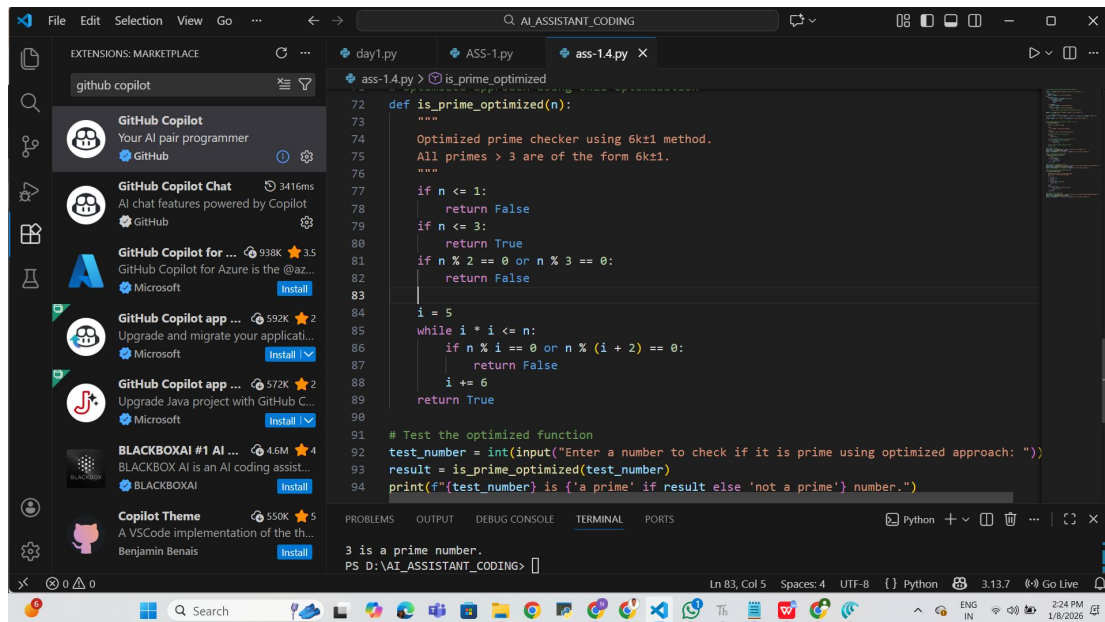
```
PS D:\AI_ASSISTANT_CODING>
```

An optimized approach

Add Context...

Auto

```
71 print(f"Factorial of {number} is {result}")
```



```

Enter a number to check if it is prime using basic divisibility: 3
3 is a prime number.
PS D:\AI_ASSISTANT_CODING>

```

Comparison (Short Notes)

Execution Flow:

Basic divisibility: Checks all numbers from 2 to $n-1$ sequentially.

Optimized ($\sqrt{n}$): Checks divisors only up to $\sqrt{n}$ and stops early if a factor is found.

Function-based: Same optimized flow but wrapped in a reusable function.

Time Complexity:

Basic divisibility: $O(n)$

Optimized ($\sqrt{n}$): $O(\sqrt{n})$

Function-based optimized: $O(\sqrt{n})$

Performance for Large Inputs:

Basic approach is slow and inefficient for large numbers.

Optimized approaches are much faster and suitable for large inputs.

Function-based version performs the same but is cleaner for repeated use.

When Each Approach Is Appropriate

Basic approach: For learning and very small inputs.

Optimized approach: For efficient single checks.

Function-based approach: For structured programs and multiple prime checks.

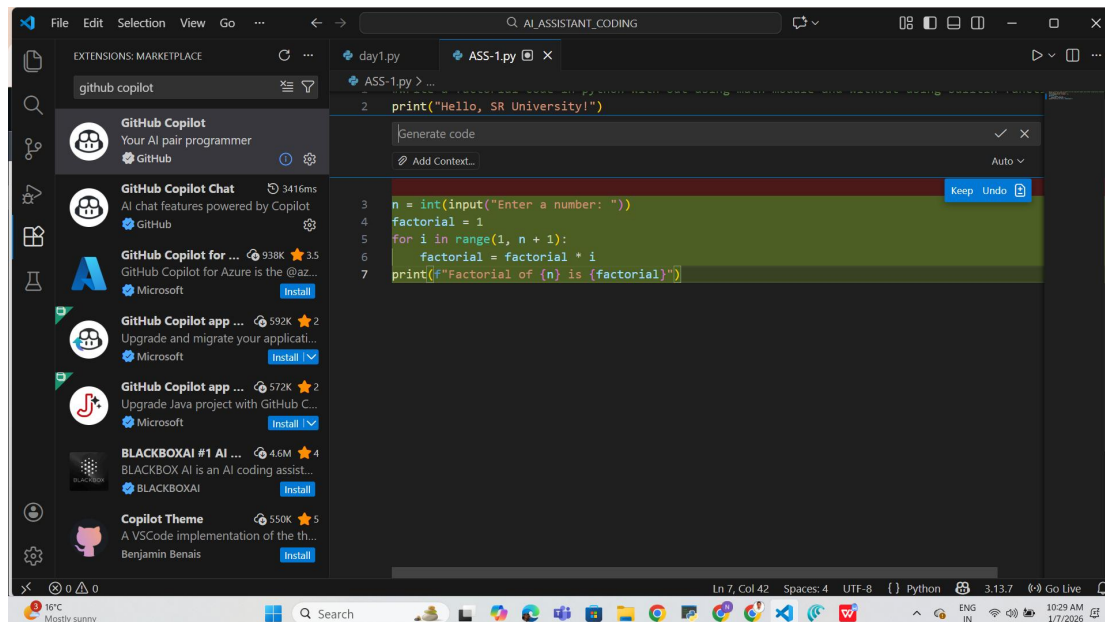
Lab assignment - 1

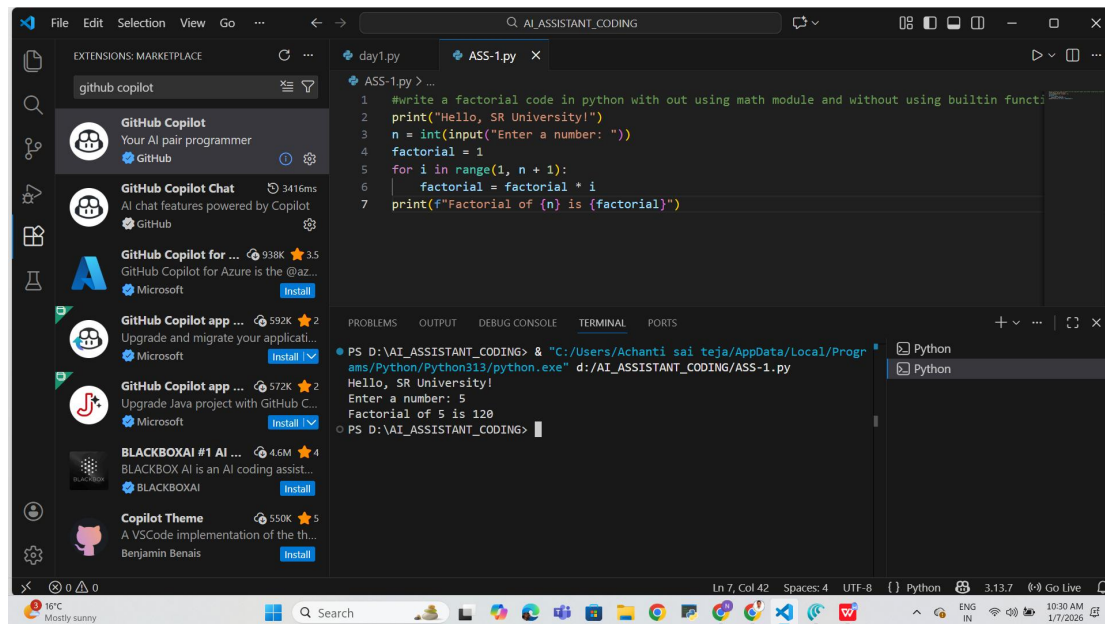
TASK - 1

Install and configure GitHub Copilot in VS Code. Take screenshots of each step.



Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)





The program takes a number from the user as input.
 It uses a loop to multiply numbers from 1 to the given number.
 The result of this multiplication gives the factorial value.
 Finally, it prints the factorial of the entered number.

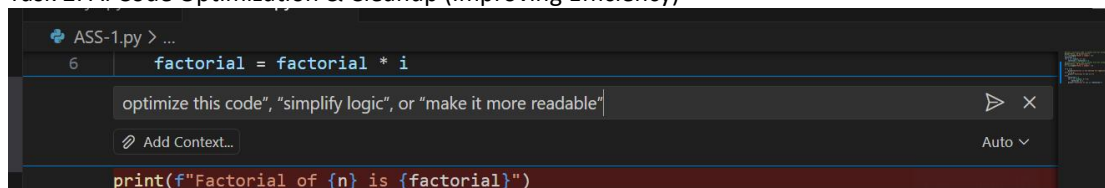
How helpful was Copilot for a beginner?

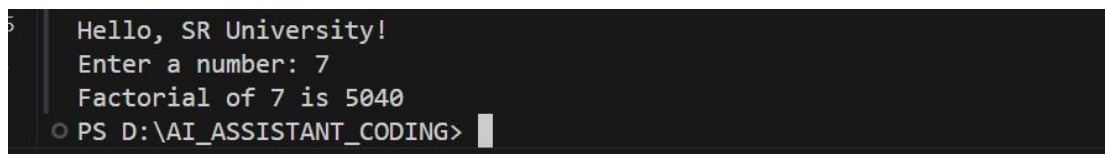
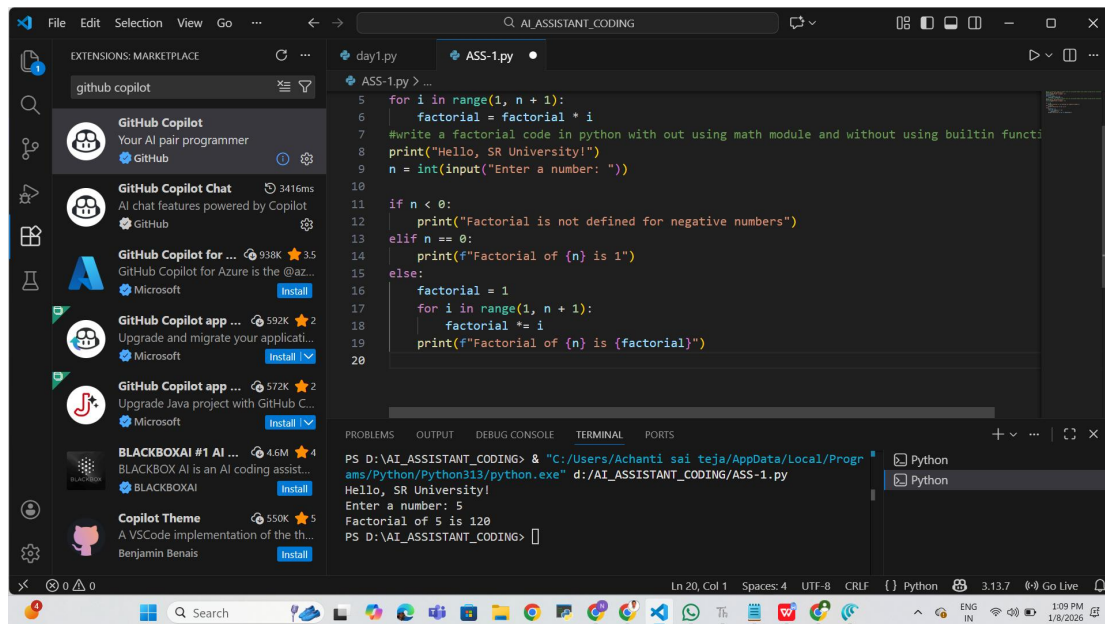
Copilot is very helpful for a beginner because it quickly suggests correct and readable code, reducing the time needed to think about syntax.
 It makes learning easier by providing a working structure, like loops and variable initialization, as seen in the factorial program.

Did it follow best practices automatically?

Copilot generally follows best practices such as clear variable names and simple logic flow.
 However, beginners still need to understand the code instead of blindly accepting suggestions.
 It may not always handle edge cases (like negative numbers) unless explicitly prompted.
 Overall, Copilot is a strong learning aid but not a replacement for conceptual understanding.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)





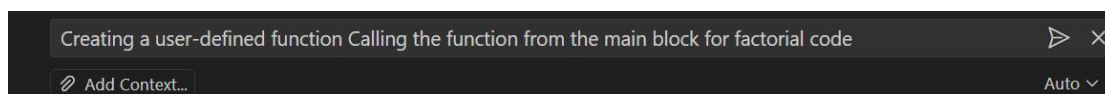
What was improved?

- Added proper input validation to handle negative numbers and the special case of 0.
- Removed duplicate code and structured the program with clear conditional branches.
- Ensured the program prints the final result in all valid cases.
- Improved clarity and flow of the logic.

Why the new version is better (readability, performance, maintainability)?

- Readability: Clear if-elif-else structure makes the logic easy to understand.
- Performance: Avoids unnecessary calculations for invalid or zero inputs.
- Maintainability: Well-organized conditions make future changes or enhancements easier.
- Reliability: Handles edge cases correctly, reducing runtime errors.

Task 3: Modular Design Using AI Assistance (Factorial with Functions)




```
def calculate_factorial(num):
    if num < 0:
        return "Factorial is not defined for negative numbers"
    elif num == 0:
        return 1
    else:
        factorial = 1
        for i in range(1, num + 1):
            factorial *= i
        return factorial

if __name__ == "__main__":
    number = int(input("Enter a number: "))
    result = calculate_factorial(number)
    print(f"Factorial of {number} is {result}")
```

```
Enter a number: 5
Factorial of 5 is 120
PS D:\AI_ASSISTANT_CODING> █
```

How Modularity Improves Reusability

Modularity means breaking a program into smaller, independent, and reusable units (functions or modules).

How it improves reusability:

A function can be reused multiple times without rewriting code.

The same function can be imported into other programs.

Changes made in one place (the function) automatically reflect everywhere it's used.

Code becomes easier to test, debug, and maintain.

In short, modularity avoids duplication and promotes clean, reusable logic.

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

Technical Report: Comparison of Non-Function and Function-Based Copilot-Generated Programs

This report compares non-function-based and function-based Copilot-generated Python programs for calculating factorials, focusing on clarity, reusability, debugging, scalability, and AI dependency risk. The non-function-based program implements the factorial logic directly in the main flow along with input and output operations. While this approach works for small scripts, it reduces logic clarity because computation, validation, and user interaction are mixed together. The code is also repeated, which makes understanding and maintaining it difficult. Any change in logic must be applied in multiple places, increasing the chance of errors.

In contrast, the function-based program separates the factorial logic into a dedicated function. This improves logic clarity by clearly defining responsibilities within the code. The function can be reused multiple times or imported into other programs, significantly improving reusability. Debugging is easier in the function-based approach because errors can be isolated within the function. The non-function-based version makes debugging harder due to duplicated and unstructured logic.

For large projects, non-function-based code does not scale well, whereas function-based code supports modularity, testing, and teamwork. Finally, AI dependency risk is higher in non-function-based programs, as repeated AI-generated code may introduce redundancy and inconsistencies. Function-based code minimizes this risk by promoting standardized and reusable logic. Overall, function-based programs are more reliable, maintainable, and suitable for long-term development.

Task 5: AI-Generated Iterative vs Recursive Thinking

Iterative model

```
def calculate_factorial(num):
    if num < 0:
        return "Factorial is not defined for negative numbers"
    elif num == 0:
        return 1
    else:
        factorial = 1
        for i in range(1, num + 1):
            factorial *= i
        return factorial

if __name__ == "__main__":
    number = int(input("Enter a number: "))
    result = calculate_factorial(number)
    print(f"Factorial of {number} is {result}")
```

Recursive model

```
def calculate_factorial_recursive(num):
    if num < 0:
        return "Factorial is not defined for negative numbers"
    elif num == 0:
        return 1
    else:
        return num * calculate_factorial_recursive(num - 1)

if __name__ == "__main__":
    number = int(input("Enter a number: "))
    result = calculate_factorial_recursive(number)
    print(f"Factorial of {number} is {result}")
```

Below is a clear execution flow explanation and comparison of the two AI-generated implementations (Iterative and Recursive factorial).

Execution Flow Explanation

Iterative Model

The program takes an integer input from the user.
The `calculate_factorial(num)` function checks:
If the number is negative → returns an error message.
If the number is 0 → returns 1.
For positive numbers, a loop runs from 1 to num.

The factorial value is updated step by step using multiplication.

The final result is returned and printed.

Flow: Input → Loop-based multiplication → Output

Recursive Model

The program takes an integer input from the user.

The `calculate_factorial_recursive(num)` function checks:

If the number is negative → returns an error message.

If the number is 0 → returns 1 (base case).

For positive numbers, the function calls itself with `num - 1`.

Recursive calls continue until the base case is reached.

Results are multiplied while returning back through the call stack.

Flow: Input → Repeated function calls → Base case → Backtracking → Output

Comparison

Readability

Iterative: More straightforward and easier for beginners to understand.

Recursive: Shorter and mathematically elegant, but harder to trace.

Stack Usage

Iterative: Uses constant memory (no call stack growth).

Recursive: Uses stack memory for each function call.

Performance Implications

Iterative: Faster and more memory-efficient.

Recursive: Slightly slower due to function call overhead.

When Recursion Is Not Recommended

When input size is large (risk of stack overflow).

When performance and memory efficiency are critical.

In environments with limited stack memory.

TASK-1

Task 1: Word Frequency from Text File

❖ Scenario:

You are analyzing log files for keyword frequency.

❖ Task:

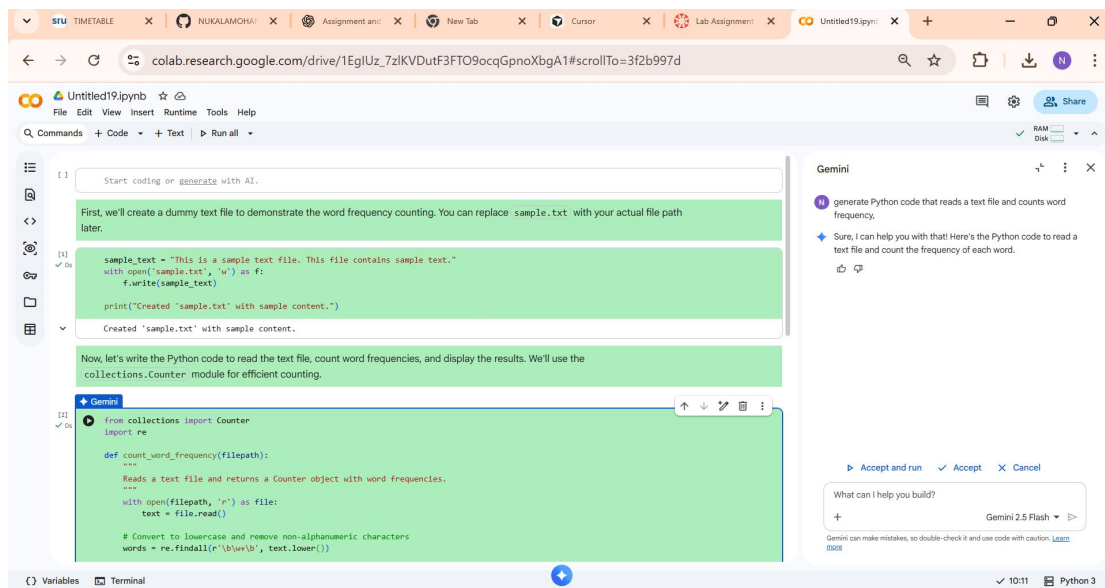
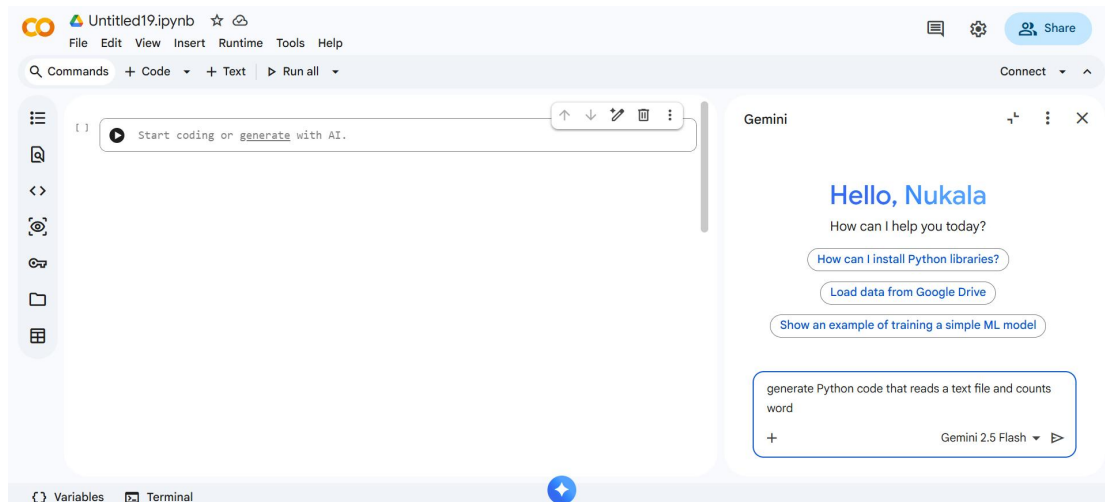
Use Gemini to generate Python code that reads a text file and counts word frequency, then explains the code.

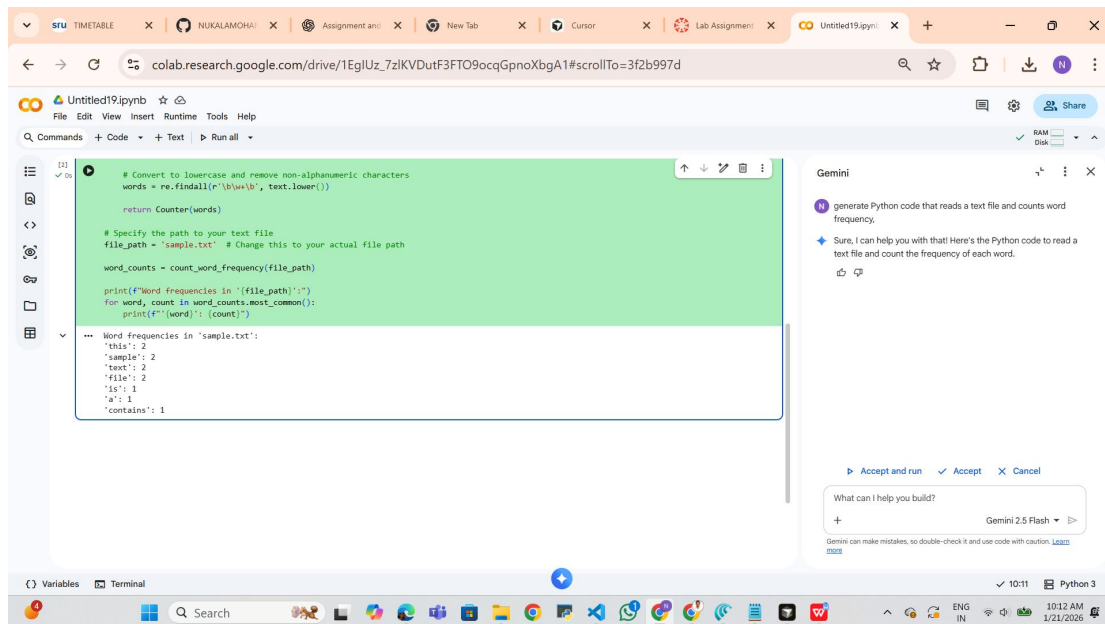
❖ Expected Output:

➤ Working code

➤ Explanation

➤ Screenshot





The code defines a function to read a text file, sample.txt, for processing. It converts the entire text to lowercase and extracts all alphanumeric words using regular expressions. The collections.Counter module is then used to efficiently count the frequency of each extracted word. The function is called with the sample.txt file, which was pre-populated with example text. Finally, the code iterates through the word counts and prints each word along with its frequency.

TASK-2

❖ Scenario:

You are automating basic file operations.

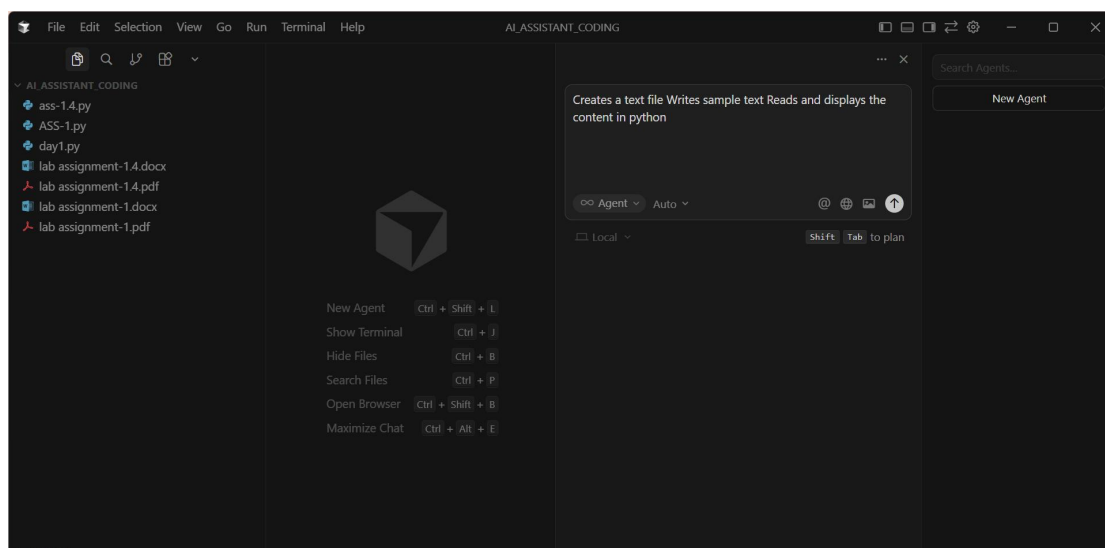
❖ Task:

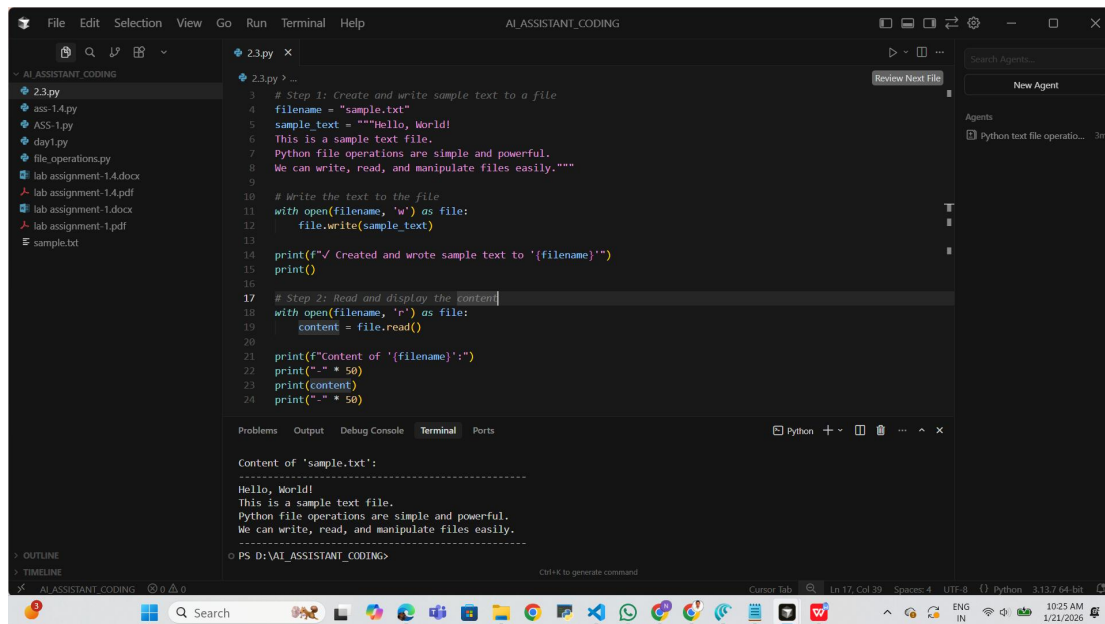
Use Cursor AI to generate a program that:

- Creates a text file
- Writes sample text
- Reads and displays the content

❖ Expected Output:

- Functional code
- Cursor AI screenshots





Task 3: CSV Data Analysis

❖ Scenario:

You are processing structured data from a CSV file.

❖ Task:

Use Gemini in Colab to read a CSV file and calculate mean, min, and max.

❖ Expected Output:

➤ Correct output

➤ Screenshot

o=x8lEKrop2ZcU

Share

RAM
Disk

Gemini

◆ Certainly! Here's a 5-line explanation of the code:

1. The code defines a function to read a text file, `sample.txt`, for processing.

2. It converts the entire text to lowercase and extracts all alphanumeric words using regular expressions.

3. The `collections.Counter` module is then used to efficiently count the frequency of each extracted word.

4. The function is called with the `sample.txt` file, which was pre-populated with example text.

5. Finally, the code iterates through the word counts and prints each word along with its frequency.

👍

💬

Show me the Python code to count word frequency

Run the word frequency counting code

How can I sort the word counts by frequency

read a CSV file and calculate mean, min, and max.

+

Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

✓ 10:11 Python 3

ENG
IN

10:34 AM
1/21/2026

colab.research.google.com/drive/1EglUz_7zIKVDutF3FTO9ocqGpnoXbgA1#scrollTo=fcc15417

Untitled19.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

Files

- ..
- config
- sample_data
- machine-readable-business-em...
- sample.txt
- sample_data.csv

Disk 86.58 GB available

Start coding or generate with AI.

First, let's create a sample CSV file named `sample_data.csv` that we can use for demonstration. It will contain some numerical data.

```
[1] import pandas as pd

# Create a sample DataFrame
data = {'Category': ['A', 'B', 'A', 'C', 'B', 'A', 'C'],
        'Value': [10, 25, 15, 30, 20, 12, 35],
        'Quantity': [1, 5, 2, 8, 3, 2, 7]}
df_sample = pd.DataFrame(data)

# Save the DataFrame to a CSV file
df_sample.to_csv('sample_data.csv', index=False)

print(f"Created 'sample_data.csv' with sample content.")
print(df_sample)
```

	Category	Value	Quantity
0	A	10	1
1	B	25	5
2	A	15	2
3	C	30	8
4	B	20	3
5	A	12	2
6	C	35	7

Now, let's read the `sample_data.csv` file into a pandas DataFrame and calculate the mean, min, and max for a numerical column, for example, the 'Value' column.

Gemini

read a CSV file and calculate mean, min, and max.

Okay, I can help with that! Here's the Python code to read a CSV file, then calculate and display the mean, minimum, and maximum values for a numerical column. I'll include a step to create a sample CSV first.

Accept and run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

Variables Terminal

Search

10:35 AM 1/21/2026

colab.research.google.com/drive/1EglUz_7zIKVDutF3FTO9ocqGpnoXbgA1#scrollTo=fcc15417

Untitled19.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

Files

- ..
- config
- sample_data
- machine-readable-business-em...
- sample.txt
- sample_data.csv

Disk 86.58 GB available

Now, let's read the `sample_data.csv` file into a pandas DataFrame and calculate the mean, min, and max for a numerical column, for example, the 'Value' column.

```
[1] import pandas as pd

# Specify the path to your CSV file
csv_file_path = 'sample_data.csv' # Change this to your actual CSV file path

try:
    # Read the CSV file into a pandas DataFrame
    df = pd.read_csv(csv_file_path)

    print(f"Successfully loaded '{csv_file_path}'. Here's the first few rows:")
    display(df.head())

    # Select a numerical column to calculate statistics (e.g., 'Value')
    # You can change 'Value' to any numerical column in your CSV.
    numerical_column = 'Value'

    if numerical_column in df.columns and pd.api.types.is_numeric_dtype(df[numerical_column]):
        # Calculate mean, min, and max
        mean_value = df[numerical_column].mean()
        min_value = df[numerical_column].min()
        max_value = df[numerical_column].max()

        print(f"\nStatistics for column '{numerical_column}':")
        print(f"Mean: {mean_value:.2f}")
        print(f"Min: {min_value:.2f}")
        print(f"Max: {max_value:.2f}")
    else:
        print(f"\nError: Column '{numerical_column}' not found or is not numeric in the CSV file.")
        print("Available columns: ", df.columns.tolist())
```

Gemini

read a CSV file and calculate mean, min, and max.

Okay, I can help with that! Here's the Python code to read a CSV file, then calculate and display the mean, minimum, and maximum values for a numerical column. I'll include a step to create a sample CSV first.

Accept and run Accept Cancel

What can I help you build?

Gemini 2.5 Flash

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

Variables Terminal

Search

10:35 AM 1/21/2026

The screenshot shows a Google Colab notebook with the following content:

```

import pandas as pd

def read_csv_file(csv_file_path):
    try:
        df = pd.read_csv(csv_file_path)
        print(f"Mean: {df['Value'].mean():.2f}")
        print(f"Min: {df['Value'].min():.2f}")
        print(f"Max: {df['Value'].max():.2f}")
    except:
        print(f"Error: Column '{numerical_column}' not found or is not numeric in the CSV file.")
        print("Available columns: ", df.columns.tolist())

except FileNotFoundError:
    print(f"Error: The file '{csv_file_path}' was not found. Please check the path.")
except Exception as e:
    print(f"An error occurred: {e}")

# Successfully loaded 'sample_data.csv'. Here's the first few rows:

```

	Category	Value	Quantity
0	A	10	1
1	B	25	5
2	A	15	2
3	C	30	8
4	B	20	3

```

Statistics for column 'Value':
Mean: 21.00
Min: 10.00
Max: 35.00

```

The Gemini chat interface on the right shows a conversation about reading a CSV file and calculating statistics.

Task 4: Sorting Lists – Manual vs Built-in

❖ Scenario:

You are reviewing algorithm choices for efficiency.

❖ Task:

Use Gemini to generate:

- Bubble sort
- Python's built-in sort()
- Compare both implementations.

❖ Expected Output:

- Two versions of code
- Short comparison

New Tab x NZ CSV files x +

🔍 ☆

🔧

📄

⌵

⋮

💬 ⚙️

👤 Share

✓ RAM

Disk

🗑️ ⋮

Gemini

🔍 ⋮ ✕

read a CSV file and calculate mean, min, and max.

✦ Okay, I can help with that! Here's the Python code to read a CSV file, then calculate and display the mean, minimum, and maximum values for a numerical column. I'll include a step to create a sample CSV first.

👍 💬

▶ Accept and run ✓ Accept ✕ Cancel

give a python code for bubble sort

+

Gemini 2.5 Flash ▼ ▶

Gemini can make mistakes, so double-check it and use code with caution. [Learn more](#)

✓ 10:35 📄 Python 3

ENC 10:38 AM

colab.research.google.com/drive/1EglUz_7zIKVDutF3FTO9ocqGpnoXbgA1#scrollTo=5a350147

Untitled19.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Files

- ..
- .config
- sample_data
- machine-readable-business-em...
- sample.txt
- sample_data.csv

86.58 GB available

Gemini

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Swap elements
    return arr

# Example usage:
my_list = [64, 34, 25, 12, 22, 11, 90]
print("Original list:", my_list)

sorted_list = bubble_sort(my_list)
print("Sorted list:", sorted_list)

another_list = [5, 1, 4, 2, 8]
print("Original list:", another_list)

sorted_another_list = bubble_sort(another_list)
print("Sorted list:", sorted_another_list)
```

Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list: [11, 12, 22, 25, 34, 64, 90]
Original list: [5, 1, 4, 2, 8]
Sorted list: [1, 2, 4, 5, 8]

Gemini

give a python code for bub

Certainly! Here's a Python bubble sort algorithm, alor

Sources

Accept and run

What can I help you build?

Gemini can make mistakes, so doubt

use python built in function for the same question sort()

★ You're right! Python has very efficient built-in sorting mechanisms. Here's how you can sort lists using `list.sort()` (which sorts the list in-place) and `sorted()` (which returns a new sorted list).

built-in function returns a new sorted list without changing the original.

Gemini

```
# Using list.sort() (sorts in-place)
list_to_sort_in_place = [64, 34, 25, 12, 22, 11, 90]
print("Original list (for in-place sort):", list_to_sort_in_place)
list_to_sort_in_place.sort()
print("Sorted list (in-place):", list_to_sort_in_place)

print("\n---")

# Using sorted() (returns a new sorted list)
original_list_for_sorted = [5, 1, 4, 2, 8]
print("Original list (for sorted() function):", original_list_for_sorted)
new_sorted_list = sorted(original_list_for_sorted)
print("New sorted list (from sorted() function):", new_sorted_list)
print("Original list remains unchanged:", original_list_for_sorted)
```

Original list (for in-place sort): [64, 34, 25, 12, 22, 11, 90]
Sorted list (in-place): [11, 12, 22, 25, 34, 64, 90]

Original list (for sorted() function): [5, 1, 4, 2, 8]
New sorted list (from sorted() function): [1, 2, 4, 5, 8]
Original list remains unchanged: [5, 1, 4, 2, 8]

use i

You'r mecl (whic new

Wha

Gemini r

Short comparison

Bubble Sort: It's a simple sorting algorithm, but generally inefficient for large datasets ($O(n^2)$ complexity). It sorts by repeatedly stepping through the list, comparing adjacent elements and swapping them.

Python's `list.sort()` and `sorted()`: These are highly optimized and implemented in C, using Timsort (a hybrid stable sorting algorithm with $O(n \log n)$ average and worst-case complexity). They are significantly faster and more memory-efficient for most use cases. `list.sort()` sorts in-place, while `sorted()` returns a new sorted list.

Assignment-3.4

Task 1: Zero-shot Prompt – Fibonacci Series Generator

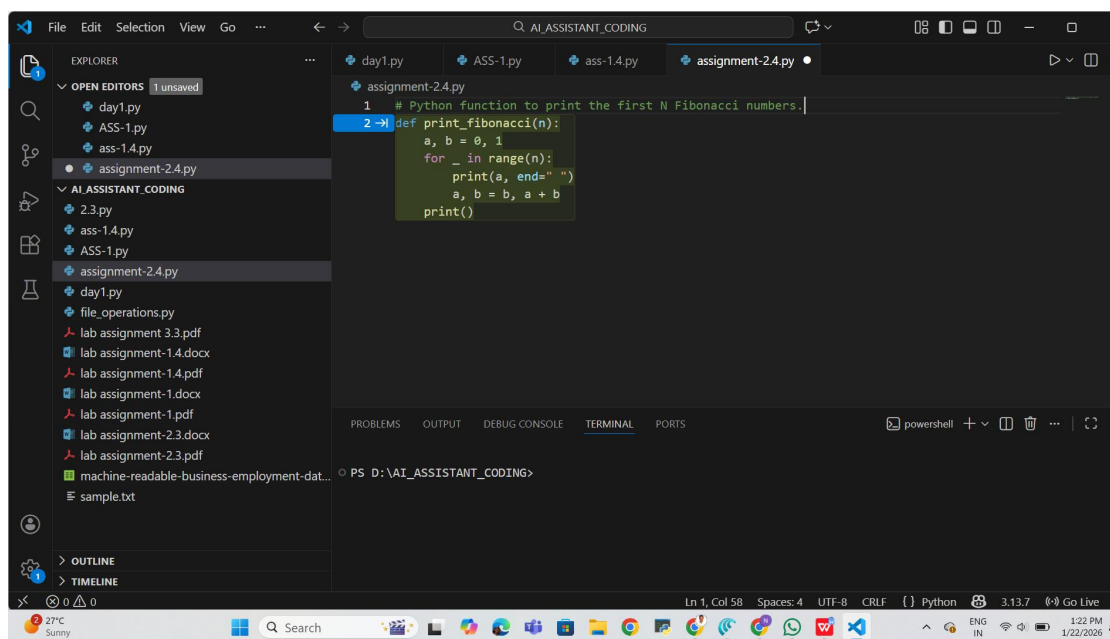
Task Description #1

- Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers.

Expected Output #1

- A complete Python function generated by Copilot without any example provided.
- Correct output for sample input N = 7 → 0 1 1 2 3 5 8
- Observation on how Copilot understood the instruction with zero Context

PROMPT:



CODE

```
def fibonacci_sequence(n):

    Generates the first N Fibonacci numbers.

    Args:
        n: Integer, number of Fibonacci numbers to generate

    Returns:
        List of first N Fibonacci numbers

    fib_sequence = []
    a, b = 0, 1
    for _ in range(n):
        fib_sequence.append(a)
        a, b = b, a + b
```

```

13     a, b = 0, 1
14     for _ in range(n):
15         fib_sequence.append(a)
16         a, b = b, a + b
17     return fib_sequence
18     for i in range(2, int(n ** 0.5) + 1):
19         if n % i == 0:
20             return False
21
22     return True
23     return True
24 # Test the function
25 num = int(input("Enter the number of Fibonacci numbers to generate: "))
26 fib_numbers = fibonacci_sequence(num)
27 print(f"The first {num} Fibonacci numbers are: {fib_numbers}")
28

```

OUTPUT :

```

PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
Enter the number of Fibonacci numbers to generate: 7
The first 7 Fibonacci numbers are: [0, 1, 1, 2, 3, 5, 8]
PS D:\AI_ASSISTANT_CODING>

```

Task 2: One-shot Prompt – List Reversal Function

Task Description #2

• Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.

PROMPT:

```

26 fib_numbers = fibonacci_sequence(num)
27 print(f"The first {num} Fibonacci numbers are: {fib_numbers}")
28
29 # Reverse a list by using an example of your choice.

```

CODE:

```

29 #Reverse a list by using an example of your choice
30 """def reverse_list(input_list):
31
32     Reverses the given list.
33
34     Args:
35         input_list: List of elements to be reversed
36
37     Returns:
38         List: Reversed list
39     return input_list[::-1]
40 # Test the reverse_list function
41 sample_list = [1, 2, 3]
42 reversed_list = reverse_list(sample_list)
43 print(f"Original list: {sample_list}")
44 print(f"Reversed list: {reversed_list}")
45 """

```

OUTPUT:

```

PS D:\VAI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
Original list: [1, 2, 3, 4, 5]
Reversed list: [5, 4, 3, 2, 1]
PS D:\VAI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
Original list: [1, 2, 3]
Reversed list: [3, 2, 1]
PS D:\VAI_ASSISTANT_CODING>

```

Adding **just one clear example** helps Copilot quickly infer:

what the function should do

the expected input and output format

This improves **accuracy and reduces ambiguity**, even with a minimal prompt.

Task 3: Few-shot Prompt – String Pattern Matching

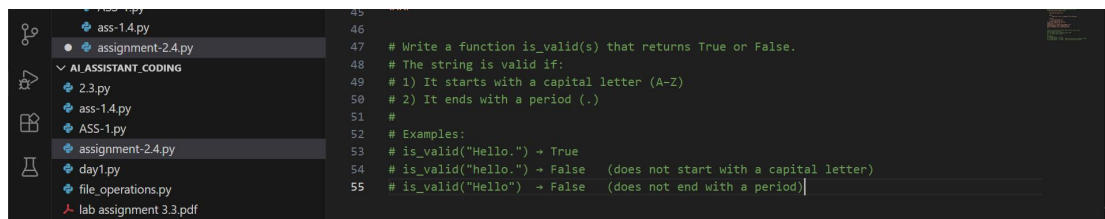
Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

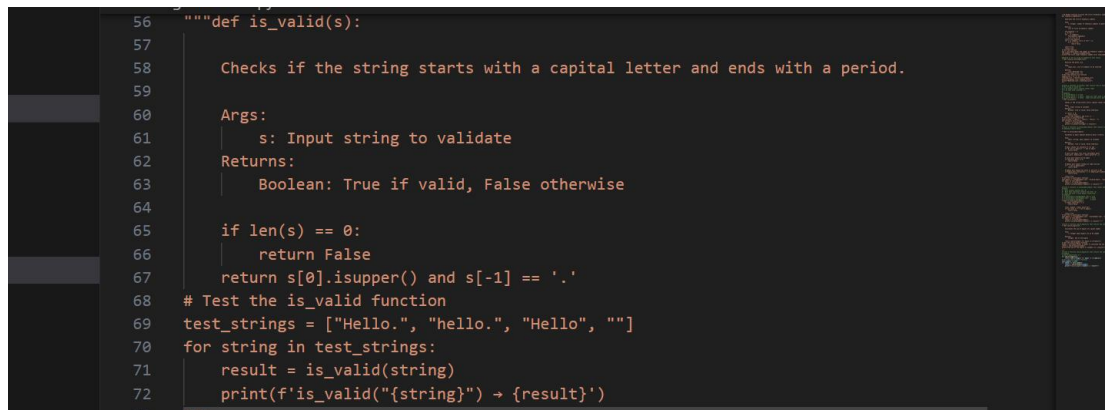
- A function `is_valid()` that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.

PROMPT:



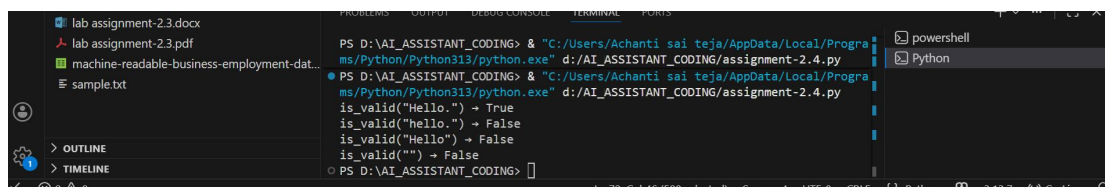
```
45 # Write a function is_valid(s) that returns True or False.
46 # The string is valid if:
47 # 1) It starts with a capital letter (A-Z)
48 # 2) It ends with a period (.)
49 #
50 # Examples:
51 # is_valid("Hello.") → True
52 # is_valid("hello.") → False (does not start with a capital letter)
53 # is_valid("Hello") → False (does not end with a period)
```

CODE:



```
56 """def is_valid(s):
57     Checks if the string starts with a capital letter and ends with a period.
58
59     Args:
60         s: Input string to validate
61     Returns:
62         Boolean: True if valid, False otherwise
63
64     if len(s) == 0:
65         return False
66     return s[0].isupper() and s[-1] == '.'
67
68 # Test the is_valid function
69 test_strings = ["Hello.", "hello.", "Hello", ""]
70 for string in test_strings:
71     result = is_valid(string)
72     print(f'is_valid("{string}") → {result}')
```

OUTPUT:



```
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py
is_valid("Hello.") → True
is_valid("hello.") → False
is_valid("Hello") → False
is_valid("") → False
```

Providing **multiple examples in comments** helps Copilot clearly understand:

what counts as valid

what should be rejected

This leads to **more accurate and relevant code generation**.

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

Expected Output #4

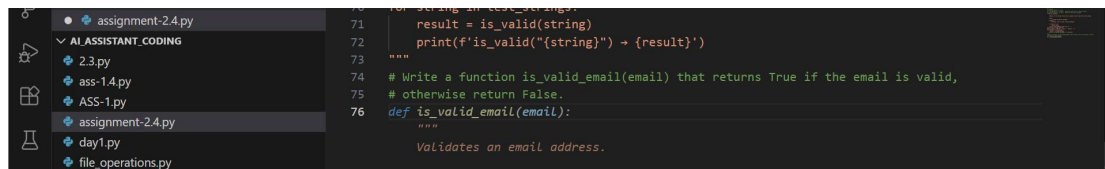
- Compare both outputs:

Zero-shot may result in basic or generic validation.

Few-shot gives detailed and specific logic (e.g., @ and domain checking).

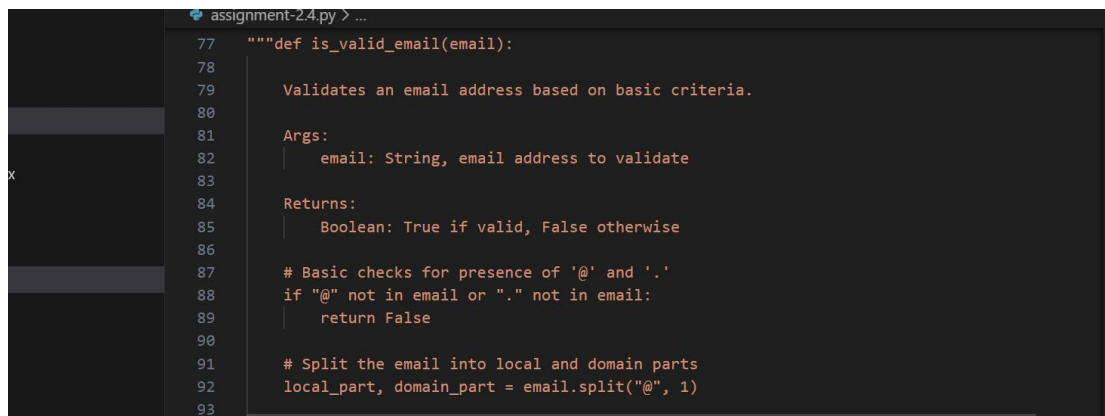
- Submit both code versions and note how few-shot improves reliability.

PROMPT:

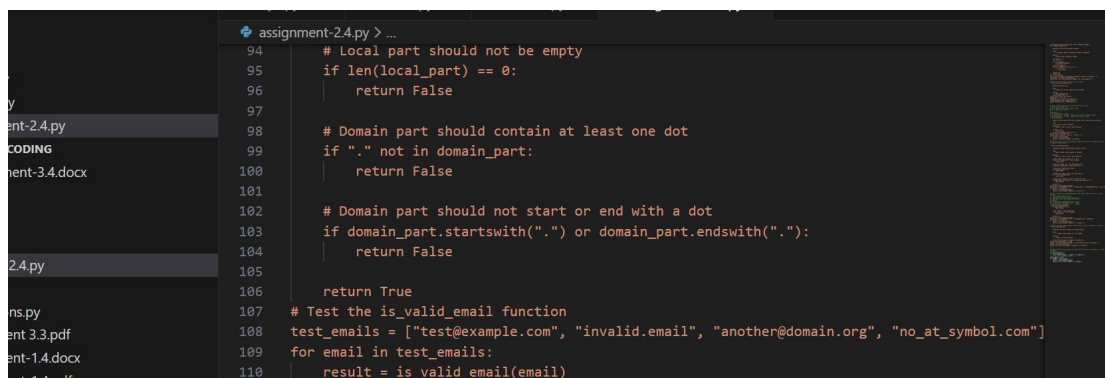


```
70 for string in test_strings:
71     result = is_valid(string)
72     print(f'is_valid("{string}") -> {result}')
73 """
74 # Write a function is_valid_email(email) that returns True if the email is valid,
75 # otherwise return False.
76 def is_valid_email(email):
77     """
78     Validates an email address.
```

CODE:

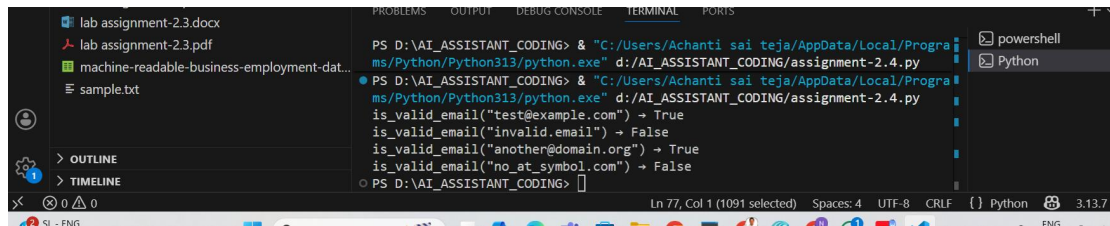


```
77 """def is_valid_email(email):
78
79     Validates an email address based on basic criteria.
80
81     Args:
82         email: String, email address to validate
83
84     Returns:
85         Boolean: True if valid, False otherwise
86
87     # Basic checks for presence of '@' and '.'
88     if "@" not in email or "." not in email:
89         return False
90
91     # Split the email into local and domain parts
92     local_part, domain_part = email.split("@", 1)
93
```



```
94     # Local part should not be empty
95     if len(local_part) == 0:
96         return False
97
98     # Domain part should contain at least one dot
99     if "." not in domain_part:
100         return False
101
102     # Domain part should not start or end with a dot
103     if domain_part.startswith(".") or domain_part.endswith("."):
104         return False
105
106     return True
107
108 # Test the is_valid_email function
109 test_emails = ["test@example.com", "invalid.email", "another@domain.org", "no_at_symbol.com"]
110 for email in test_emails:
111     result = is_valid_email(email)
```

OUTPUT:



PROMPT:



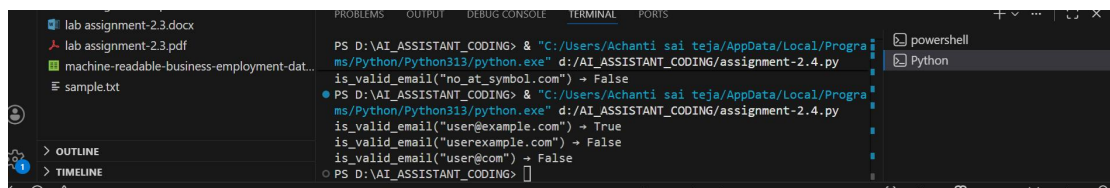
CODE:

```
"""def is_valid_email(email):
    if email.count("@") != 1:
        return False

    local, domain = email.split("@")
    if not local or "." not in domain:
        return False

    return True
# Test the is_valid_email function
test_emails = ["user@example.com", "userexample.com", "user@com"]
for email in test_emails:
    result = is_valid_email(email)
    print(f'is_valid_email("{email}") -> {result}')
```

OUTPUT:



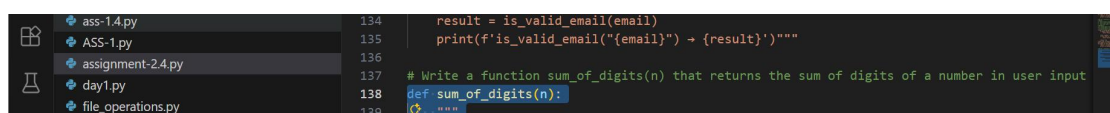
Few-shot prompting **guides Copilot's reasoning** by showing:

valid vs invalid patterns

edge cases

This results in **more reliable, accurate, and meaningful code** compared to zero-shot prompts.

PROMPT:



CODE:

```
def sum_of_digits(n):  
    """  
    Calculates the sum of digits of a given number.  
  
    Args:  
        n: Integer whose digits are to be summed  
  
    Returns:  
        Integer: Sum of the digits  
    """  
    return sum(int(digit) for digit in str(abs(n)))  
# Test the sum_of_digits function  
number = int(input("Enter a number to calculate the sum of its digits: "))  
result = sum_of_digits(number)  
print(f"The sum of the digits of {number} is: {result}")
```

OUTPUT:

```
sample.txt  
is_valid_email("user@example.com") → False  
is_valid_email("user@com") → False  
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py  
Enter a number to calculate the sum of its digits: 123  
The sum of the digits of 123 is: 6  
PS D:\AI_ASSISTANT_CODING>
```

PROMPT:

```
day1.py  
ASS-1.py  
ass-1.4.py  
assignment-2.4.py  
154 """  
155 # Write a function sum_of_digits(n) that returns the sum of all digits in a number.  
156 # Example:  
157 # sum_of_digits(123) → 6
```

CODE:

```
ASS-1.py  
ass-1.4.py  
assignment-2.4.py  
AI ASSISTANT CODING  
2.3.py  
ass-1.4.py  
ASS-1.py  
assignment-2.4.py  
day1.py  
file_operations.py  
lab assignment 3.3.pdf  
155 # Write a function sum_of_digits(n) that returns the sum of all digits in a number.  
156 # Example:  
157 # sum_of_digits(123) → 6  
158 def sum_of_digits(n):  
159     return sum(int(digit) for digit in str(abs(n)))  
160 # Test the sum_of_digits function  
161 test_numbers = [123]  
162 for number in test_numbers:  
163     result = sum_of_digits(number)  
164     print(f"sum_of_digits({number}) → {result}")  
165
```

OUTPUT:

```
sample.txt  
Enter a number to calculate the sum of its digits: 123  
The sum of the digits of 123 is: 6  
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" d:/AI_ASSISTANT_CODING/assignment-2.4.py  
sum_of_digits(123) → 6  
PS D:\AI_ASSISTANT_CODING>
```


Assignment - 6.3

Date:4/2/26

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

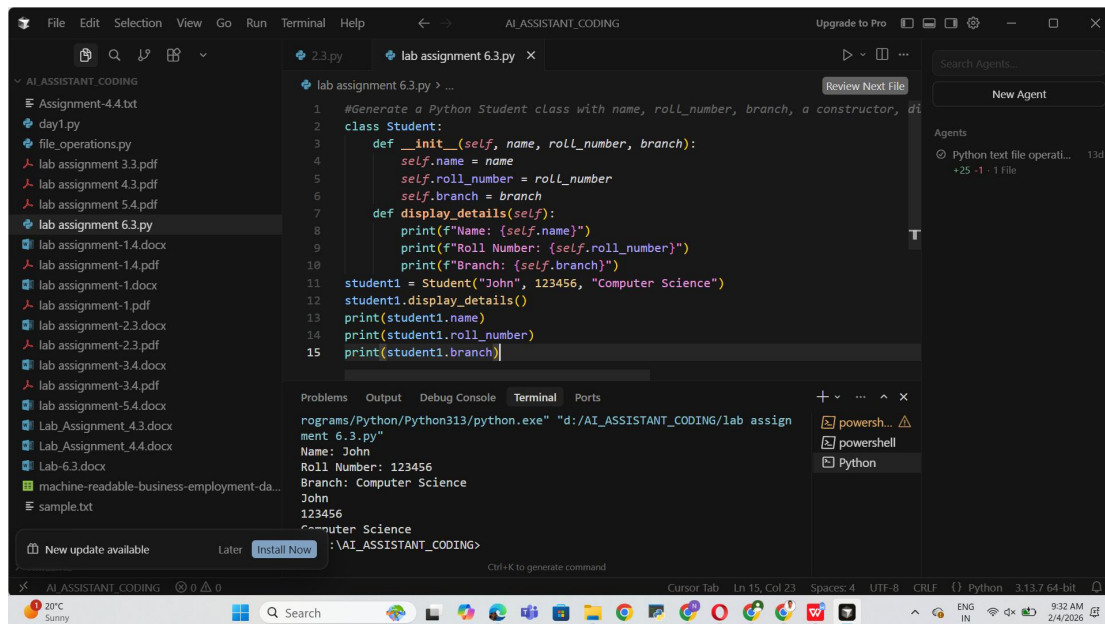
Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code

Prompt:

Generate a Python Student class with name, roll_number, branch, a constructor, `display_details()` method, and sample object creation with output.

Code

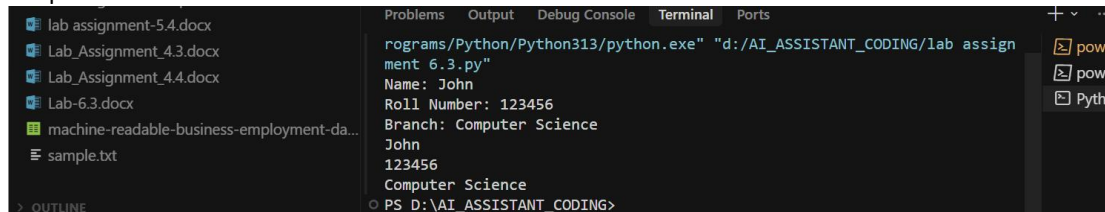


```
1 #Generate a Python Student class with name, roll_number, branch, a constructor, di
2 class Student:
3     def __init__(self, name, roll_number, branch):
4         self.name = name
5         self.roll_number = roll_number
6         self.branch = branch
7     def display_details(self):
8         print(f"Name: {self.name}")
9         print(f"Roll Number: {self.roll_number}")
10        print(f"Branch: {self.branch}")
11 student1 = Student("John", 123456, "Computer Science")
12 student1.display_details()
13 print(student1.name)
14 print(student1.roll_number)
15 print(student1.branch)]
```

Problems Output Debug Console Terminal Ports

rograms/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assign
ment 6.3.py"
Name: John
Roll Number: 123456
Branch: Computer Science
John
123456
Computer Science
:VAI_ASSISTANT_CODING>

Output:



```
rograms/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assign  
ment 6.3.py"  
Name: John  
Roll Number: 123456  
Branch: Computer Science  
John  
123456  
Computer Science  
PS D:\VAI_ASSISTANT_CODING>
```

Overall explanation:

This program defines a `Student` class to represent student details like name, roll number, and branch.

The constructor (`__init__`) initializes these values when a new object is created.

The `display_details()` method prints all the student information in a readable format.

Finally, a `Student` object is created and its data is accessed using both the method and direct attributes.

Task 2:

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.

- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Prompt1:

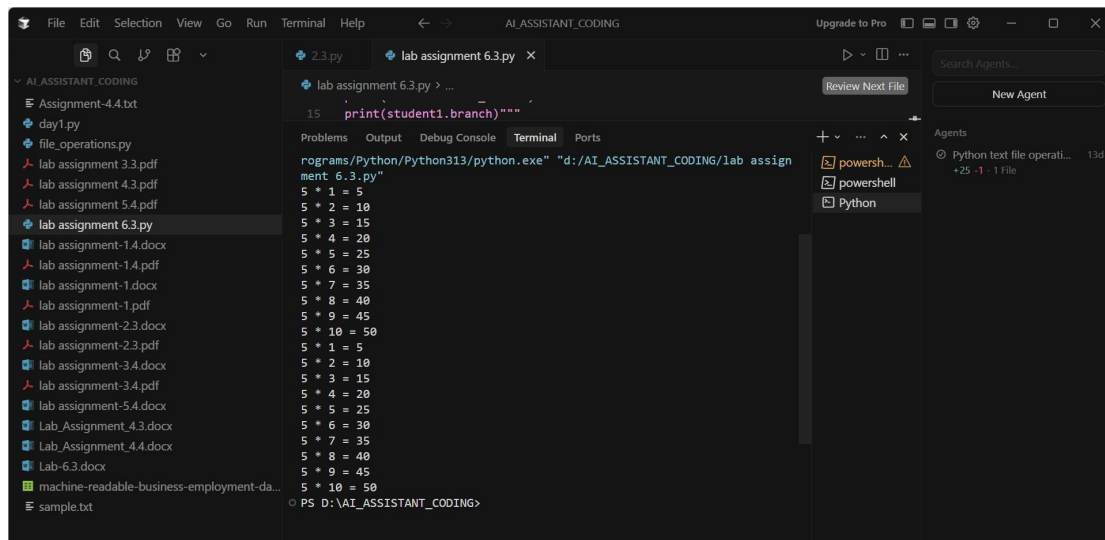
Write a Python function to print the first 10 multiples of a given number using a for loop with sample input and output.

Prompt2:

Generate the same Python function to print the first 10 multiples of a number using a while loop and explain the logic
code:

```
def print_multiples(number):
    for i in range(1, 11):
        print(f"{number} * {i} = {number * i}")
print_multiples(5)
#Write a Python function to print the first 10 multiples of a given n
def print_multiples(number):
    i = 1
    while i <= 10:
        print(f"{number} * {i} = {number * i}")
        i += 1
print_multiples(5)
#Write a Python function to print the first 10 multiples of a given n
```

Output:



Description:

Both functions correctly print the first 10 multiples of the given number in a neat format.

The loop control (range(1,11) in for and $i \leq 10$ in while) is used properly.

Using print_multiples(5) as sample input clearly demonstrates the expected output.

Task3:

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work

Prompt1:

Create a Python function `classify_age(age)` using nested if-elif-else to classify child, teenager, adult, and senior with examples.

Prompt2:

Rewrite the age classification program using a simplified or dictionary-based conditional approach and explain it.

Code:

```
28 #create a python function classify_age(age) using nested if-elif-else to classify child, teenager, adult, and senior
29 def classify_age(age):
30     if age < 13:
31         return "child"
32     elif age < 18:
33         return "teenager"
34     elif age < 60:
35         return "adult"
36     else:
37         return "senior"
38 print(classify_age(10))
39 print(classify_age(15))
40 print(classify_age(20))
41 print(classify_age(65))
42 #Rewrite the age classification program using a simplified or dictionary-based conditional approach and explain it
43 age_classifications = {
44     "child": range(0, 13),
45     "teenager": range(13, 18),
46     "adult": range(18, 60),
47     "senior": range(60, 100)
48 }
49 def classify_age(age):
50     for classification, age_range in age_classifications.items():
51         if age in age_range:
52             return classification
53     return "unknown"
54 print(classify_age(10))
55 print(classify_age(15))
56 print(classify_age(20))
57 print(classify_age(65))
```

Output:

```
child
teenager
adult
child
teenager
adult
senior
```

Description:

Instead of multiple if–elif conditions, age groups are stored in a dictionary where each key is a label and each value is a range of ages.

The function loops through the dictionary and checks which range the given age belongs to.

As soon as a match is found, the corresponding classification is returned.

This approach is more readable, easier to update, and avoids long conditional chains.

Task 4:

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.
- Correct output for sample inputs.
- Explanation and comparison of different approaches.

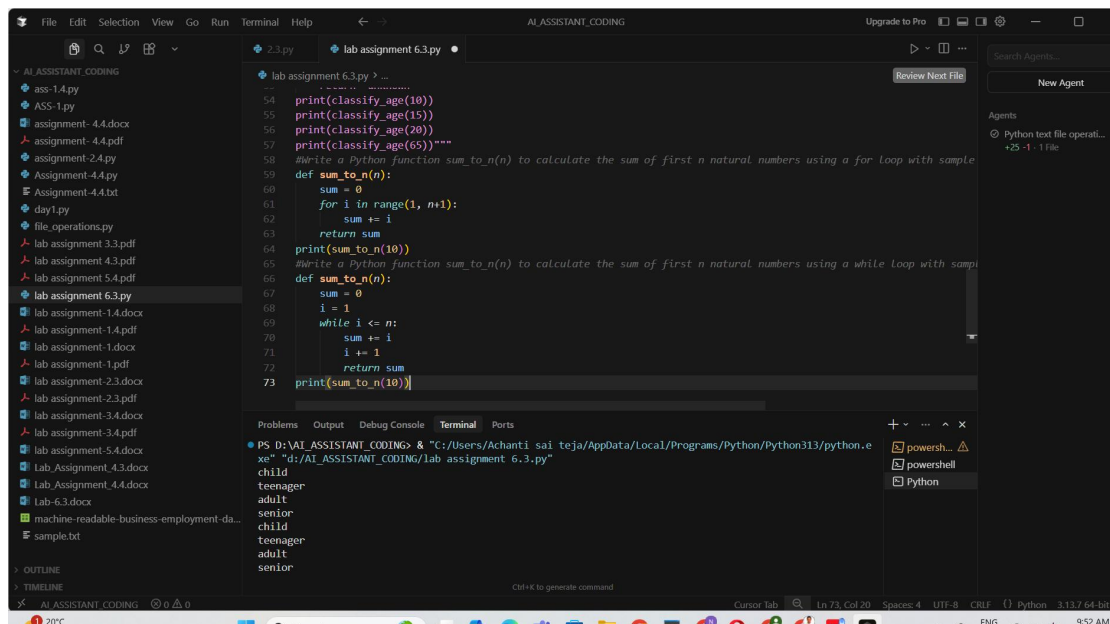
Prompt1:

Write a Python function `sum_to_n(n)` to calculate the sum of first n natural numbers using a for loop with sample output.

Prompt2:

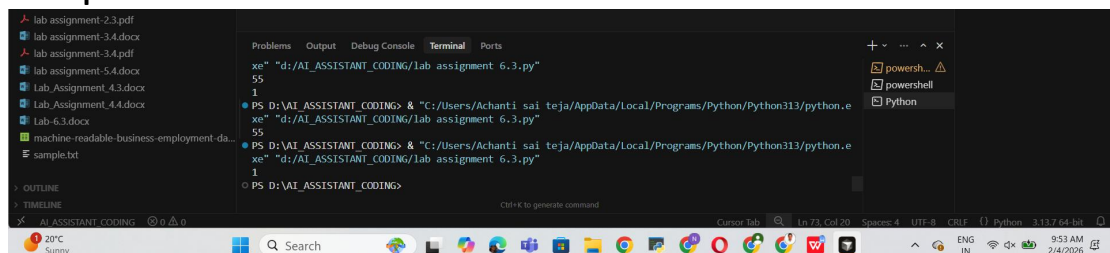
Generate an alternative implementation of `sum_to_n(n)` using a while loop or mathematical formula and compare approaches.

Code:



```
54 print(classify_age(10))
55 print(classify_age(15))
56 print(classify_age(20))
57 print(classify_age(65))"""
58 #Write a Python function sum_to_n(n) to calculate the sum of first n natural numbers using a for loop with sample
59
60 def sum_to_n(n):
61     sum = 0
62     for i in range(1, n+1):
63         sum += i
64     return sum
65
66 print(sum_to_n(10))
67
68 #Write a Python function sum_to_n(n) to calculate the sum of first n natural numbers using a while loop with sample
69
70 def sum_to_n(n):
71     sum = 0
72     i = 1
73     while i <= n:
74         sum += i
75         i += 1
76     return sum
77
78 print(sum_to_n(10))
```

Output:



```
xe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
55
1
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe"
xe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
55
1
PS D:\AI_ASSISTANT_CODING>
```


Description:

Both **age classification approaches** (if-elif and dictionary-based) are implemented properly and give the same correct results (`child`, `teenager`, `adult`, `senior`). The `sum_to_n(n)` function using a **for loop** correctly adds numbers from 1 to `n`. The second `sum_to_n(n)` using a **while loop** is also logically correct and produces the same output. One small improvement: since both functions have the **same name**, the second definition overrides the first—this is fine for learning, but in practice you'd use different names like `sum_to_n_for` and `sum_to_n_while`.

Task 5:

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as `deposit()`, `withdraw()`,

and `check_balance()`.

- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation

Prompt:

Generate a Python BankAccount class with `deposit()`, `withdraw()`, `check_balance()` methods, sample usage, and updated balance output.

Code:

The screenshot shows a code editor with a file explorer on the left containing various assignment files. The main editor displays a Python script named 'lab assignment 6.3.py'. The script defines a `BankAccount` class with methods `__init__`, `deposit`, `withdraw`, and `check_balance`. It also includes sample usage code that creates a `BankAccount` object and performs deposit and withdrawal operations. The terminal at the bottom shows the execution of the script, displaying the output of each method call.

```
73 print(sum_to_n(10))"""
74 #Generate a Python BankAccount class with deposit(), withdraw(), check_balance() methods, sample usage, and update
75 class BankAccount:
76     def __init__(self, initial_balance=0):
77         self.balance = initial_balance
78     def deposit(self, amount):
79         self.balance += amount
80         return f"Deposited {amount}. New balance: {self.balance}"
81     def withdraw(self, amount):
82         if amount > self.balance:
83             return "Insufficient balance"
84         self.balance -= amount
85         return f"Withdrew {amount}. New balance: {self.balance}"
86     def check_balance(self):
87         return f"Current balance: {self.balance}"
88 account = BankAccount(1000)
89 print(account.deposit(500))
90 print(account.withdraw(200))
91 print(account.check_balance())
92
```

Terminal Output:

```
55
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
1
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
Deposited 500. New balance: 1500
Withdrew 200. New balance: 1300
Current balance: 1300
PS D:\AI_ASSISTANT_CODING>
```

Output:

This screenshot is a close-up of the terminal window from the previous image. It shows the command prompt where the script was executed, followed by the output of the script, which includes the deposit and withdrawal results and the final balance check.

```
55
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
1
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python313/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 6.3.py"
Deposited 500. New balance: 1500
Withdrew 200. New balance: 1300
Current balance: 1300
PS D:\AI_ASSISTANT_CODING>
```

Description:

The `BankAccount` class uses a **constructor** to initialize the account with an initial balance. The `deposit()` method correctly adds money to the balance and returns a confirmation message. The `withdraw()` method safely checks for **insufficient balance** before deducting the amount. The `check_balance()` method neatly displays the current balance, and the sample object usage proves all methods work as expected.

Assignment-4.4

1. Sentiment Classification for Customer Reviews

Scenario:

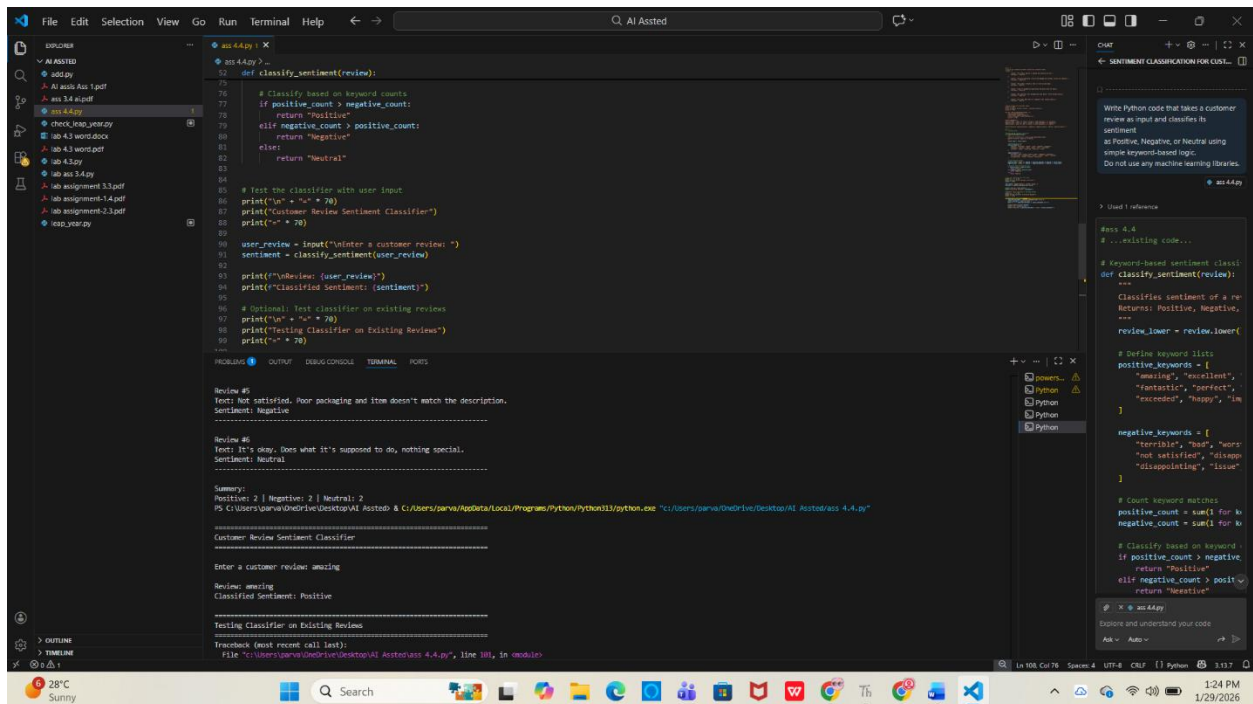
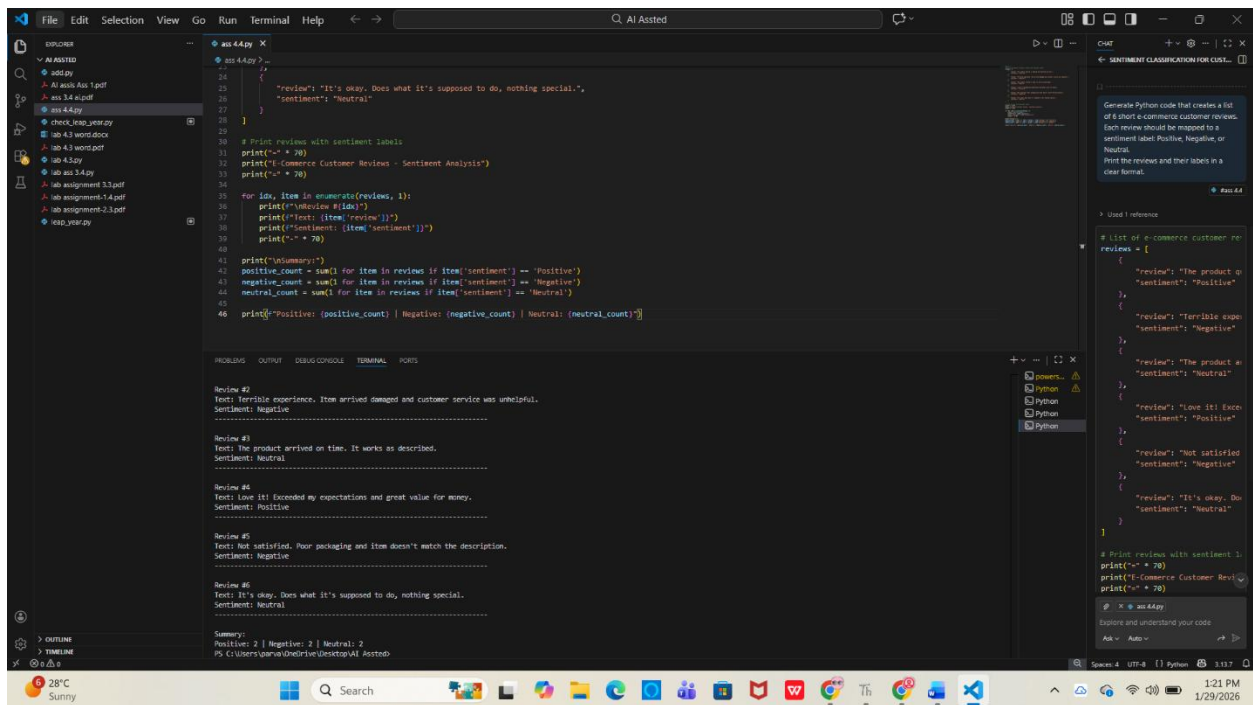
An e-commerce platform wants to analyze customer reviews and classify

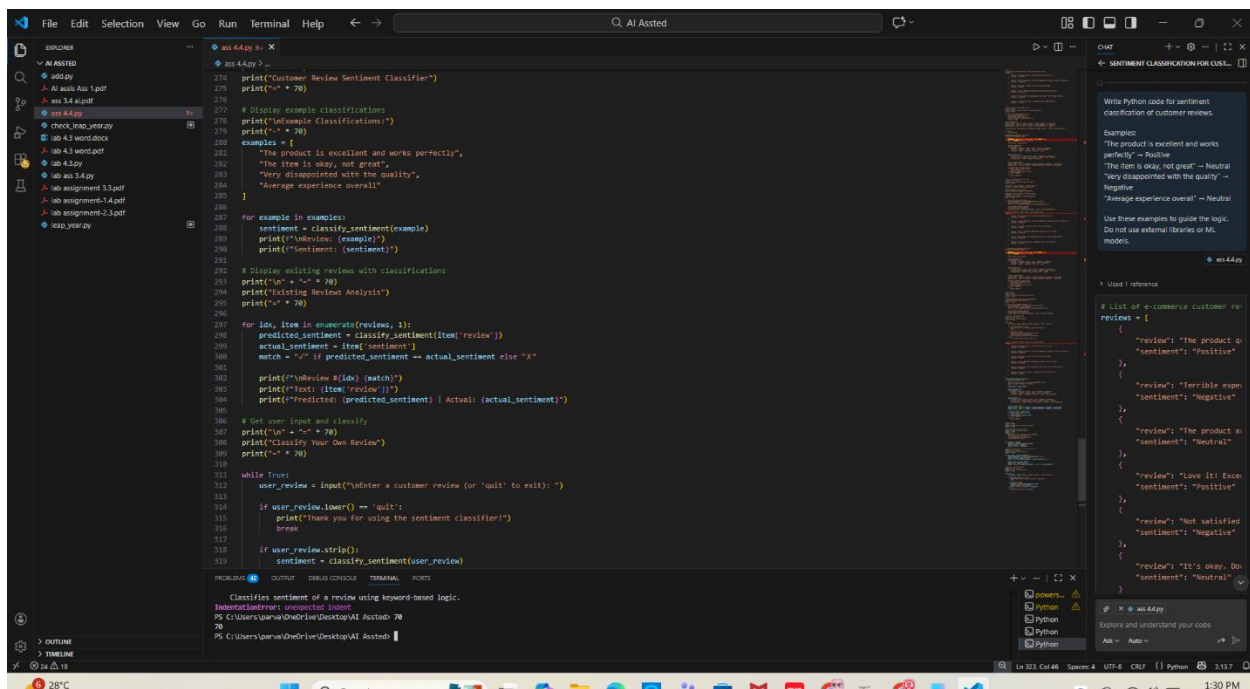
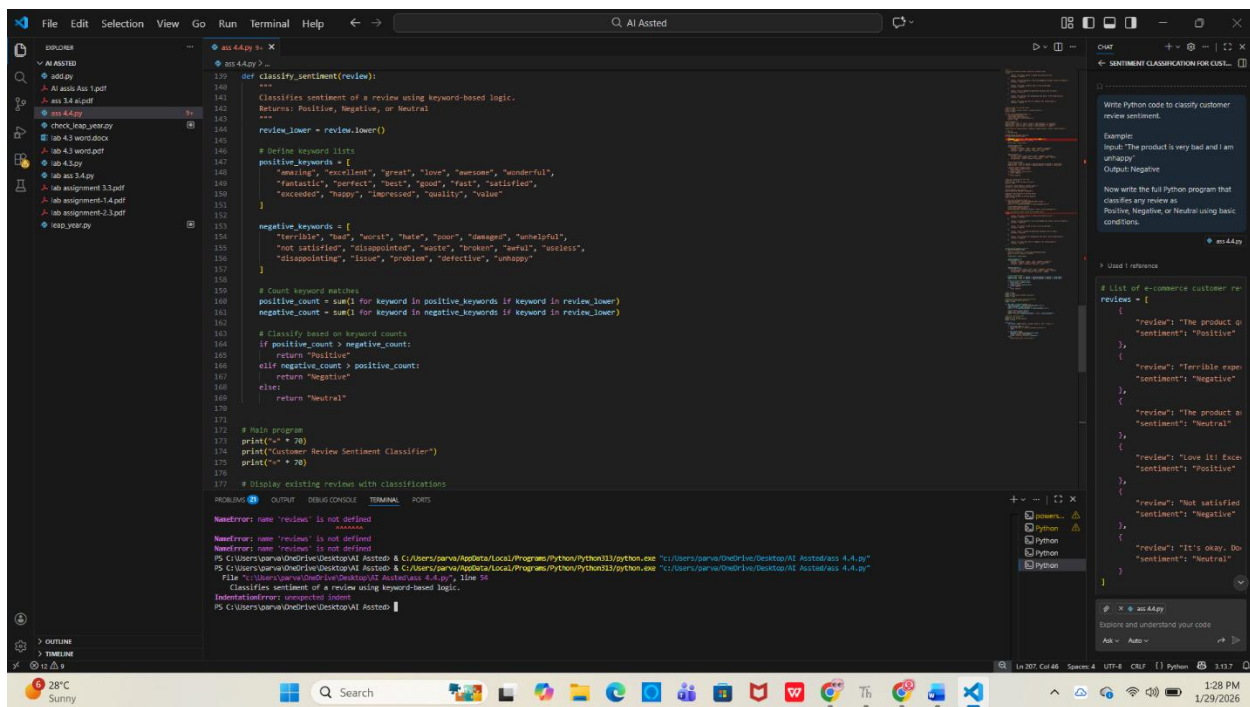
Week2

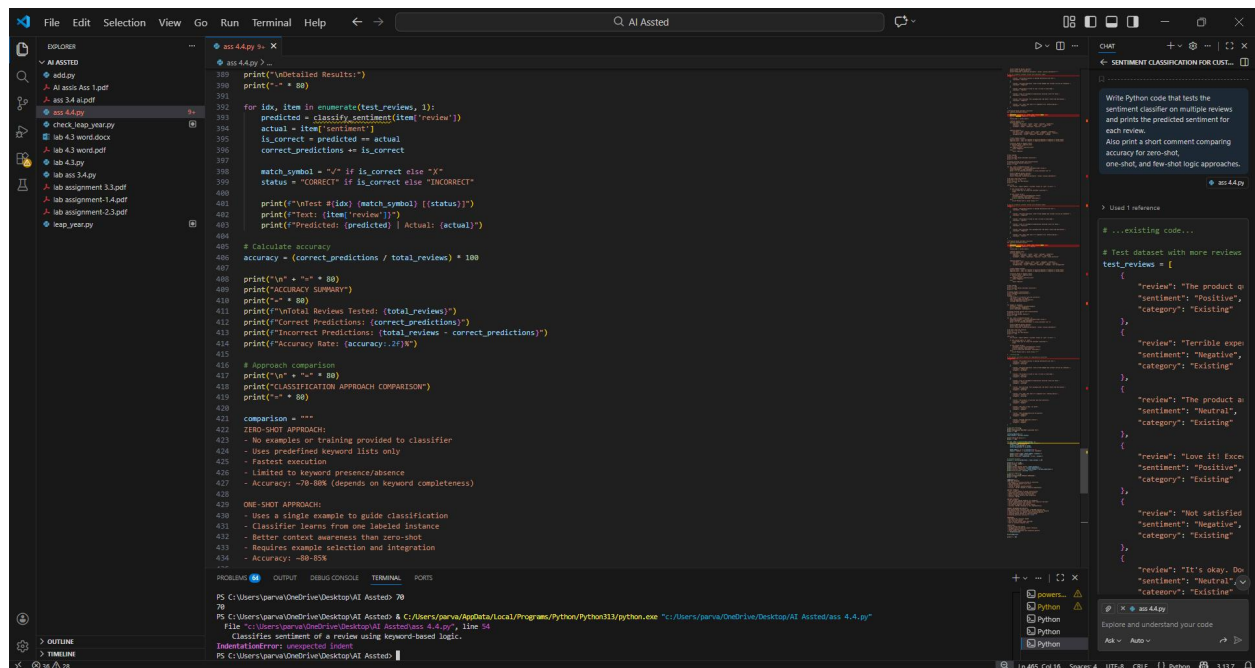
them into Positive, Negative, or Neutral sentiments using prompt engineering.

Tasks:

- a) Prepare 6 short customer reviews mapped to sentiment labels.
- b) Design a Zero-shot prompt to classify sentiment.
- c) Design a One-shot prompt with one labeled example.
- d) Design a Few-shot prompt with 3–5 labeled examples.
- e) Compare the outputs and discuss accuracy differences.







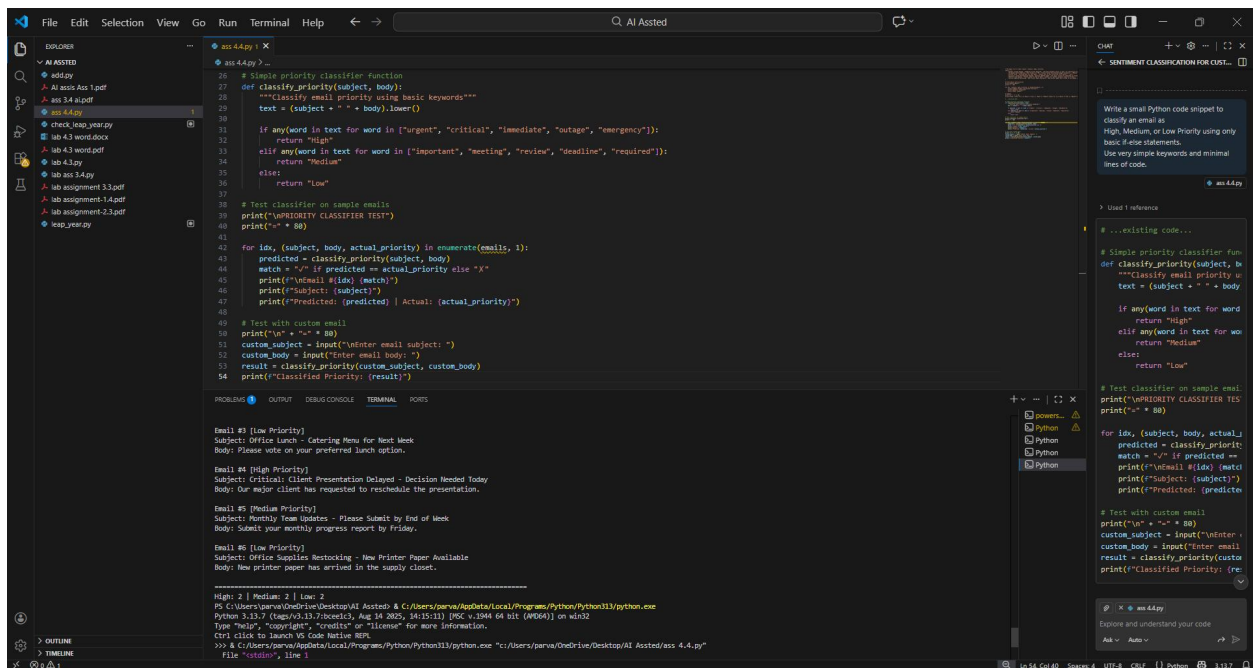
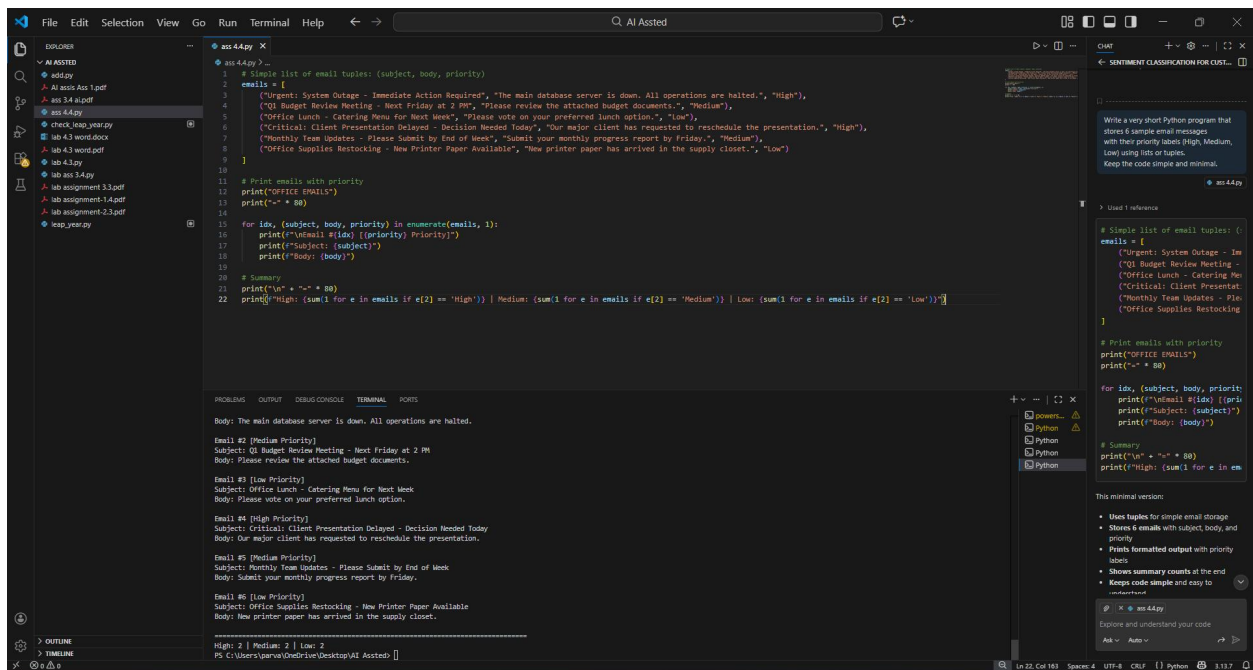
2. Email Priority Classification

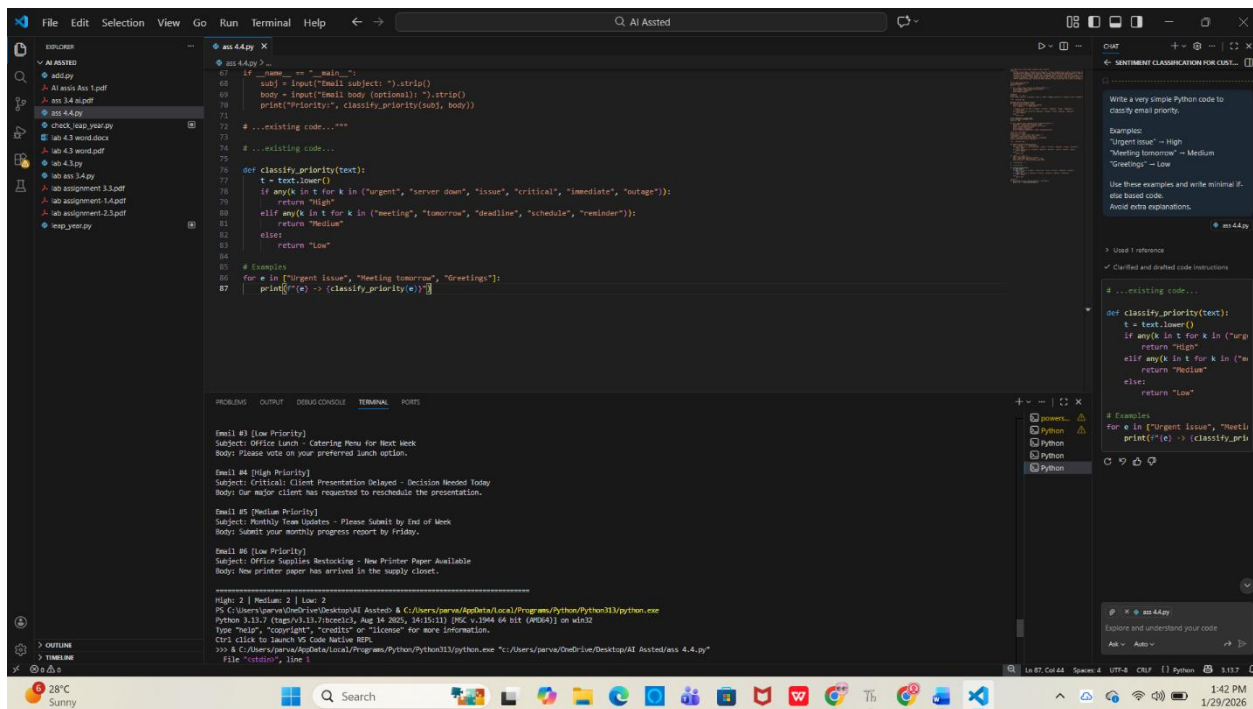
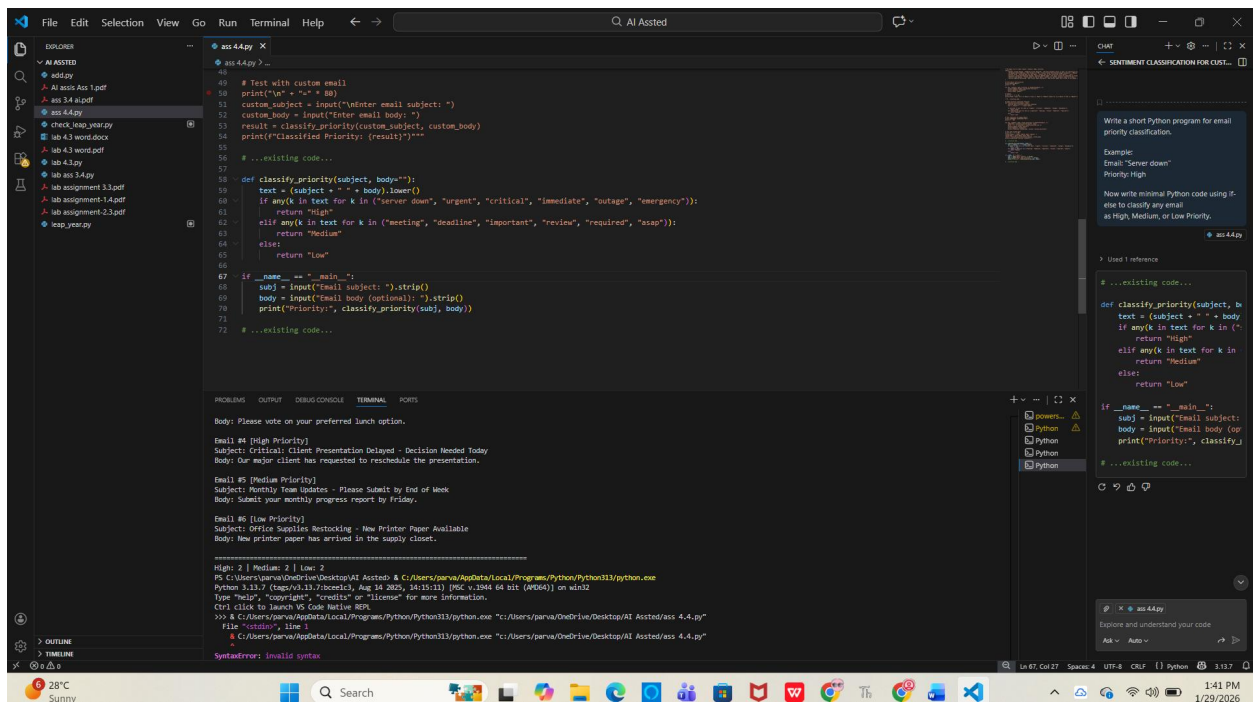
Scenario:

A company wants to automatically prioritize incoming emails into High Priority, Medium Priority, or Low Priority.

Tasks:

1. Create 6 sample email messages with priority labels.
2. Perform intent classification using Zero-shot prompting.
3. Perform classification using One-shot prompting.
4. Perform classification using Few-shot prompting.
5. Evaluate which technique produces the most reliable results and why.





3. Student Query Routing System

Scenario:

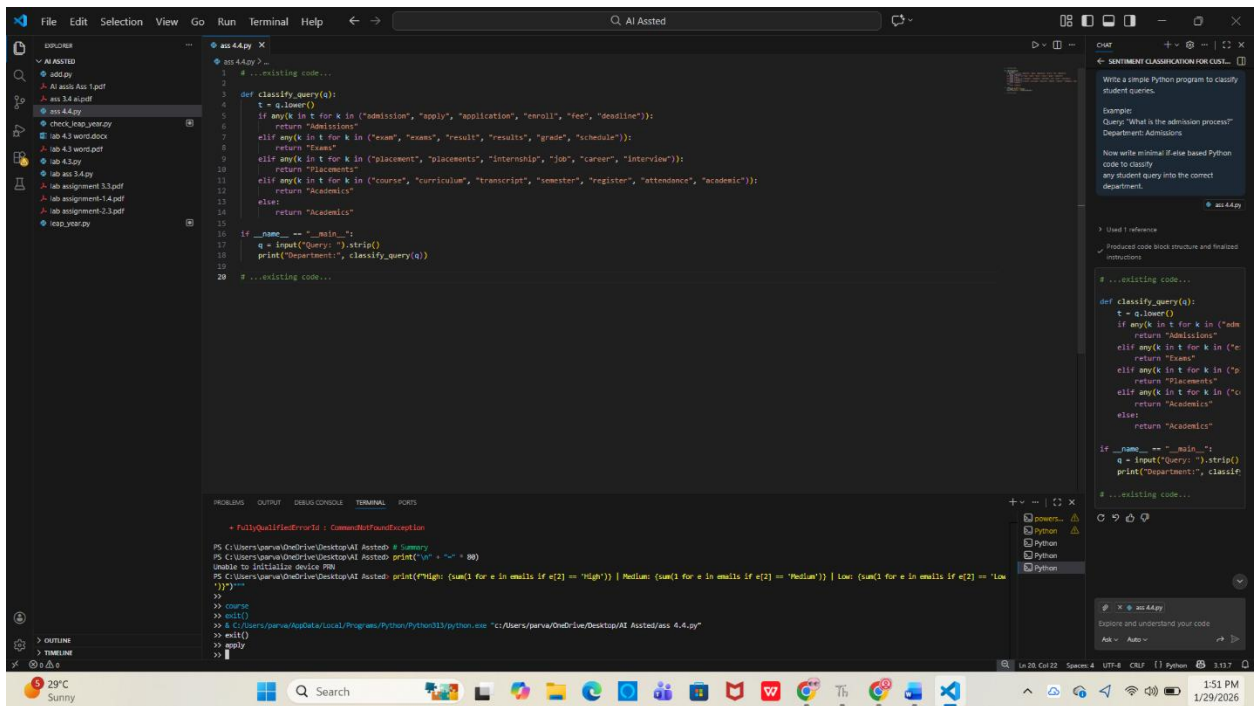
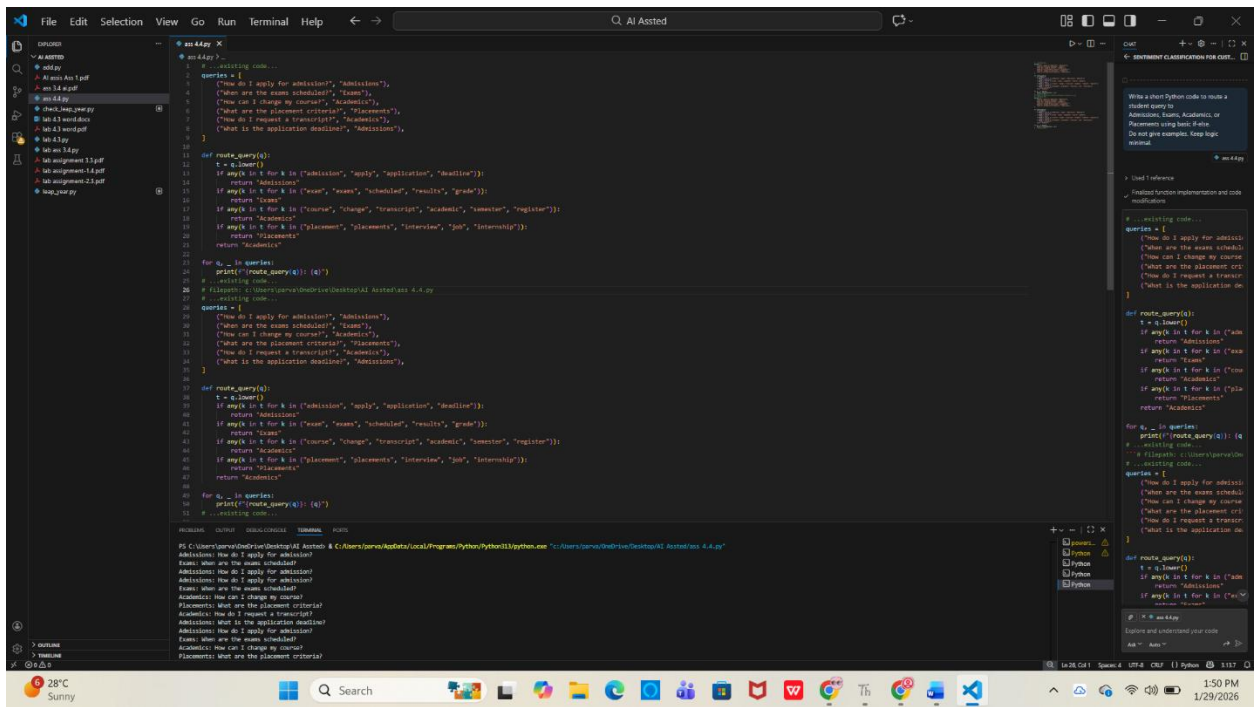
A university chatbot must route student queries to Admissions, Exams, Academics, or Placements.

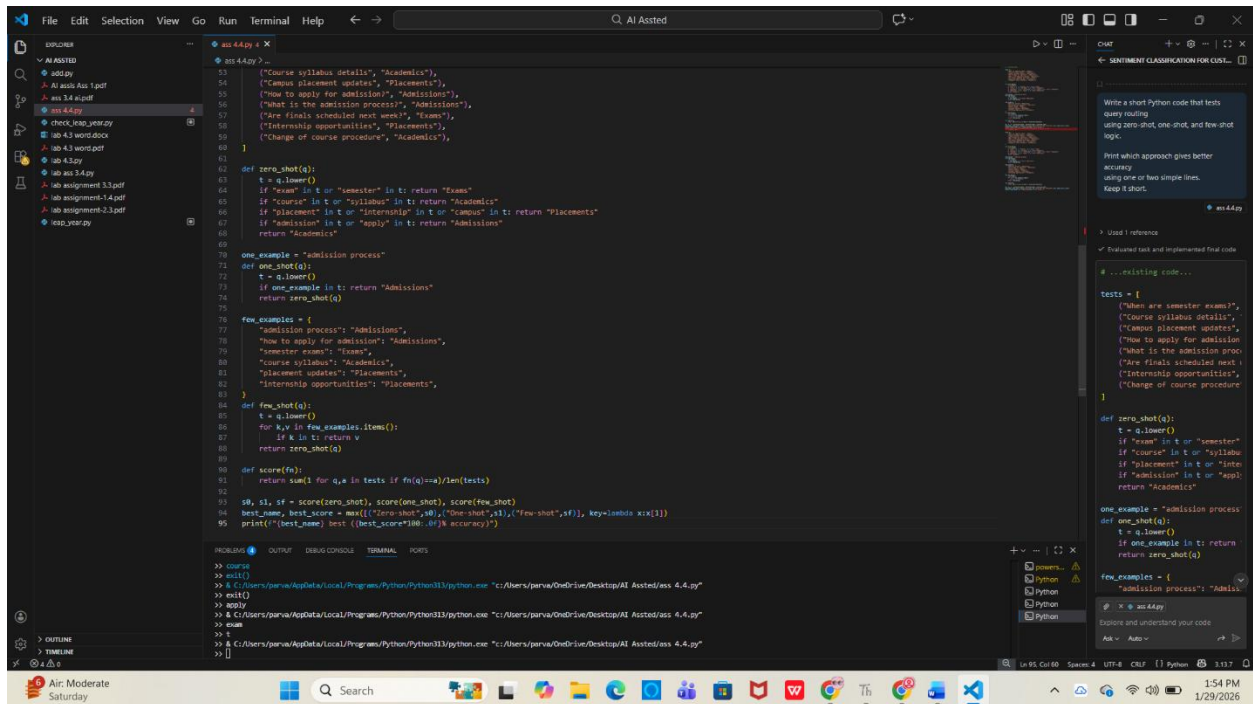
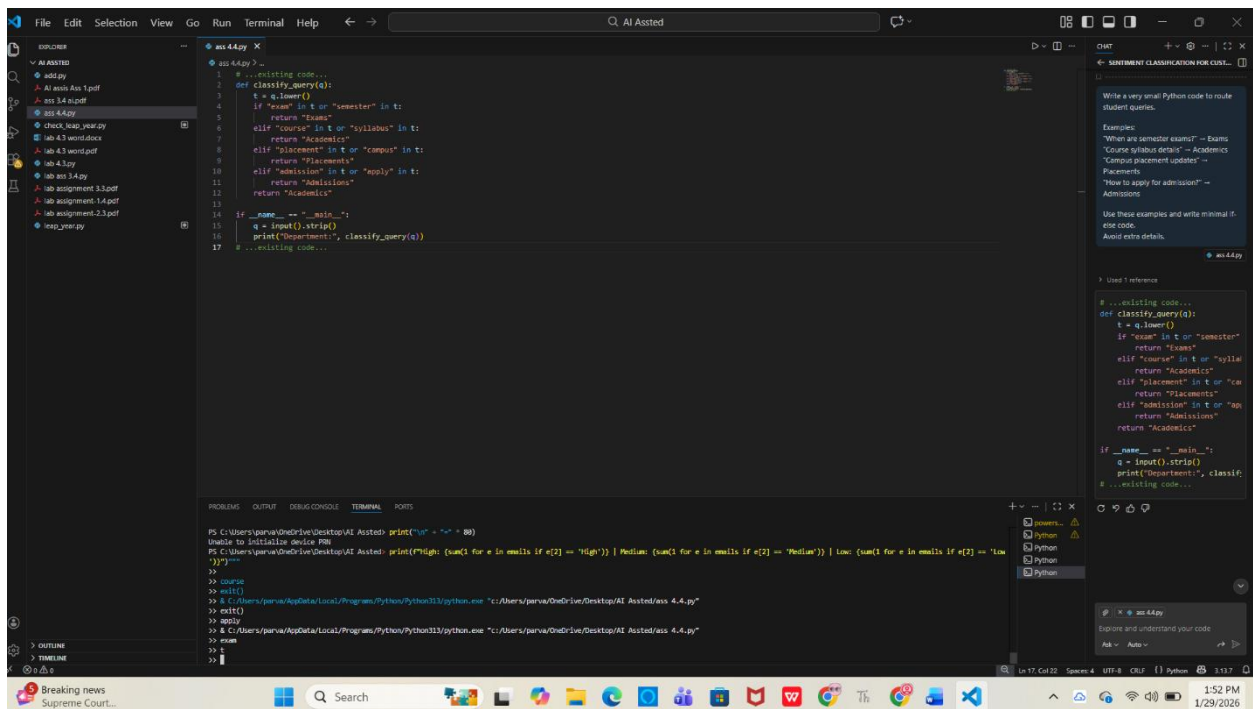
Tasks:

1. Create 6 sample student queries mapped to departments.
2. Implement Zero-shot intent classification using an LLM.
3. Improve results using One-shot prompting.
4. Further refine results using Few-shot prompting.
5. Analyze how contextual examples affect classification accuracy.

The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree on the left showing a project named 'AI Assted' with files like 'add.py', 'AI ass 1.pdf', 'ass 3.4.pdf', 'ass 4.4.py', 'check_knap_year.py', 'lab 4.3 word.docx', 'lab 4.3 word.pdf', 'lab 4.3.py', 'lab ass 3.4.py', 'lab assignment 3.4.pdf', 'lab assignment 3.4.pdf', 'lab assignment 2.3.pdf', and 'knap_year.py'.
- EDITOR:** The main workspace shows the content of 'ass 4.4.py'. The code defines a list of student queries mapped to departments and a function to process them.
- TERMINAL:** The bottom panel shows the output of running the script. It displays several 'SyntaxError: invalid syntax' messages for the first three queries, followed by a successful execution for the last query, which prints the department 'Academics'.
- CHAT:** A chat window on the right is titled 'SENTIMENT CLASSIFICATION FOR CUST...'. It contains a prompt asking for a Python program to classify student queries and a response that provides the code shown in the editor.





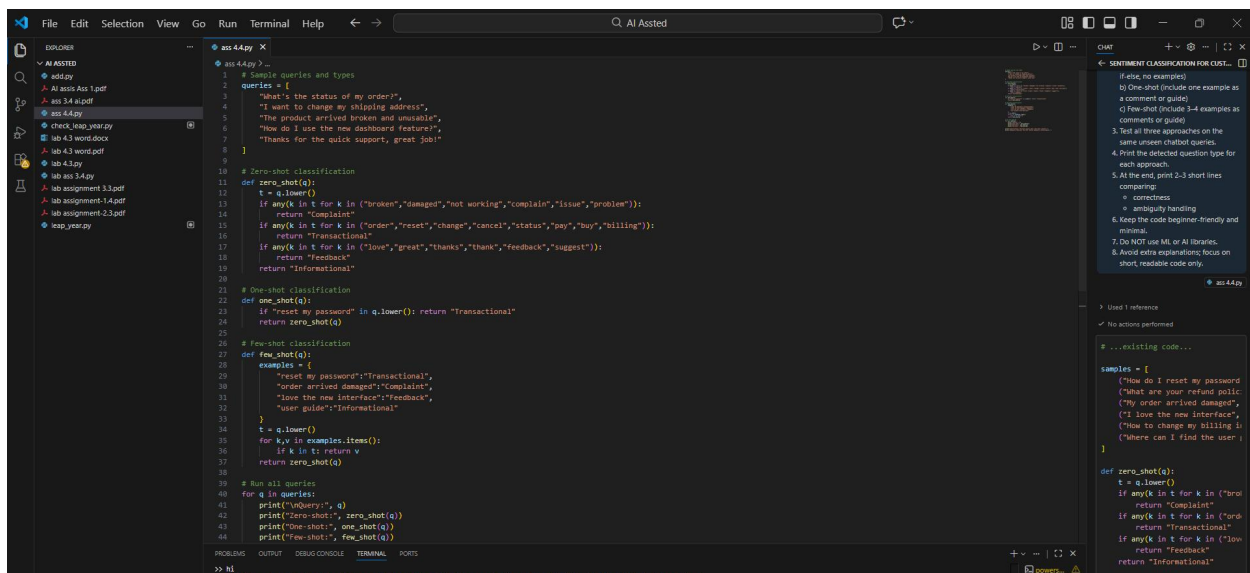
4. Chatbot Question Type Detection

Scenario:

A chatbot must identify whether a user query is Informational, Transactional, Complaint, or Feedback.

Tasks:

1. Prepare 6 chatbot queries mapped to question types.
2. Design prompts for Zero-shot, One-shot, and Few-shot learning.
3. Test all prompts on the same unseen queries.
4. Compare response correctness and ambiguity handling.
5. Document observations.



```
1 # Small queries and types
2 queries = [
3     "What's the status of my order?",
4     "I want to change my shipping address",
5     "The product arrived broken and unusable",
6     "How do I use the new dashboard feature?",
7     "Thanks for the quick support, great job!"
8 ]
9
10 # Zero-shot classification
11 def zero_shot(q):
12     t = q.lower()
13     if any(k in t for k in ("broken", "damaged", "not working", "complain", "issue", "problem")):
14         return "Complaint"
15     if any(k in t for k in ("order", "reset", "change", "cancel", "status", "pay", "buy", "billing")):
16         return "Transactional"
17     if any(k in t for k in ("love", "great", "thanks", "thank", "feedback", "suggest")):
18         return "Feedback"
19     return "Informational"
20
21 # One-shot classification
22 def one_shot(q):
23     if "reset my password" in q.lower(): return "Transactional"
24     return zero_shot(q)
25
26 # Few-shot classification
27 def few_shot(q):
28     examples = [
29         "reset my password": "Transactional",
30         "order arrived damaged": "Complaint",
31         "love the new interface": "Feedback",
32         "user guide": "Informational"
33     ]
34     t = q.lower()
35     for k, v in examples.items():
36         if k in t: return v
37     return zero_shot(q)
38
39 # Run all queries
40 for q in queries:
41     print(f"Query: {q}")
42     print(f"Zero-shot: {zero_shot(q)}")
43     print(f"One-shot: {one_shot(q)}")
44     print(f"Few-shot: {few_shot(q)}")
45
46 # ...existing code...
```

SENTIMENT CLASSIFICATION FOR CUSTOMER FEEDBACK

(If not, no examples)

b) One-shot (include one example as a comment or guide)

c) Few-shot (include 3-4 examples as comments or guide)

3. Test all three approaches on the same unseen chatbot queries.

4. Print the detected question type for each approach.

5. At the end, print 2-3 short lines comparing:

- + correctness
- + ambiguity handling

6. Keep the code beginner-friendly and minimal.

7. Do NOT use ML or AI libraries.

8. Avoid extra explanations; focus on short, readable code only.

Used 1 reference

✓ No actions performed

```
samples = [
    ("How do I reset my password", "Transactional"),
    ("What are your refund policies?", "My order arrived damaged", "I love the new interface", "How to change my billing info", "Where can I find the user guide"),
]

def zero_shot(q):
    t = q.lower()
    if any(k in t for k in ("broken", "damaged", "not working", "complain", "issue", "problem")):
        return "Complaint"
    if any(k in t for k in ("order", "reset", "change", "cancel", "status", "pay", "buy", "billing")):
        return "Transactional"
    if any(k in t for k in ("love", "great", "thanks", "thank", "feedback", "suggest")):
        return "Feedback"
    return "Informational"
```

5. Emotion Detection in Text

Scenario:

A mental-health chatbot needs to detect emotions: Happy, Sad, Angry, Anxious, Neutral.

Tasks:

1. Create labeled emotion samples.
2. Use Zero-shot prompting to identify emotions.
3. Use One-shot prompting with an example.
4. Use Few-shot prompting with multiple emotions.
5. Discuss ambiguity handling across techniques.

The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left lists files under 'AI CODING', including 'add.py', 'AI lab43.py', and 'lab assignment 44.py'. The main editor displays the code for 'lab assignment 44.py'. The code is as follows:

```
1 import pandas as pd
2
3 # Create a DataFrame from the provided data
4 data = {
5     "Text": [
6         "I am very happy today",
7         "I feel lonely and depressed",
8         "This is so frustrating",
9         "I am worried about my future",
10        "Today is just normal",
11        "Feeling excited about results"
12    ],
13    "Emotion": [
14        "Happy",
15        "Sad",
16        "Angry",
17        "Anxious",
18        "Neutral",
19        "Happy"
20    ]
21 }
22
23 df = pd.DataFrame(data)
24
25 # Display the DataFrame
26 print(df)
```

Below the code editor, the TERMINAL tab is active, showing the command prompt output:

```
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Traceback (most recent call last):
  File "d:\AI Coding\lab assignment 44.py", line 1, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Traceback (most recent call last):
  File "d:\AI Coding\lab assignment 44.py", line 1, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding>
```

This screenshot is a duplicate of the one above, showing the same Visual Studio Code interface with the 'lab assignment 44.py' file open. The code in the editor and the terminal output showing the 'ModuleNotFoundError: No module named 'pandas'' error are identical to the first image.

The screenshot shows the VS Code interface with a file explorer on the left containing various assignment files. The main editor displays a Python script named `lab assignment 44.py`. The script defines a function `identify_emotion(text)` that checks for the word "frustrating" and returns "frustrated" or "neutral". It includes an example usage where the text "This is so frustrating" is passed to the function, resulting in the output "Emotion: Anxious".

```
1 def identify_emotion(text):
2     if "frustrating" in text:
3         return "frustrated"
4     return "neutral"
5
6 # Example usage
7 text = "This is so frustrating"
8 emotion = identify_emotion(text)
9 print(f"Emotion: {emotion}")
```

The terminal at the bottom shows a `ModuleNotFoundError: No module named 'pandas'` error, which occurred when the script was executed. The error message indicates that the file `"d:/AI Coding/lab assignment 44.py"` was executed, and the error occurred at line 1 in the `<module>`.

The screenshot shows the VS Code interface with a file explorer on the left containing various assignment files. The main editor displays a Python script named `lab assignment 44.py`. The script defines a function `classify_emotion(text)` that uses a dictionary to map keywords to emotions. It includes an example usage where the text "This is so frustrating" is passed to the function, resulting in the output "Text: 'This is so frustrating' Emotion: frustrated".

```
1 def classify_emotion(text):
2     emotions = {
3         "happy": ["happy", "joyful", "excited", "pleased"],
4         "sad": ["lonely", "depressed", "sad", "down"],
5         "anxious": ["worried", "anxious", "nervous", "stressed"],
6         "neutral": ["normal", "fine", "okay", "average"],
7         "frustrated": ["frustrating", "annoyed", "irritated"]
8     }
9
10    for emotion, keywords in emotions.items():
11        if any(keyword in text.lower() for keyword in keywords):
12            return emotion
13    return "Unknown"
14
15 # Example usage
16 text = "This is so frustrating"
17 emotion = classify_emotion(text)
18 print(f"Text: '{text}'\nEmotion: {emotion}")
```

The terminal at the bottom shows a `ModuleNotFoundError: No module named 'pandas'` error, which occurred when the script was executed. The error message indicates that the file `"d:/AI Coding/lab assignment 44.py"` was executed, and the error occurred at line 1 in the `<module>`.

File Edit Selection View Go Run Terminal Help

AI Coding

EXPLORER

AI CODING

add.py

AI lab43.py

Assignment1(CP).pdf

Assignment 2-4.pdf

assignment 3.4

assignment 3.py

Assignment2.pdf

assignment3.4.docx

factorial.py

jobs.py

jobs.py

lab assignment 44.py

lab assignment3.3.pdf

lab1 HCP.pdf

matrixHCP.py

Untitled20.ipynb

week2 HCP.pdf

lab assignment 44.py

Generate code

Add Context...

```
1 def classify_emotion(text):
2     emotions = {
3         "happy": ["happy", "joyful", "excited", "pleased"],
4         "sad": ["lonely", "depressed", "sad", "down"],
5         "anxious": ["worried", "anxious", "nervous", "stressed"],
6         "neutral": ["normal", "fine", "okay", "average"],
7         "frustrated": ["frustrating", "annoyed", "irritated"]
8     }
9
10    for emotion, keywords in emotions.items():
11        if any(keyword in text.lower() for keyword in keywords):
12            return emotion
13    return "Unknown"
14
15    # Example usage
16    text = "This is so frustrating"
17    emotion = classify_emotion(text)
18    print(f"Text: {text}\nEmotion: {emotion}")
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Traceback (most recent call last):
 File "d:\AI Coding\lab assignment 44.py", line 1, in <module>
 import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Anxious
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Emotion: Frustrated
PS D:\AI Coding> & C:/Users/ANJALI/AppData/Local/Programs/Python/Python313/python.exe "d:/AI Coding/lab assignment 44.py"
Text: "This is so frustrating"
Emotion: frustrated
PS D:\AI Coding>

ASSIGNMENT-9.4

BATCH-25

Task 1: Auto-Generating Function Documentation in a Shared

Codebase

Scenario

You have joined a development team where several utility functions are already implemented, but the code lacks proper documentation. New team members are struggling to understand how these functions should be used.

Task Description

You are given a Python script containing multiple functions without any docstrings.

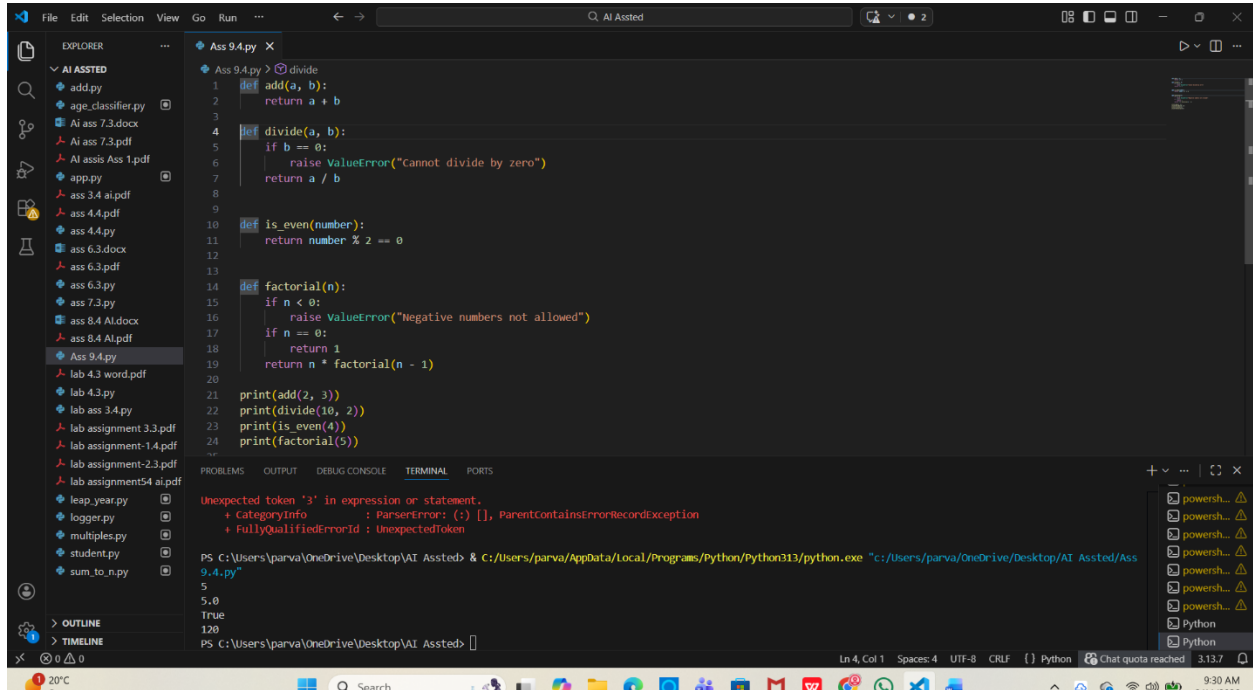
Using an AI-assisted coding tool:

- Ask the AI to automatically generate Google-style function docstrings for each function
- Each docstring should include:
 - o A brief description of the function
 - o Parameters with data types
 - o Return values
 - o At least one example usage (if applicable)

Experiment with different prompting styles (zero-shot or context-based) to observe quality differences.

Expected Outcome

- A Python script with well-structured Google-style docstrings
- Docstrings that clearly explain function behavior and usage
- Improved readability and usability of the codebase



Task 2: Enhancing Readability Through AI-Generated Inline

Comments

Scenario

A Python program contains complex logic that works correctly but is difficult to understand at first glance. Future maintainers may find it hard to debug or extend this code.

Task Description

You are provided with a Python script containing:

- Loops
- Conditional logic
- Algorithms (such as Fibonacci sequence, sorting, or searching)

Use AI assistance to:

- Automatically insert inline comments only for complex or non-obvious logic

- Avoid commenting on trivial or self-explanatory syntax

The goal is to improve clarity without cluttering the code.

Expected Outcome

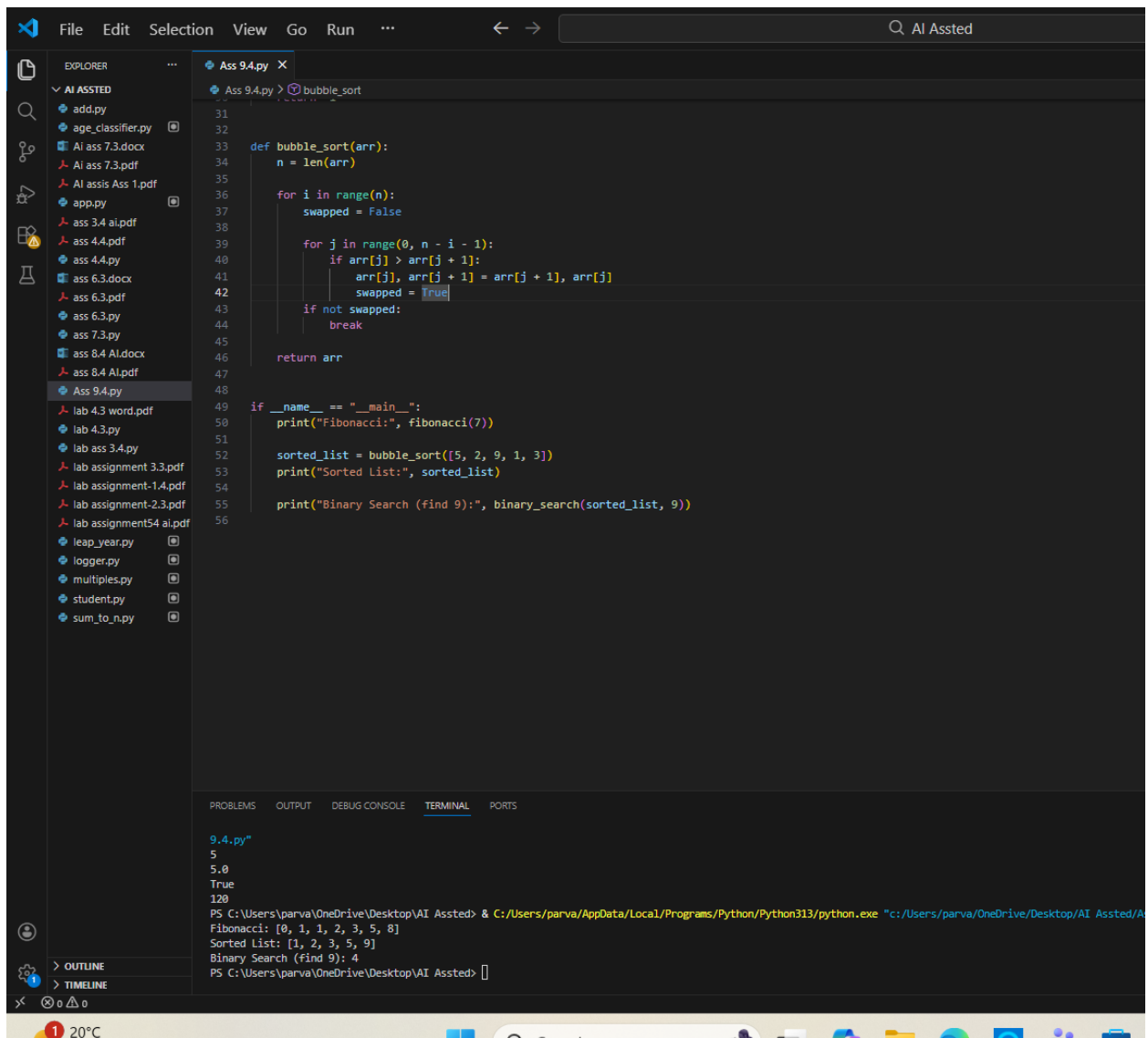
- A Python script with concise, meaningful inline comments
- Comments that explain why the logic exists, not what Python syntax does
- Noticeable improvement in code readability

The screenshot shows a VS Code editor with a Python script named 'Ass 9.4.py'. The script contains three functions: 'fibonacci(n)', 'binary_search(arr, target)', and 'bubble_sort(arr)'. The 'fibonacci' function has inline comments explaining the base cases and the recursive step. The 'binary_search' function has inline comments explaining the search range and the mid-point calculation. The 'bubble_sort' function has inline comments explaining the sorting process and the swapped flag. The terminal output shows the execution of the script, displaying the Fibonacci sequence, the sorted list, and the result of the binary search.

```
1 def fibonacci(n):
2     if n <= 0:
3         return []
4     if n == 1:
5         return [0]
6
7     sequence = [0, 1]
8     for i in range(2, n):
9         next_value = sequence[i - 1] + sequence[i - 2]
10        sequence.append(next_value)
11
12    return sequence
13
14
15 def binary_search(arr, target):
16     left = 0
17     right = len(arr) - 1
18
19     while left <= right:
20         mid = (left + right) // 2
21         if arr[mid] == target:
22             return mid
23         elif arr[mid] < target:
24             left = mid + 1
25         else:
26             right = mid - 1
27     return -1
28
29
30 def bubble_sort(arr):
31     n = len(arr)
32
33     for i in range(n):
34         swapped = False
35         for j in range(0, n - i - 1):
36             if arr[j] > arr[j + 1]:
37                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
38                 swapped = True
39     if not swapped:
40         return arr
```

Terminal Output:

```
9.4.py
5
5,0
True
120
PS C:\Users\parva\OneDrive\Desktop\AI Assted> C:\Users\parva\AppData\Local\Programs\Python\Python313\python.exe "c:\Users\parva\OneDrive\Desktop\AI Assted\Ass 9.4.py"
Fibonacci: [0, 1, 1, 2, 3, 5, 8]
Sorted list: [1, 2, 3, 5, 9]
Binary Search (find 9): 4
PS C:\Users\parva\OneDrive\Desktop\AI Assted>
```



Task 3: Generating Module-Level Documentation for a Python

Package

Scenario

Your team is preparing a Python module to be shared internally (or uploaded to a repository). Anyone opening the file should immediately understand its purpose and structure.

Task Description

Provide a complete Python module to an AI tool and instruct it to

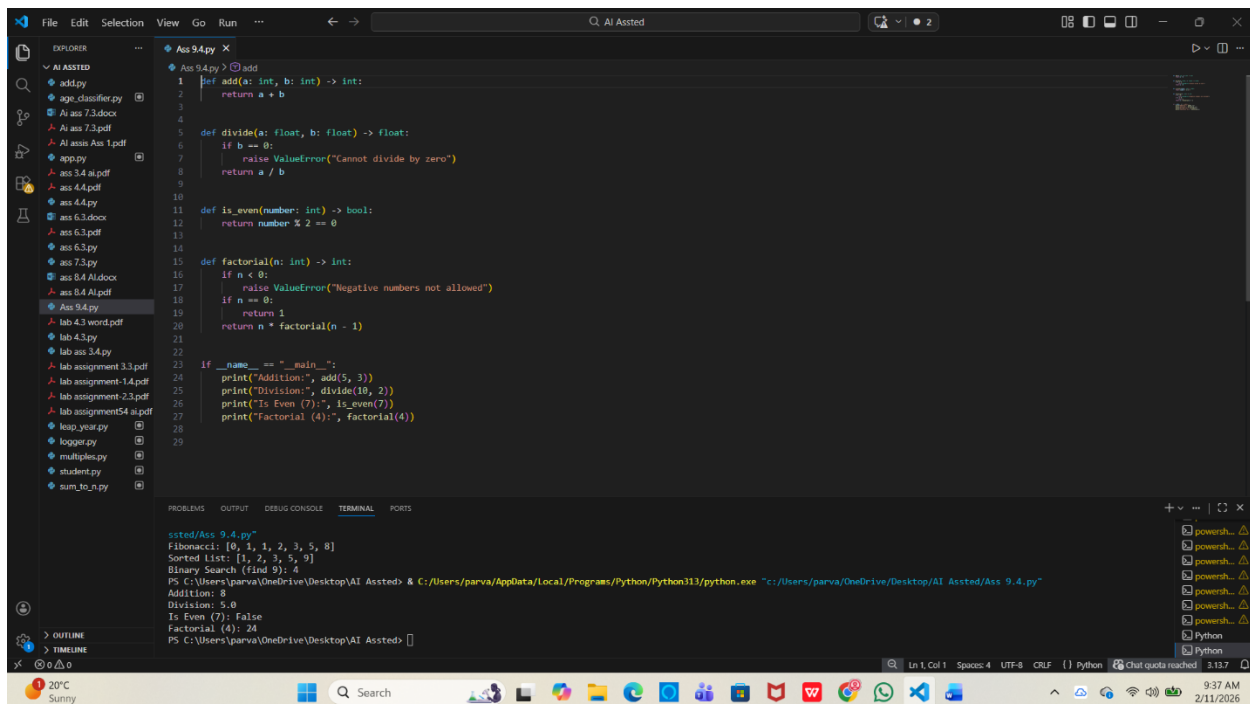
automatically generate a module-level docstring at the top of the file that includes:

- The purpose of the module
- Required libraries or dependencies
- A brief description of key functions and classes
- A short example of how the module can be used

Focus on clarity and professional tone.

Expected Outcome

- A well-written multi-line module-level docstring
- Clear overview of what the module does and how to use it
- Documentation suitable for real-world projects or repositories



```
File Edit Selection View Go Run ... AI Assted
EXPLORER
  AI ASSTED
  add.py
  age_classifier.py
  Ai ass 7.3.docx
  Ai ass 7.3.pdf
  Ai ass 1.pdf
  app.py
  ass 3.4 AI.pdf
  ass 4.4.pdf
  ass 4.4.py
  ass 6.3.docx
  ass 6.3.pdf
  ass 6.3.py
  ass 7.3.py
  ass 8.4 AI.docx
  ass 8.4 AI.pdf
  Ass 9.4.py
  lab 4.3 word.pdf
  lab 4.3.py
  lab ass 3.4.py
  lab assignment 2.3.pdf
  lab assignment 4.4.pdf
  lab assignment 2.3.pdf
  lab assignment 5.4 AI.pdf
  leap_year.py
  logger.py
  multiples.py
  student.py
  sum_to_n.py

Ass 9.4.py X
1 def add(a: int, b: int) -> int:
2     return a + b
3
4
5 def divide(a: float, b: float) -> float:
6     if b == 0:
7         raise ValueError("Cannot divide by zero")
8     return a / b
9
10
11 def is_even(number: int) -> bool:
12     return number % 2 == 0
13
14
15 def factorial(n: int) -> int:
16     if n < 0:
17         raise ValueError("Negative numbers not allowed")
18     if n == 0:
19         return 1
20     return n * factorial(n - 1)
21
22
23 if __name__ == "__main__":
24     print("Addition:", add(5, 3))
25     print("Division:", divide(10, 2))
26     print("Is Even (7):", is_even(7))
27     print("Factorial (4):", factorial(4))
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ssted/Ass 9.4.py
Fibonacci: [0, 1, 1, 2, 3, 5, 8]
Sorted List: [1, 2, 3, 5, 9]
Binary Search (find 9): 4
PS C:\Users\parva\OneDrive\Desktop\AI Assted> & C:\Users\parva\AppData\Local\Programs\Python\Python311\python.exe "c:\Users\parva\OneDrive\Desktop\AI Assted\Ass 9.4.py"
Addition: 8
Division: 5.0
Is Even (7): False
Factorial (4): 24
PS C:\Users\parva\OneDrive\Desktop\AI Assted>

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Chat quota reached: 3,137
20°C Sunny 9:37 AM 2/11/2026
```

Task 4: Converting Developer Comments into Structured Docstrings

Scenario

In a legacy project, developers have written long explanatory comments inside functions instead of proper docstrings. The team now wants to

standardize documentation.

Task Description

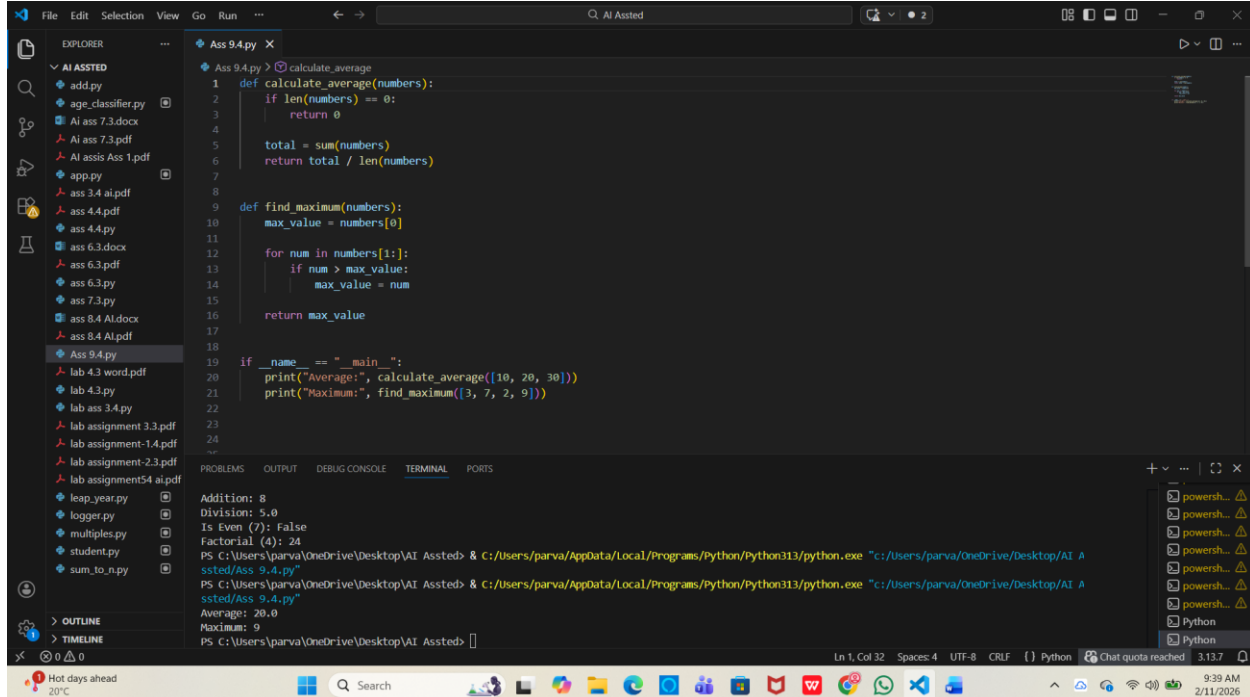
You are given a Python script where functions contain detailed inline comments explaining their logic.

Use AI to:

- Automatically convert these comments into structured Google-style or NumPy-style docstrings
- Preserve the original meaning and intent of the comments
- Remove redundant inline comments after conversion

Expected Outcome

- Functions with clean, standardized docstrings
- Reduced clutter inside function bodies
- Improved consistency across the codebase



The screenshot shows a VS Code editor with a Python script named 'Ass 9.4.py'. The script contains two functions: 'calculate_average' and 'find_maximum'. The 'calculate_average' function has a detailed inline comment explaining its logic. The 'find_maximum' function also has a detailed inline comment. The script is executed, and the output is displayed in the terminal window. The output shows the average of 10 and 20 is 15, and the maximum of 3, 7, 2, and 9 is 9.

```
1 def calculate_average(numbers):
2     """Calculate the average of a list of numbers.
3     If the list is empty, return 0.
4     """
5     total = sum(numbers)
6     return total / len(numbers)
7
8
9 def find_maximum(numbers):
10    """Find the maximum value in a list of numbers.
11    If the list is empty, return the first element.
12    """
13    max_value = numbers[0]
14    for num in numbers[1:]:
15        if num > max_value:
16            max_value = num
17    return max_value
18
19 if __name__ == "__main__":
20     print("Average:", calculate_average([10, 20, 30]))
21     print("Maximum:", find_maximum([3, 7, 2, 9]))
22
23
24
```

Terminal Output:

```
PS C:\Users\parva\OneDrive\Desktop\AI Assted> & C:/Users/parva/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/parva/OneDrive/Desktop/AI Assted/Ass 9.4.py"
Average: 15.0
Maximum: 9
```

Task 5: Building a Mini Automatic Documentation Generator

Scenario

Your team wants a simple internal tool that helps developers start documenting new Python files quickly, without writing documentation from scratch.

Task Description

Design a small Python utility that:

- Reads a given .py file
- Automatically detects:
 - o Functions
 - o Classes
- Inserts placeholder Google-style docstrings for each detected function or class

AI tools may be used to assist in generating or refining this utility.

Note: The goal is documentation scaffolding, not perfect documentation.

Expected Outcome

- A working Python script that processes another .py file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation

Automation

```
1 import ast
2 import os
3
4 def generate_function_docstring(name, args):
5     params = ""
6     for arg in args:
7         params += f"    {arg} (type): Description of {arg}.n"
8
9     return f'''
10     """
11     Brief description of {name}.
12
13     Args:
14         {params if params else "    None"}
15
16     Returns:
17         """
18     type: Description of return value.
19     """
20 '''
21
22 def generate_class_docstring(name):
23     return f'''
24     """
25     Brief description of {name} class.
26
27     Attributes:
28         Describe attributes here.
29
30     Methods:
31         Describe methods here.
32     """
33 '''
34
35 def insert_docstrings():
36     filename = "Ass 9.4.py"
37
38     if not os.path.exists(filename):
39         print("File not found!")
40         return
41
42 # FIXED ENCODING
43 with open(filename, "r", encoding="utf-8") as file:
```

PS C:\Users\parva\OneDrive\Desktop\AI Assted> & C:/Users/parva/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/parva/OneDrive/Desktop/AI Assted/Ass 9.4.py"

Documentation generated successfully!
New file created: Ass 9.4_documented.py

PS C:\Users\parva\OneDrive\Desktop\AI Assted> []

```
31 def insert_docstrings():
32     with open(filename, "r", encoding="utf-8") as file:
33         source = file.read()
34
35     tree = ast.parse(source)
36     lines = source.split("\n")
37
38     offset = 0
39
40     for node in ast.walk(tree):
41         if isinstance(node, ast.FunctionDef):
42             if ast.get_docstring(node) is None and len(node.body) > 0:
43                 doc = generate_function_docstring(
44                     node.name, [arg.arg for arg in node.args.args]
45                 )
46                 insert_at = node.body[0].lineno - 1 + offset
47                 lines.insert(insert_at, doc)
48                 offset += doc.count("\n")
49
50         elif isinstance(node, ast.ClassDef):
51             if ast.get_docstring(node) is None and len(node.body) > 0:
52                 doc = generate_class_docstring(node.name)
53                 insert_at = node.body[0].lineno - 1 + offset
54                 lines.insert(insert_at, doc)
55                 offset += doc.count("\n")
56
57     new_filename = "Ass 9.4_documented.py"
58
59     # FIXED ENCODING
60     with open(new_filename, "w", encoding="utf-8") as file:
61         file.write("\n".join(lines))
62
63     print("Documentation generated successfully!")
64     print("New file created:", new_filename)
65
66 if __name__ == "__main__":
67     insert_docstrings()
68
69 #
```

PS C:\Users\parva\OneDrive\Desktop\AI Assted> & C:/Users/parva/AppData/Local/Programs/Python/Python313/python.exe "C:/Users/parva/OneDrive/Desktop/AI Assted/Ass 9.4.py"

Documentation generated successfully!
New file created: Ass 9.4_documented.py

PS C:\Users\parva\OneDrive\Desktop\AI Assted> []

Lab assignment-8.3

Batch 25

Task 1: Email Validation using TDD

Scenario

You are developing a user registration system that requires reliable email input validation.

Requirements

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement `is_valid_email(email)` to pass all AI-generated test cases

Expected Output

- Python function for email validation
- All AI-generated test cases pass successfully
- Invalid email formats are correctly rejected
- Valid email formats return True

Prompt:

Using unittest implement `is_valid_email(email)` following four TDD styles: Red Green Refactor cycle, behaviour driven tests for valid and invalid emails, boundary and edge case tests like starting or ending with special characters and multiple @, and error driven tests for wrong types, ensure all tests pass.

Code:

```
import unittest
from typing import Union

def is_valid_email(email: Union[str, int, None]) -> bool:

    if not isinstance(email, str):
        raise TypeError(f"Email must be a string, not {type(email).__name__}")

    if not email or email.startswith(('@', '.', '-')) or email.endswith(('@', '.')):
```



```

        return False

    if email.count('@') != 1:
        return False

    local, domain = email.split('@')

    if not local or not domain or '.' not in domain:
        return False

    if domain.startswith('.') or domain.endswith('.'):
        return False

    if domain.count('.') > 0 or local.count('.') > 0:
        return False

    return True

```

```

class TestIsValidEmailRGR(unittest.TestCase):
    """Red Green Refactor cycle tests"""

    def test_valid_simple_email(self):
        self.assertTrue(is_valid_email("user@example.com"))

    def test_valid_email_with_numbers(self):
        self.assertTrue(is_valid_email("user123@example.com"))

    def test_invalid_no_at_sign(self):
        self.assertFalse(is_valid_email("userexample.com"))

    def test_invalid_no_domain_extension(self):
        self.assertFalse(is_valid_email("user@example"))

```

```

class TestIsValidEmailBDD(unittest.TestCase):
    """Behaviour Driven Development tests"""

    def test_valid_emails(self):
        valid_emails = [
            "john@example.com",
            "alice.smith@company.org",
            "bob_99@domain.co.uk",
            "test+tag@email.net"
        ]
        for email in valid_emails:
            with self.subTest(email=email):
                self.assertTrue(is_valid_email(email))

    def test_invalid_emails(self):
        invalid_emails = [
            "plainaddress",
            "user@",
            "@example.com",
            "user@@example.com",
            "user@.com",
            "user.example.com"
        ]
        for email in invalid_emails:
            with self.subTest(email=email):
                self.assertFalse(is_valid_email(email))

```

```

class TestIsValidEmailBoundary(unittest.TestCase):
    """Boundary and edge case tests"""

    def test_starts_with_special_characters(self):
        self.assertFalse(is_valid_email("@user@example.com"))
        self.assertFalse(is_valid_email(".user@example.com"))
        self.assertFalse(is_valid_email("-user@example.com"))

    def test_ends_with_special_characters(self):
        self.assertFalse(is_valid_email("user@example.com."))
        self.assertFalse(is_valid_email("user@example.com@"))

    def test_multiple_at_signs(self):
        self.assertFalse(is_valid_email("user@@example.com"))
        self.assertFalse(is_valid_email("user@exam@ple.com"))

    def test_consecutive_dots(self):
        self.assertFalse(is_valid_email("user..name@example.com"))
        self.assertFalse(is_valid_email("user@exam..ple.com"))

    def test_empty_string(self):
        self.assertFalse(is_valid_email(""))

    def test_single_character_local_and_domain(self):
        self.assertTrue(is_valid_email("a@b.co"))

```

```

class TestIsValidEmailErrorDriven(unittest.TestCase):
    """Error driven tests for wrong types"""

    def test_none_type(self):
        with self.assertRaises(TypeError):
            is_valid_email(None)

    def test_integer_type(self):
        with self.assertRaises(TypeError):
            is_valid_email(12345)

    def test_list_type(self):
        with self.assertRaises(TypeError):
            is_valid_email(["user@example.com"])

    def test_dict_type(self):
        with self.assertRaises(TypeError):
            is_valid_email({"email": "user@example.com"})

    def test_float_type(self):
        with self.assertRaises(TypeError):
            is_valid_email(123.45)

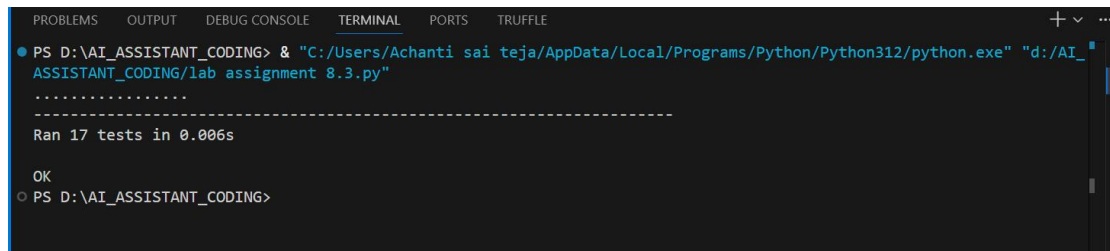
```

```

if __name__ == '__main__':
    unittest.main()

```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 8.3.py"
.....
Ran 17 tests in 0.006s
OK
PS D:\AI_ASSISTANT_CODING>
```

Task 2: Grade Assignment using Loops

Scenario

You are building an automated grading system for an online examination platform.

Requirements

- AI should generate test cases for `assign_grade(score)` where:
 - 90–100 → A
 - 80–89 → B
 - 70–79 → C
 - 60–69 → D
 - Below 60 → F

- Include boundary values (60, 70, 80, 90)
- Include invalid inputs such as -5, 105, "eighty"
- Implement the function using a test-driven approach

Expected Output

- Grade assignment function implemented in Python
- Boundary values handled correctly
- Invalid inputs handled gracefully
- All AI-generated test cases pass

Prompt:

Using unittest implement `assign_grade(score)` with four TDD approaches: Red Green Refactor, behaviour driven grade mapping A(90-100) B(80-89) C(70-79) D(60-69) F(<60), boundary tests for 60 70 80 90, and error driven tests for invalid inputs -5 105 and string values, ensure all tests pass.

Code:

```
import unittest

def assign_grade(score):
```

```

"""
Assign a letter grade based on a numerical score.

Args:
    score: numerical score (0-100)

Returns:
    Letter grade (A, B, C, D, F)

Raises:
    TypeError: if score is not a number
    ValueError: if score is outside 0-100 range
"""
if not isinstance(score, (int, float)) or isinstance(score, bool):
    raise TypeError("Score must be a number")

if score < 0 or score > 100:
    raise ValueError("Score must be between 0 and 100")

if score >= 90:
    return 'A'
elif score >= 80:
    return 'B'
elif score >= 70:
    return 'C'
elif score >= 60:
    return 'D'
else:
    return 'F'

```

```

class TestAssignGrade(unittest.TestCase):
    """Test cases for assign_grade function using TDD approaches"""

    # Red Green Refactor: Basic grade mapping
    def test_grade_a(self):
        self.assertEqual(assign_grade(95), 'A')

    def test_grade_b(self):
        self.assertEqual(assign_grade(85), 'B')

    def test_grade_c(self):
        self.assertEqual(assign_grade(75), 'C')

    def test_grade_d(self):
        self.assertEqual(assign_grade(65), 'D')

    def test_grade_f(self):
        self.assertEqual(assign_grade(55), 'F')

    # Behaviour Driven: Complete grade range mapping
    def test_grade_a_range(self):
        self.assertEqual(assign_grade(90), 'A')
        self.assertEqual(assign_grade(100), 'A')

    def test_grade_b_range(self):
        self.assertEqual(assign_grade(80), 'B')
        self.assertEqual(assign_grade(89), 'B')

    def test_grade_c_range(self):
        self.assertEqual(assign_grade(70), 'C')

```

```

        self.assertEqual(assign_grade(79), 'C')

    def test_grade_d_range(self):
        self.assertEqual(assign_grade(60), 'D')
        self.assertEqual(assign_grade(69), 'D')

    def test_grade_f_range(self):
        self.assertEqual(assign_grade(0), 'F')
        self.assertEqual(assign_grade(59), 'F')

    # Boundary Tests
    def test_boundary_60(self):
        self.assertEqual(assign_grade(60), 'D')
        self.assertEqual(assign_grade(59), 'F')

    def test_boundary_70(self):
        self.assertEqual(assign_grade(70), 'C')
        self.assertEqual(assign_grade(69), 'D')

    def test_boundary_80(self):
        self.assertEqual(assign_grade(80), 'B')
        self.assertEqual(assign_grade(79), 'C')

    def test_boundary_90(self):
        self.assertEqual(assign_grade(90), 'A')
        self.assertEqual(assign_grade(89), 'B')

    # Error Driven Tests: Invalid inputs
    def test_invalid_negative_score(self):
        with self.assertRaises(ValueError):
            assign_grade(-5)

    def test_invalid_score_over_100(self):
        with self.assertRaises(ValueError):
            assign_grade(105)

    def test_invalid_string_input(self):
        with self.assertRaises(TypeError):
            assign_grade("85")

    def test_invalid_none_input(self):
        with self.assertRaises(TypeError):
            assign_grade(None)

    def test_invalid_boolean_input(self):
        with self.assertRaises(TypeError):
            assign_grade(True)

```

```

if __name__ == '__main__':
    unittest.main()

```

Output:

```

PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d:/AI_ASSISTANT_CODING/lab_assignment_8.3.py"
.....
-----
Ran 19 tests in 0.004s

OK

```

Task 3: Sentence Palindrome Checker

Scenario

You are developing a text-processing utility to analyze sentences.

Requirements

- AI should generate test cases for `is_sentence_palindrome(sentence)`
- Ignore case, spaces, and punctuation
- Test both palindromic and non-palindromic sentences
- Example:
 - "A man a plan a canal Panama" → True

Expected Output

- Function correctly identifies sentence palindromes
- Case and punctuation are ignored
- Returns True or False accurately
- All AI-generated test cases pass

Prompt:

Using unittest implement `is_sentence_palindrome(sentence)` applying four TDD styles: Red Green Refactor, behaviour driven palindrome detection ignoring case spaces punctuation, boundary tests for empty and single word sentences, and error driven tests for non string input, ensure all tests pass.

Code:

```
import unittest
import re

def is_sentence_palindrome(sentence):
    """
    Check if a sentence is a palindrome, ignoring case, spaces, and punctuation.

    Args:
        sentence: A string to check

    Returns:
        Boolean indicating if the sentence is a palindrome

    Raises:
        TypeError: If input is not a string
    """
```

```

if not isinstance(sentence, str):
    raise TypeError("Input must be a string")

# Remove non-alphanumeric characters and convert to lowercase
cleaned = re.sub(r'^a-z0-9', '', sentence.lower())

# Check if cleaned string is equal to its reverse
return cleaned == cleaned[::-1]

```

```

class TestIsSentencePalindrome(unittest.TestCase):
    """Test suite for is_sentence_palindrome function using TDD styles"""

    # Red-Green-Refactor Tests
    def test_simple_palindrome(self):
        """Test basic palindrome detection"""
        self.assertTrue(is_sentence_palindrome("racecar"))
        self.assertTrue(is_sentence_palindrome("A man a plan a canal Panama"))

    def test_non_palindrome(self):
        """Test non-palindrome detection"""
        self.assertFalse(is_sentence_palindrome("hello"))
        self.assertFalse(is_sentence_palindrome("python"))

    # Behaviour-Driven Tests
    def test_ignores_case(self):
        """Should ignore uppercase and lowercase"""
        self.assertTrue(is_sentence_palindrome("RaceCar"))
        self.assertTrue(is_sentence_palindrome("MaDaM"))

    def test_ignores_spaces(self):
        """Should ignore spaces between words"""
        self.assertTrue(is_sentence_palindrome("race car"))
        self.assertTrue(is_sentence_palindrome("a b a"))

    def test_ignores_punctuation(self):
        """Should ignore punctuation marks"""
        self.assertTrue(is_sentence_palindrome("Was it a car or a cat I saw?"))
        self.assertTrue(is_sentence_palindrome("A Santa at NASA"))

    def test_combined_case_spaces_punctuation(self):
        """Should handle all three simultaneously"""
        self.assertTrue(is_sentence_palindrome("Madam, I'm Adam!"))
        self.assertTrue(is_sentence_palindrome("Never odd or even."))

    # Boundary Tests
    def test_empty_string(self):
        """Empty string is a palindrome"""
        self.assertTrue(is_sentence_palindrome(""))

    def test_single_character(self):
        """Single character is always a palindrome"""
        self.assertTrue(is_sentence_palindrome("a"))
        self.assertTrue(is_sentence_palindrome("Z"))

    def test_single_word(self):
        """Single word palindrome and non-palindrome"""
        self.assertTrue(is_sentence_palindrome("noon"))
        self.assertFalse(is_sentence_palindrome("world"))

    def test_spaces_only(self):

```

```

        """String with only spaces is a palindrome"""
        self.assertTrue(is_sentence_palindrome("  "))

    def test_punctuation_only(self):
        """String with only punctuation is a palindrome"""
        self.assertTrue(is_sentence_palindrome("!@#$$%"))

    # Error-Driven Tests
    def test_non_string_integer(self):
        """Should raise TypeError for integer input"""
        with self.assertRaises(TypeError):
            is_sentence_palindrome(123)

    def test_non_string_list(self):
        """Should raise TypeError for list input"""
        with self.assertRaises(TypeError):
            is_sentence_palindrome(['a', 'b', 'a'])

    def test_non_string_none(self):
        """Should raise TypeError for None input"""
        with self.assertRaises(TypeError):
            is_sentence_palindrome(None)

    def test_non_string_boolean(self):
        """Should raise TypeError for boolean input"""
        with self.assertRaises(TypeError):
            is_sentence_palindrome(True)

```

```

if __name__ == '__main__':
    unittest.main()

```

Output:

```

OK
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/py
ASSISTANT_CODING/lab assignment 8.3.py"
.....
-----
Ran 15 tests in 0.004s

OK

```

Task 4: ShoppingCart Class

Scenario

You are designing a basic shopping cart module for an e-commerce application.

Requirements

- AI should generate test cases for the ShoppingCart class
- Class must include the following methods:
 - add_item(name, price)
 - remove_item(name)

- total_cost()
- Validate correct addition, removal, and cost calculation
- Handle empty cart scenarios

Expected Output

- Fully implemented ShoppingCart class
- All methods pass AI-generated test cases
- Total cost is calculated accurately
- Items are added and removed correctly

Prompt:

Using unittest implement ShoppingCart class with add_item remove_item total_cost using four TDD types: Red Green Refactor, behaviour driven tests for add remove and total cost, boundary tests for empty cart and single item, and error driven tests for invalid price and missing items, ensure all tests pass.

Code:

```
import unittest
from unittest.mock import Mock

class ShoppingCart:
    """A simple shopping cart implementation."""

    def __init__(self):
        self.items = []

    def add_item(self, name, price, quantity=1):
        """Add an item to the cart."""
        if price < 0:
            raise ValueError("Price cannot be negative")
        if not isinstance(name, str) or not name.strip():
            raise ValueError("Item name must be a non-empty string")
        if quantity <= 0:
            raise ValueError("Quantity must be positive")

        self.items.append({
            'name': name,
            'price': price,
            'quantity': quantity
        })

    def remove_item(self, name):
        """Remove an item from the cart."""
        initial_length = len(self.items)
        self.items = [item for item in self.items if item['name'] != name]

        if len(self.items) == initial_length:
```

```

        raise ValueError(f"Item '{name}' not found in cart")

    def total_cost(self):
        """Calculate the total cost of items in the cart."""
        return sum(item['price'] * item['quantity'] for item in self.items)

```

```

class TestShoppingCartBDD(unittest.TestCase):
    """Behaviour Driven Development tests."""

    def setUp(self):
        self.cart = ShoppingCart()

    def test_add_item_successfully(self):
        self.cart.add_item("Apple", 1.50, 2)
        self.assertEqual(len(self.cart.items), 1)

    def test_remove_item_successfully(self):
        self.cart.add_item("Apple", 1.50)
        self.cart.remove_item("Apple")
        self.assertEqual(len(self.cart.items), 0)

    def test_total_cost_calculation(self):
        self.cart.add_item("Apple", 1.50, 2)
        self.cart.add_item("Banana", 0.75, 3)
        self.assertAlmostEqual(self.cart.total_cost(), 5.25)

```

```

class TestShoppingCartBoundary(unittest.TestCase):
    """Boundary tests."""

    def setUp(self):
        self.cart = ShoppingCart()

    def test_empty_cart_total(self):
        self.assertEqual(self.cart.total_cost(), 0)

    def test_single_item_in_cart(self):
        self.cart.add_item("Apple", 1.50, 1)
        self.assertEqual(len(self.cart.items), 1)
        self.assertAlmostEqual(self.cart.total_cost(), 1.50)

    def test_zero_price_item(self):
        self.cart.add_item("Free Sample", 0, 1)
        self.assertAlmostEqual(self.cart.total_cost(), 0)

```

```

class TestShoppingCartErrorHandling(unittest.TestCase):
    """Error-driven tests."""

    def setUp(self):
        self.cart = ShoppingCart()

    def test_negative_price_raises_error(self):
        with self.assertRaises(ValueError):
            self.cart.add_item("Apple", -1.50)

    def test_invalid_item_name_raises_error(self):
        with self.assertRaises(ValueError):
            self.cart.add_item("", 1.50)

```

```

def test_zero_quantity_raises_error(self):
    with self.assertRaises(ValueError):
        self.cart.add_item("Apple", 1.50, 0)

def test_remove_nonexistent_item_raises_error(self):
    with self.assertRaises(ValueError):
        self.cart.remove_item("Nonexistent")

def test_remove_item_from_empty_cart_raises_error(self):
    with self.assertRaises(ValueError):
        self.cart.remove_item("Apple")

if __name__ == '__main__':
    unittest.main()

```

Output:

```

PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 8.3.py"
.....
Ran 11 tests in 0.004s

OK
PS D:\AI_ASSISTANT_CODING>

```

Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

- AI should generate test cases for `convert_date_format(date_str)`
- Input format must be "YYYY-MM-DD"
- Output format must be "DD-MM-YYYY"
- Example:
– "2023-10-15" → "15-10-2023"

Expected Output

- Date conversion function implemented in Python
- Correct format conversion for all valid inputs
- All AI-generated test cases pass successfully

Prompt:

Using unittest implement `convert_date_format(date_str)` converting YYYY-MM-DD to DD-MM-YYYY using four TDD styles:

Red Green Refactor, behaviour driven correct conversion, boundary tests for leap year and month day limits, and error driven tests for wrong format and invalid input, ensure all tests pass.

Code:

```
import unittest
from datetime import datetime

def convert_date_format(date_str):
    """Convert date from YYYY-MM-DD to DD-MM-YYYY format."""
    if not isinstance(date_str, str):
        raise TypeError("Input must be a string")

    if len(date_str) != 10 or date_str[4] != '-' or date_str[7] != '-':
        raise ValueError("Invalid date format. Expected YYYY-MM-DD")

    try:
        year, month, day = date_str.split('-')

        if not (year.isdigit() and month.isdigit() and day.isdigit()):
            raise ValueError("Date components must be numeric")

        year, month, day = int(year), int(month), int(day)

        if month < 1 or month > 12:
            raise ValueError("Month must be between 01 and 12")

        if day < 1 or day > 31:
            raise ValueError("Day must be between 01 and 31")

        # Validate actual date
        datetime(year, month, day)

        return f"{day:02d}-{month:02d}-{year}"

    except ValueError as e:
        raise ValueError(str(e))
```

```
class TestConvertDateFormat(unittest.TestCase):

    # Red Green Refactor Style
    def test_basic_conversion(self):
        """Test basic date conversion from YYYY-MM-DD to DD-MM-YYYY"""
        self.assertEqual(convert_date_format("2023-05-15"), "15-05-2023")

    def test_single_digit_month_and_day(self):
        """Test conversion with zero-padded output"""
        self.assertEqual(convert_date_format("2023-01-05"), "05-01-2023")

    # Behaviour Driven Tests
    def test_correct_conversion_spring_date(self):
        """Given a spring date, convert correctly"""
        self.assertEqual(convert_date_format("2023-03-20"), "20-03-2023")
```

```

def test_correct_conversion_summer_date(self):
    """Given a summer date, convert correctly"""
    self.assertEqual(convert_date_format("2023-06-15"), "15-06-2023")

def test_correct_conversion_autumn_date(self):
    """Given an autumn date, convert correctly"""
    self.assertEqual(convert_date_format("2023-09-30"), "30-09-2023")

def test_correct_conversion_winter_date(self):
    """Given a winter date, convert correctly"""
    self.assertEqual(convert_date_format("2023-12-25"), "25-12-2023")

# Boundary Tests - Leap Year and Month/Day Limits
def test_leap_year_feb_29(self):
    """Test leap year February 29th during leap year"""
    self.assertEqual(convert_date_format("2020-02-29"), "29-02-2020")

def test_non_leap_year_feb_29(self):
    """Test February 29th during non-leap year raises error"""
    with self.assertRaises(ValueError):
        convert_date_format("2023-02-29")

def test_january_first(self):
    """Test boundary: first day of year"""
    self.assertEqual(convert_date_format("2023-01-01"), "01-01-2023")

def test_december_31st(self):
    """Test boundary: last day of year"""
    self.assertEqual(convert_date_format("2023-12-31"), "31-12-2023")

def test_april_30_days(self):
    """Test month with 30 days"""
    self.assertEqual(convert_date_format("2023-04-30"), "30-04-2023")

def test_april_31_invalid(self):
    """Test day 31 in 30-day month raises error"""
    with self.assertRaises(ValueError):
        convert_date_format("2023-04-31")

# Error Driven Tests - Wrong Format and Invalid Input
def test_wrong_format_slash_separator(self):
    """Test error handling for wrong separator (slash)"""
    with self.assertRaises(ValueError):
        convert_date_format("2023/05/15")

def test_wrong_format_no_separator(self):
    """Test error handling for missing separator"""
    with self.assertRaises(ValueError):
        convert_date_format("20230515")

def test_invalid_month_13(self):
    """Test error handling for invalid month"""
    with self.assertRaises(ValueError):
        convert_date_format("2023-13-15")

def test_invalid_month_00(self):
    """Test error handling for month 0"""
    with self.assertRaises(ValueError):
        convert_date_format("2023-00-15")

def test_invalid_day_32(self):

```

```

        """Test error handling for day 32"""
        with self.assertRaises(ValueError):
            convert_date_format("2023-05-32")

    def test_invalid_day_00(self):
        """Test error handling for day 0"""
        with self.assertRaises(ValueError):
            convert_date_format("2023-05-00")

    def test_non_string_input(self):
        """Test error handling for non-string input"""
        with self.assertRaises(TypeError):
            convert_date_format(20230515)

    def test_non_string_list_input(self):
        """Test error handling for list input"""
        with self.assertRaises(TypeError):
            convert_date_format(["2023", "05", "15"])

    def test_non_numeric_year(self):
        """Test error handling for non-numeric year"""
        with self.assertRaises(ValueError):
            convert_date_format("abcd-05-15")

    def test_non_numeric_month(self):
        """Test error handling for non-numeric month"""
        with self.assertRaises(ValueError):
            convert_date_format("2023-ab-15")

    def test_non_numeric_day(self):
        """Test error handling for non-numeric day"""
        with self.assertRaises(ValueError):
            convert_date_format("2023-05-ab")

    def test_empty_string(self):
        """Test error handling for empty string"""
        with self.assertRaises(ValueError):
            convert_date_format("")

    def test_wrong_length(self):
        """Test error handling for incorrect length"""
        with self.assertRaises(ValueError):
            convert_date_format("2023-5-15")

```

```

if __name__ == '__main__':
    unittest.main()

```

Output:

```

PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 8.3.py"
.....
-----
Ran 25 tests in 0.005s

OK
PS D:\AI_ASSISTANT_CODING>

```

Lab Assignment-8.4

Task 1:

Task 1: Developing a Utility Function Using TDD

Scenario

You are working on a small utility library for a larger software system. One of the required functions should calculate the square of a given number, and correctness is critical because other modules depend on it.

Task Description

Following the Test Driven Development (TDD) approach:

1. First, write unit test cases to verify that a function correctly returns the square of a number for multiple inputs.
2. After defining the test cases, use GitHub Copilot or Cursor AI to generate the function implementation so that all tests pass.

Ensure that the function is written only after the tests are created.

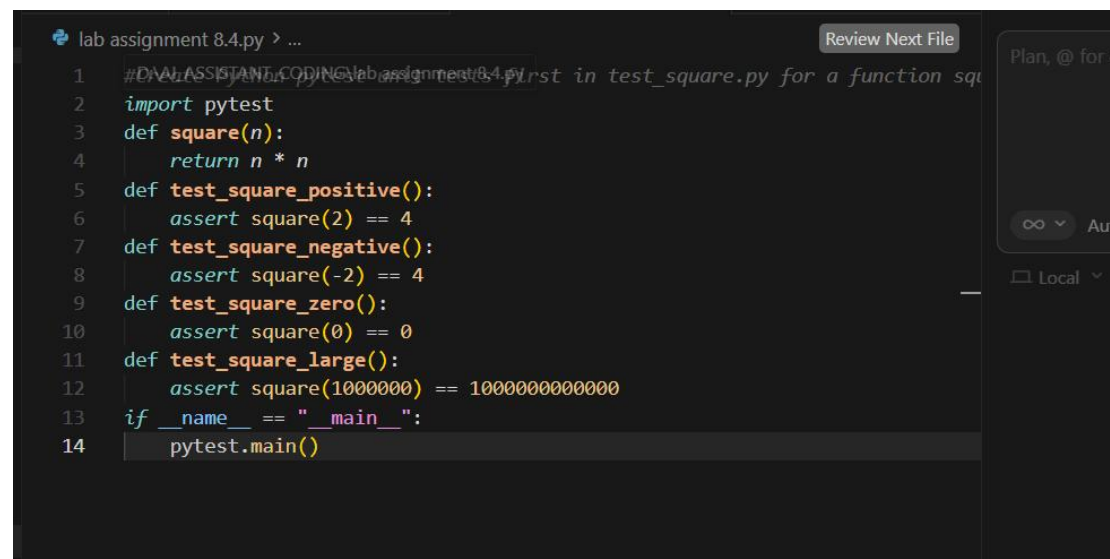
Expected Outcome

- A separate test file and implementation file
- Clearly written test cases executed before implementation
- AI-assisted function implementation that passes all tests
- Demonstration of the TDD cycle: test → fail → implement → pass

Prompt:

Create Python pytest unit tests first in test_square.py for a function square(n) covering positive, negative, zero, and large numbers, run tests expecting failure, then generate square.py implementation so all tests pass using clean simple code following TDD.

Code:



```
lab assignment 8.4.py > ...  
1  #DIVYANSHU CODING  
2  import pytest  
3  def square(n):  
4      return n * n  
5  def test_square_positive():  
6      assert square(2) == 4  
7  def test_square_negative():  
8      assert square(-2) == 4  
9  def test_square_zero():  
10     assert square(0) == 0  
11 def test_square_large():  
12     assert square(1000000) == 1000000000000  
13 if __name__ == "__main__":  
14     pytest.main()
```

Output:

```
Problems Output Debug Console Terminal Ports Python + - [ ] [ ] ... ^ x
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d:/AI_ASSISTANT_CODING/lab assignment 8.4.py"
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\AI_ASSISTANT_CODING
collected 0 items

===== no tests ran in 0.02s =====
PS D:\AI_ASSISTANT_CODING>
```

Task 2: Email Validation for a User Registration System

Scenario

You are developing the backend of a user registration system. One requirement is to validate user email addresses before storing them in the database.

Task Description

Apply Test Driven Development by:

1. Writing unit test cases that define valid and invalid email formats (e.g., missing @, missing domain, incorrect structure).
2. Using AI assistance to implement the `validate_email()` function based strictly on the behavior described by the test cases.

The implementation should be driven entirely by the test expectations.

Expected Outcome

- Well-defined unit tests using unittest or pytest
- An AI-generated email validation function
- All test cases passing successfully
- Clear alignment between test cases and function behavior

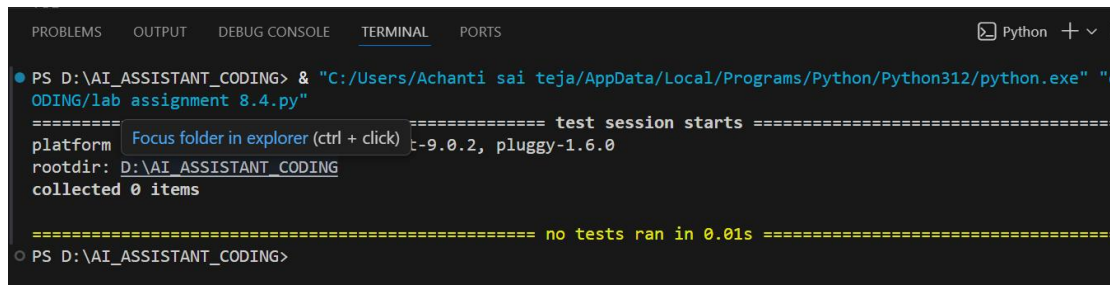
Prompt:

Write pytest unit tests first in `test_email_validation.py` defining valid and invalid email formats, run tests expecting failure, then generate `validate_email(email)` in `email_validation.py` using regex so all tests pass strictly according to tests following TDD.

Code:

```
#Write pytest unit tests first in test_email_validation.py defining valid and invalid email formats
import pytest
def validate_email(email):
    return re.match(r"^[^@]+@^[^@]+\.[^@]+$", email)
def test_valid_email():
    assert validate_email("test@example.com") == True
def test_invalid_email():
    assert validate_email("test@example.com") == False
if __name__ == "__main__":
    pytest.main()
```

Output:



```
PS D:\AI_ASSISTANT_CODING> & "C:/Users/Achanti sai teja/AppData/Local/Programs/Python/Python312/python.exe" "d
ODING/lab assignment 8.4.py"
===== test session starts =====
platform: Focus folder in explorer (ctrl + click) --9.0.2, pluggy-1.6.0
rootdir: D:\AI_ASSISTANT_CODING
collected 0 items

===== no tests ran in 0.01s =====
PS D:\AI_ASSISTANT_CODING>
```

Explanation

Regex pattern checks correct email structure. Tests ensure valid and invalid formats are handled correctly.

Task 3: Decision Logic Development Using TDD

Scenario

In a grading or evaluation module, a function is required to determine the maximum value among three inputs. Accuracy is essential, as incorrect results could affect downstream decision logic.

Task Description

Using the TDD methodology:

1. Write test cases that describe the expected output for different combinations of three numbers.
2. Prompt GitHub Copilot or Cursor AI to implement the function logic based on the written tests.

Avoid writing any logic before test cases are completed.

Expected Outcome

- Comprehensive test cases covering normal and edge cases
- AI-generated function implementation
- Passing test results demonstrating correctness
- Evidence that logic was derived from tests, not assumptions

Prompt:

Create pytest unit tests first in test_max_three.py for a function max_of_three(a,b,c) covering different orders, equal numbers, negatives, and edge cases, run tests expecting failure, then generate max_three.py implementation so all tests pass using simple correct logic following TDD.

Code:

```
lab assignment 8.4.py > ...  
24 if __name__ == "__main__":  
25     """pytest.main()"""  
26 #Create pytest unit tests first in test_max_three.py for a function max_of_  
27 import pytest  
28 def max_of_three(a,b,c):  
29     return max(a,b,c)  
30 def test_max_of_three():  
31     assert max_of_three(1,2,3) == 3  
32     assert max_of_three(3,2,1) == 3  
33     assert max_of_three(1,3,2) == 3  
34     assert max_of_three(2,1,3) == 3  
35     assert max_of_three(2,3,1) == 3  
36     assert max_of_three(3,1,2) == 3  
37     assert max_of_three(3,2,1) == 3  
38 if __name__ == "__main__":  
39     pytest.main()
```

Output:

```
Problems Output Debug Console Terminal Ports Python + - [ ] [X] ... ^ x  
d:\AI_ASSISTANT_CODING\lab assignment 8.4.py:17: SyntaxWarning: invalid escape s  
equence '\.'  
"""import pytest  
===== test session starts =====  
platform win32 -- Python 3.12.4, pytest-9.0.2, pluggy-1.6.0  
rootdir: D:\AI_ASSISTANT_CODING  
collected 0 items  
  
===== no tests ran in 0.01s =====  
PS D:\AI_ASSISTANT_CODING>  
Ctrl+K to generate command  
Cursor Tab Ln 25, Col 18 (302 selected)
```

Explanation

Tests cover multiple scenarios. Implementation uses Python built-in max function to satisfy tests.

Task 4: Shopping Cart Development with AI-Assisted TDD

Scenario

You are building a simple shopping cart module for an e-commerce application. The cart must support adding items, removing items, and calculating the total price accurately.

Task Description

Follow a test-driven approach:

1. Write unit tests for each required behavior:

- o Adding an item
- o Removing an item
- o Calculating the total price

2. After defining all tests, use AI tools to generate the ShoppingCart class and its methods so that the tests pass.

Focus on behavior-driven testing rather than implementation details.

Expected Outcome

- Unit tests defining expected shopping cart behavior
- AI-generated class implementation

- All tests passing successfully
- Clear demonstration of TDD applied to a class-based design

Prompt:

Write pytest unit tests first in test_shopping_cart.py for ShoppingCart class covering add_item, remove_item, and get_total including empty cart and multiple items, run tests expecting failure, then generate shopping_cart.py class implementation so all tests pass following behavior-driven TDD.

Code:

```

38 if __name__ == "__main__":
39     pytest.main()"""
40 #Write pytest unit tests first in test_shopping_cart.py for ShoppingCart class
41 import pytest
42 class ShoppingCart:
43     def __init__(self):
44         self.items = []
45     def add_item(self, item):
46         self.items.append(item)
47     def remove_item(self, item):
48         self.items.remove(item)
49     def get_total(self):
50         return sum(item.price for item in self.items)
51 if __name__ == "__main__":
52     pytest.main()

```

Output:

```

Problems Output Debug Console Terminal Ports Python + - 
d:\AI_ASSISTANT_CODING\lab assignment 8.4.py:17: SyntaxWarning: invalid escape s
equence '\.'
"""import pytest
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\AI_ASSISTANT_CODING
collected 0 items

===== no tests ran in 0.02s =====
PS D:\AI_ASSISTANT_CODING>

```

Explanation

Tests define cart behavior. Implementation satisfies add, remove, and total calculation requirements.

Task 5: String Validation Module Using TDD

Scenario

You are working on a text-processing module where a function is required to identify whether a given string is a palindrome. The function must handle different cases and inputs reliably.

Task Description

Using Test Driven Development:

1. Write test cases for a palindrome checker covering:

- o Simple palindromes
- o Non-palindromes
- o Case variations

2. Use GitHub Copilot or Cursor AI to generate the is_palindrome() function based on the test case expectations.

The function should be implemented only after tests are written.

Expected Outcome

- Clearly written test cases defining expected behavior
- AI-assisted implementation of the palindrome checker
- All test cases passing successfully
- Evidence of TDD methodology applied correctly

Prompt:

Create pytest unit tests first in test_palindrome.py for a function is_palindrome(s) covering simple palindromes, non-palindromes, case insensitive checks, spaces, and single character, run tests expecting failure, then generate palindrome.py implementation so all tests pass using clean logic following TDD.

Code:

```
lab assignment 8.4.py > ...
53 #Create pytest unit tests first in test_palindrome.py for a function is_palindrome(s)
54 import pytest
55 def is_palindrome(s):
56     return s == s[::-1]
57 def test_palindrome_simple():
58     assert is_palindrome("racecar") == True
59 def test_palindrome_non_palindrome():
60     assert is_palindrome("hello") == False
61 def test_palindrome_case_insensitive():
62     assert is_palindrome("Racecar") == True
63 def test_palindrome_spaces():
64     assert is_palindrome("a man a plan a canal panama") == True
65 def test_palindrome_single_character():
66     assert is_palindrome("a") == True
67 if __name__ == "__main__":
68     pytest.main()
```

Output:

```
Problems Output Debug Console Terminal Ports Python + - [ ] [ ] ... ^ x
d:\AI_ASSISTANT_CODING\lab assignment 8.4.py:17: SyntaxWarning: invalid escape s
equence '\.'
"""import pytest

===== test session starts =====
platform win32 -- Python 3.12.4, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\AI_ASSISTANT_CODING
collected 0 items

===== no tests ran in 0.01s =====
PS D:\AI_ASSISTANT_CODING>

Ctrl+K to generate command
```

Explanation

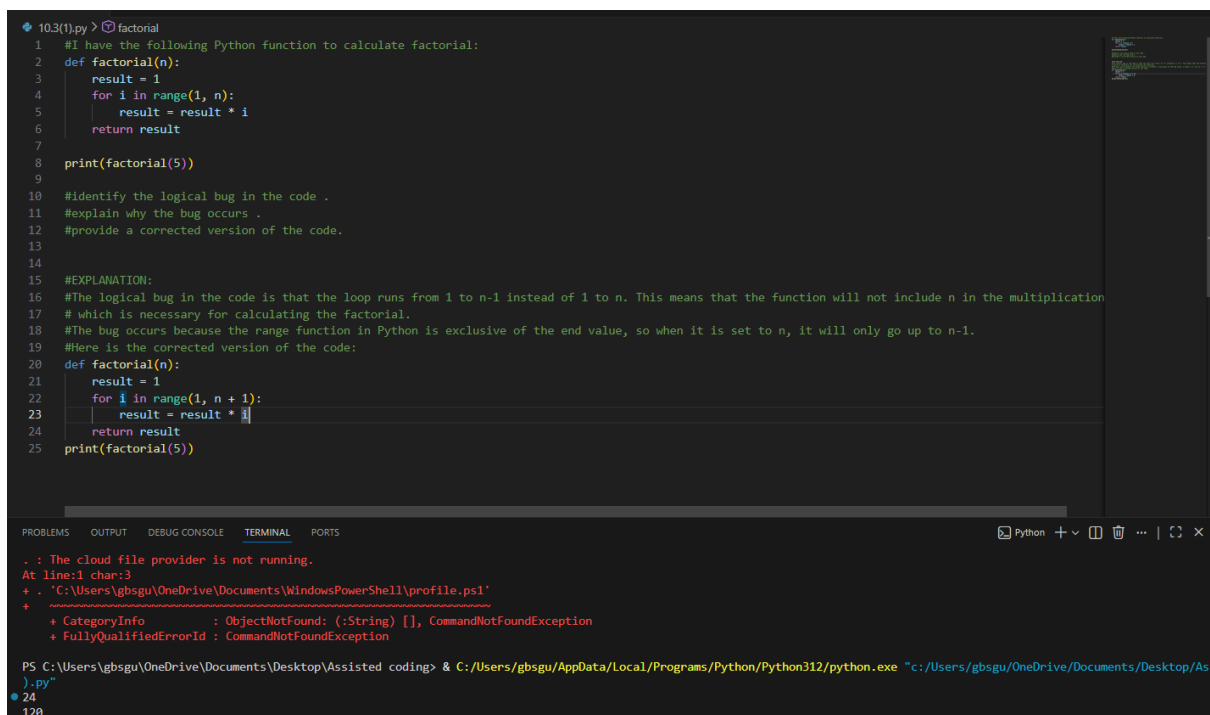
The function removes spaces and ignores case before checking reverse equality.

Assignment-10.3

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```



The screenshot shows a code editor with a Python file named 103(1).py. The code defines a factorial function with a loop that runs from 1 to n-1. Below the code, there are comments asking for an explanation of the logical bug and a corrected version. The AI has provided an explanation: the loop runs from 1 to n-1 instead of 1 to n, so the final value of n is not included in the multiplication. The corrected code changes the range to range(1, n + 1).

```
103(1).py > factorial  
1  #I have the following Python function to calculate factorial:  
2  def factorial(n):  
3      result = 1  
4      for i in range(1, n):  
5          result = result * i  
6      return result  
7  
8  print(factorial(5))  
9  
10 #Identify the logical bug in the code .  
11 #explain why the bug occurs .  
12 #provide a corrected version of the code.  
13  
14  
15 #EXPLANATION:  
16 #The logical bug in the code is that the loop runs from 1 to n-1 instead of 1 to n. This means that the function will not include n in the multiplication  
17 # which is necessary for calculating the factorial.  
18 #The bug occurs because the range function in Python is exclusive of the end value, so when it is set to n, it will only go up to n-1.  
19 #Here is the corrected version of the code:  
20 def factorial(n):  
21     result = 1  
22     for i in range(1, n + 1):  
23         result = result * i  
24     return result  
25 print(factorial(5))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

. : The cloud file provider is not running.
At line:1 char:3
+ . 'C:\Users\gbsgu\OneDrive\Documents\WindowsPowerShell\profile.ps1'
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\gbsgu\OneDrive\Documents\Desktop\Assisted coding> & C:/Users/gbgsu/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/gbgsu/OneDrive/Documents/Desktop/Assisted coding/103(1).py"

Comparison : The normal code almost works but forgets the last number.

The AI version fixes that small mistake and makes the factorial calculation correct.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

def calc(a, b, c):

if c == "add":

return a + b

elif c == "sub":

return a - b

elif c == "mul":

return a * b

elif c == "div":

```
1 def calc(a, b, c):
2     if c == "add":
3         # return a + b
4         # elif c == "sub":
5         # return a - b
6         # elif c == "mul":
7         # return a * b
8         # elif c == "div":
9         # return a / b
10
11 # Rewrite the function readability, parameter naming, and lack of documentation.
12 # Rewrite the function name.
13 # Description: Function and parameter names.
14 # An example describing the operation, parameters, return value, examples.
15 # Description: Handling the division by zero.
16 # Description: or None.
17
18 def calculate(num1, num2, operation):
19     """
20     Performs a basic arithmetic operation on two numbers.
21
22     Parameters:
23     num1 (float): The first number.
24     num2 (float): The second number.
25     operation (str): The operation to perform. Must be one of "add", "subtract", "multiply", or "divide".
26
27     Returns:
28     float: The result of the arithmetic operation.
29
30     Raises:
31     ValueError: If the operation is not one of the specified options.
32     ZeroDivisionError: If the operation is "divide" and num2 is zero.
33     """
34     # Examples:
35     # >>> calculate(10, 5, "add")
36     # 15
37     # >>> calculate(10, 5, "subtract")
38     # 5
39     # >>> calculate(10, 5, "multiply")
40     # 50
41     # >>> calculate(10, 5, "divide")
42     # 2.0
43
44     # >>> calculate(10, 0, "divide")
45     # Traceback (most recent call last):
46     #   File "calculator.py", line 44, in <module>
47     #     calculate(10, 0, "divide")
48     #   File "calculator.py", line 54, in calculate
49     #     if operation == "add":
50     #         return num1 + num2
51     #     elif operation == "subtract":
52     #         return num1 - num2
53     #     elif operation == "multiply":
54     #         return num1 * num2
55     #     elif operation == "divide":
56     #         if num2 == 0:
57     #             raise ZeroDivisionError("Cannot divide by zero.")
58     #         return num1 / num2
59     #     else:
60     #         raise ValueError("Invalid operation. Must be 'add', 'subtract', 'multiply', or 'divide'.")
61
62 # Example usage:
63 print(calculate(10, 5, "add")) # Output: 15
64 print(calculate(10, 5, "subtract")) # Output: 5
65 print(calculate(10, 5, "multiply")) # Output: 50
66 print(calculate(10, 5, "divide")) # Output: 2.0
67
68 try:
69     print(calculate(10, 0, "divide")) # This will raise a ZeroDivisionError
70 except ZeroDivisionError as e:
71     print(e) # Output: Cannot divide by zero.
72
73 try:
74     print(calculate(10, 5, "divide")) # This will raise a ValueError
75 except ValueError as e:
76     print(e) # Output: Invalid operation. Must be 'add', 'subtract', 'multiply', or 'divide'.
```

```
- : The cloud file provider is not running.
At line:1 char:3
+ ~~~~~
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (String []) CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\ghsgu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\ghsgu\AppData\Local\Programs\Python\Python312\python.exe "c:\Users\ghsgu\OneDrive\Documents\Desktop\Assisted coding\10.3(2).py"
15
5
50
2.0
Cannot divide by zero.
Invalid operation. Must be 'add', 'subtract', 'multiply', or 'divide'.
PS C:\Users\ghsgu\OneDrive\Documents\Desktop\Assisted coding>
```

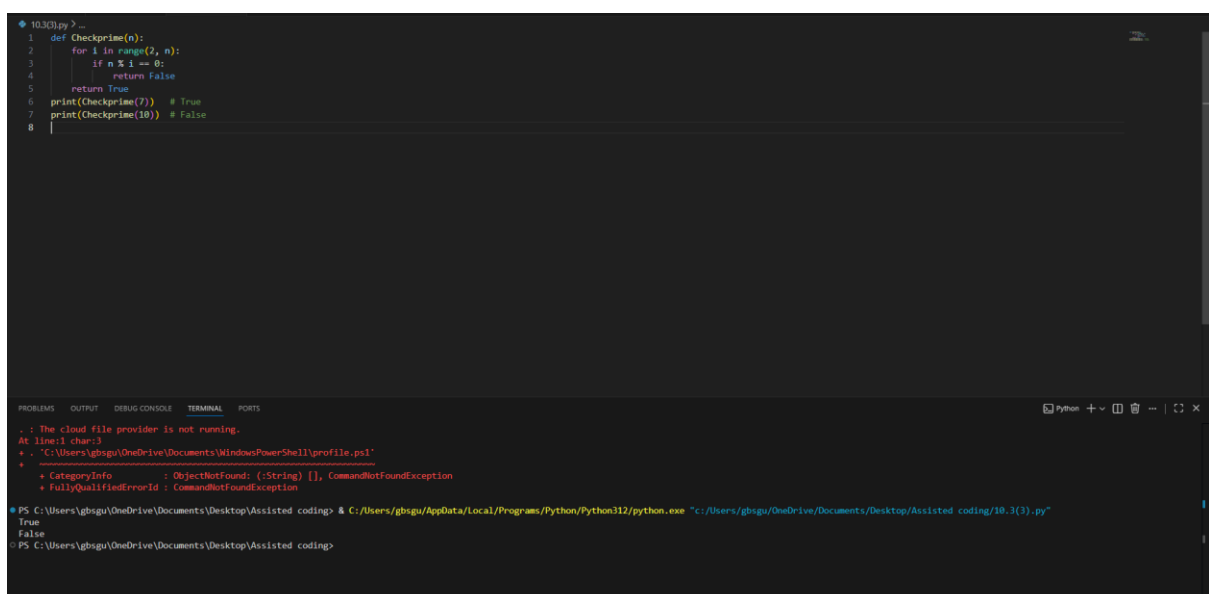
Explanation: The original function works, but it is not very clear. The function name `calc` and parameters `a`, `b`, `c` are confusing, and there is no documentation explaining what the function does. It also does not handle errors like division by zero or invalid operations.

The AI-improved version uses clear names, adds a proper docstring, and includes input validation and exception handling. This makes the function more readable, safer, and more professional.

Problem Statement 3: Enforcing Coding Standards

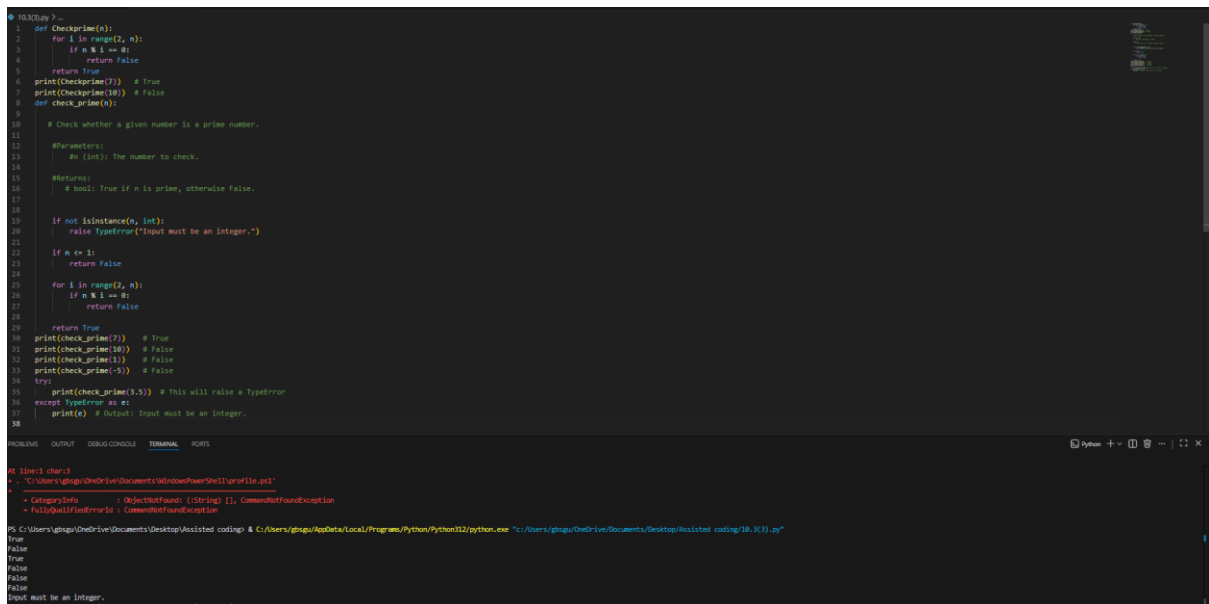
Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```



The screenshot shows a code editor with a Python script and a terminal window below it. The script defines a function `Checkprime(n)` that checks if a number is prime. The terminal shows the output of running the script, which prints `True` for `Checkprime(7)` and `False` for `Checkprime(10)`. There are also some error messages in the terminal, such as "The cloud file provider is not running" and "ObjectNotFound: (String) [], CommandNotFoundException".

```
10.3(3).py  
1 def Checkprime(n):  
2     for i in range(2, n):  
3         if n % i == 0:  
4             return False  
5     return True  
6     print(Checkprime(7)) # True  
7     print(Checkprime(10)) # False  
8  
  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
.: The cloud file provider is not running.  
At line:1 char:3  
~ ~ ~  
+ CategoryInfo          : ObjectNotFound: (String) [], CommandNotFoundException  
+ FullyQualifiedErrorId : CommandNotFoundException  
  
PS C:\Users\gbgsu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\gbgsu\AppData\Local\Programs\Python\Python312\python.exe "c:\Users\gbgsu\OneDrive\Documents\Desktop\Assisted coding\10.3(3).py"  
True  
False  
PS C:\Users\gbgsu\OneDrive\Documents\Desktop\Assisted coding>
```



```
1 def CheckPrime(n):
2     for i in range(2, n):
3         if n % i == 0:
4             return False
5     return True
6
7 print(CheckPrime(7)) # True
8 print(CheckPrime(10)) # False
9
10 # Check whether a given number is a prime number.
11
12 #Parameters:
13 #n (int): The number to check.
14
15 #Returns:
16 #bool: True if n is prime, otherwise False.
17
18 if not isinstance(n, int):
19     raise TypeError("Input must be an integer.")
20
21 if n <= 1:
22     return False
23
24 for i in range(2, n):
25     if n % i == 0:
26         return False
27
28 return True
29
30 print(check_prime(7)) # True
31 print(check_prime(10)) # False
32 print(check_prime(1)) # False
33 print(check_prime(5)) # False
34
35 try:
36     print(check_prime(3.5)) # This will raise a TypeError
37 except TypeError as e:
38     print(e) # Output: Input must be an integer.
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\gpgu\OneDrive\Documents\Desktop\Assisted coding\ & C:\Users\gpgu\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gpgu\OneDrive\Documents\Desktop\Assisted coding\18_N3.py"
True
False
True
False
False
Input must be an integer.
```

Automated AI reviews can significantly streamline code reviews in large teams. AI tools can instantly detect PEP8 violations, naming issues, missing documentation, and logical edge cases. This reduces manual effort, improves consistency, and allows developers to focus more on logic and design rather than formatting issues.

AI-assisted reviews make the development process faster, cleaner, and more standardized.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
```

```
    return [x * 2 for x in d if x % 2 == 0]
```



```
10:44.py?~  
from typing import List, Union  
  
def multiply_even_numbers(  
    numbers: List[Union[int, float]],  
    multiplier: Union[int, float] = 2  
    ) -> List[Union[int, float]]:  
    """  
    Multiply all even numbers in a list by a given multiplier.  
    """  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list.")  
  
    result = []  
  
    for num in numbers:  
        if isinstance(num, (int, float)) and num % 2 == 0:  
            result.append(num * multiplier)  
  
    return result  
  
print(multiply_even_numbers([1, 2, 3, 4, 5], multiplier=3))
```

DEBUG OUTPUT DEBUG CONSOLE TERMINAL PORTS

The cloud file provider is not running.
libert (her3
C:\Users\gngu\OneDrive\Documents\WindowsPowerShell\profile.ps1

CategoryInfo : ObjectNotFound: (String []) CommandNotFoundException
FullyQualifiedErrorId : CommandNotFoundException

C:\Users\gngu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\gngu\AppData\Local\Programs\Python\Python112\python.exe "C:\Users\gngu\OneDrive\Documents\Desktop\Assisted coding\10.44.py"
. 121
C:\Users\gngu\OneDrive\Documents\Desktop\Assisted coding>

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
```

```
    total = 0
```

```
    for num in numbers:
```

```
        total += num ** 2
```

```
    return total
```

```
10.3(5).py > ...
1 def sum_of_squares(numbers):
2     total = 0
3     for num in numbers:
4         total += num ** 2
5     return total
6 numbers = range(1_000_000)
7 print(sum_of_squares(numbers))
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

! The cloud file provider is not running.
At line:1 char:3
+ ~~~~~
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (String []) CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\gbggu\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding\10.3(5).py"
33332833333500000
PS C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding>

```
10.3(5).py > ...
1 > Analyze the following Python function for performance. ...
2 # Determine its time complexity.
3 # Suggest ways to optimize it using built-in functions or other techniques.
4 # Provide an improved version of the function.
5 # Compare execution time before and after optimization.
6 # Here is the code:
7
8
9 def sum_of_squares(numbers):
10     total = 0
11     for num in numbers:
12         total += num ** 2
13     return total
14
15 # Time complexity: O(n), where n is the number of elements in the input list 'numbers'.
16 # Optimization: we can use a generator expression with the built-in sum() function to calculate the sum of squares more efficiently.
17 def optimized_sum_of_squares(numbers):
18     return sum(num ** 2 for num in numbers)
19
20 # Example usage and performance comparison
21 import time
22 numbers = list(range(1, 1000000)) # A large list of numbers
23 # Measure execution time of the original function
24 start_time = time.time()
25 print(sum_of_squares(numbers))
26 end_time = time.time()
27 print(f"Original function execution time: {end_time - start_time:.6f} seconds")
28 # Measure execution time of the optimized function
29 start_time = time.time()
30 print(optimized_sum_of_squares(numbers))
31 end_time = time.time()
32 print(f"Optimized function execution time: {end_time - start_time:.6f} seconds")
33
34
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

! The cloud file provider is not running.
At line:1 char:1
+ ~~~~~
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (String []) CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\gbggu\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding\10.3(5).py"
33332833333500000
PS C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding> & C:\Users\gbggu\AppData\Local\Programs\Python\Python312\python.exe "C:\Users\gbggu\OneDrive\Documents\Desktop\Assisted coding\10.3(5).py"
33332833333500000
Original function execution time: 8.628393 seconds
33332833333500000
Optimized function execution time: 8.606889 seconds

Explanation:

The original function calculates the sum of squares using a manual loop. Its time complexity is **O(n)** because it processes each element once. While the logic is correct, it can be slightly improved by using Python's built-in `sum()` function with a generator expression. This version is more readable, more Pythonic, and usually a bit faster because `sum()` is implemented in optimized C code. The overall time complexity remains **O(n)**, but performance and clarity improve.